

APX GUIDE

Questions

ASSESSMENT TEST

Acerca de esta guía: Este documento es una guía para el examen de APX, está inspirada en un documento encontrado en línea, otorgando el crédito correspondiente a quien corresponda. El examen real consta de 40 preguntas y tiene una duración de dos horas. El documento se divide principalmente en tres partes; **Assessment Test:** La evaluación diagnóstica inicial. Tras cada pregunta en esta sección hay un botón ans que lleva directamente a su respuesta correspondiente, **Answers and Justifications:** El apartado de soluciones detalladas. De forma dual, cada elemento cuenta con un botón BACK que permite regresar a la pregunta respectiva y **Classification of Questions:** Una clasificación temática sugerida de las preguntas para facilitar el estudio por áreas específicas de conocimiento. A esta sección se puede acceder directamente mediante el botón en forma de › ubicado en la esquina inferior izquierda de cada página.

Cualquier error, comentario o sugerencia respecto a este material puede ser comunicado directamente a mi correo electrónico: `christian.a.solis@accenture.com`.

1. **¿Cuáles son los tipos de excepciones en Java?**
   A. Throwable
   B. Object
   C. Checked
   D. Unchecked
   E. Todas las opciones son correctas.

   ans

2. **¿Cuáles de los siguientes comandos de Git se utilizan para gestionar ramas en un repositorio? Selecciona todas las que correspondan.**
   A. git branch
   B. git merge
   C. git init
   D. git checkout
   E. git commit

   ans

3. **En el contexto del versionado semántico (Semantic Versioning), si un desarrollador cambia la versión de una biblioteca de 1.4.3 a 2.0.0 ¿Qué implicaciones tiene este cambio según las reglas del versionado semántico?**
   A. Señala un cambio en el nivel de parche, resolviendo errores sin agregar nuevas funcionalidades.
   B. Ninguna opción es correcta.
   C. Marca la adición de nuevas funcionalidades manteniendo la compatibilidad hacia atrás.
   D. Representa cambios importantes o incompatibles con versiones anteriores, que pueden afectar la compatibilidad.
   E. Indica una mejora menor que no afecta la compatibilidad hacia atrás.

   ans

4. **¿Cuál de los siguientes componentes es parte de una solicitud HTTP?**
   A. Headers, scripts, funciones.
   B. URL, cookies, archivos adjuntos.
   C. Encabezados de la respuesta, cuerpo de la respuesta, estado de la respuesta.
   D. URL, headers, cuerpo de la solicitud.

   ans

5. **¿Cuál de las siguientes opciones describe correctamente un servicio web (web service)?**
   A. Una aplicación o componente que se comunica con otras aplicaciones a través de la red utilizando protocolos estándar.
   B. Un servicio que se utiliza exclusivamente para la transferencia de archivos grandes entre servidores.
   C. Una interfaz gráfica que permite a los usuarios interactuar con una aplicación web.
   D. Un servicio que se ejecuta localmente en el dispositivo del usuario sin necesidad de conectarse a una red.

   ans

6. **¿Cuál de las siguientes afirmaciones sobre el ciclo `for` mejorado introducido en Java 5 es verdadera?**
   A. El ciclo `for` mejorado solo se puede utilizar con matrices.
   B. El ciclo `for` mejorado se puede utilizar con cualquier tipo de colección y matriz.
   C. El ciclo `for` mejorado permite la modificación de los elementos de la colección durante la iteración.
   D. El ciclo `for` mejorado no requiere que la colección implemente la interfaz Iterable.

   ans

7. **¿Cuál de las siguientes características es fundamental en una base de datos relacional?**
   A. Almacenamiento de datos en formato JSON.
   B. Ninguna opción es correcta.
   C. Utilización de nodos y relaciones para representar datos.
   D. Organización de datos en tablas con filas y columnas.
   E. Almacenamiento de datos en un sistema de archivos distribuido.

   ans

8. **¿Cuál es el archivo utilizado para agregar la dependencia del JDBC Utility a una aplicación?**
   A. config.xml
   B. config.yaml
   C. pom.xml
   D. pom.yaml
   E. settings.xml

   ans

9. **Given:**

```
    public class Test {
    int[] array = {1, 2, 3, 4, 5};
    public static void main(String[] args) {
        new Test().go(1);
    }
    void go(int x) {
        array[x % 5] = x++;
        go(x);
    }
    }
```

What is the result?
A. A java.lang.ArrayIndexOutOfBoundsException is thrown.
B. A java.lang.IllegalStateException is thrown.
C. A java.lang.StackOverflowError is thrown.
D. The program runs with no output.
E. Compilation fails

ans

10. **Given:**

```
class Parent {
  private void display() {
    System.out.println("Parent");
  }
}
class Child extends Parent {
  public void display() {
    System.out.println("Child");
  }
}
public class Test {
  public static void main(String[] args) {
    Parent obj = new Child();
    ((Child) obj).display();
  }
}
```

What is the result?
A. Child
B. Parent
C. Compilation error
D. Runtime error

ans

11. **¿Cuál de las siguientes sentencias acerca del patrón de diseño Singleton es verdadera?**
A. El patrón Singleton es usado principalmente para facilitar la herencia y el polimorfismo en la programación orientada a objetos.
B. El patrón Singleton permite que una clase sea instanciada múltiples veces y provee un punto global de acceso a la última instancia creada.
C. El patrón Singleton es usado para crear una colección de objetos que pueden ser accedidos globalmente.
D. El patrón Singleton asegura que una clase tiene sólo una instancia y provee un punto global de acceso a ella.

ans

12. **What will be the result of the following code execution involving streams?**

```
    import java.util.Arrays;
    import java.util.List;
    public class Test {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        long count = numbers.stream()
                            .map(n -> n * 2)
                            .filter(n -> n > 5)
                            .count();
        System.out.println(count);
    }
    }
```

A. 0
B. 2
C. 3
D. 4
E. Compilation fails.

ans

13. **¿Qué comando permite enviar una solicitud de transacción a un servidor APX?**
A. `apx show in -project path=./TESTT001-01-ES`
B. `apx add req -name="TESTT001-01-ES-test.json" -y -details`
C. `apx send req -name="TESTT001-01-ES-test.json" -y -details`
D. `apx check -repair`

ans

14. **Given:**

```
    public class Test {
    public static void main(String... arguments) {
        final char a = 'A', d = 'D';
        char grade = 'B';
        switch (grade) {
        case a:
        case 'B': System.out.print("B");
        case 'C': System.out.print("C");
        case d:
        case 'F': System.out.print("F");
        }
    }
    }
```

What is the result?
A. A
B. BF
C. Compilation fails
D. C
E. ACF
F. BCF
G. B

ans

15. **What will be the output of the following code snippet?**

```
class StaticNonStaticBlockTest {
  static {
    System.out.println("Static block");
  }
  {
    System.out.println("Instance block");
  }
  StaticNonStaticBlockTest() {
    System.out.println("Constructor");
  }
  public static void staticMethod() {
    System.out.println("Static method");
  }
  public static void main(String[] args) {
    StaticNonStaticBlockTest.staticMethod();
    new StaticNonStaticBlockTest();
  }
}
```

A. Static method Static block Instance block Constructor
B. Static block Instance block Constructor Static method
C. Static block Static method Instance block Constructor
D. Compilation fails.

ans

16. **¿Cual es la consola para la configuración de la aplicación en APX Online?**
    A. Chrome Console
    B. APX Operation Console
    C. Apache Felix Online Console
    D. Builder Console
    E. JBoss Console

ans

17. **Given:**

```
1  public class StringInternExample {
2    public static void main(String[] args) {
3      String str1 = new String("example");
4      String str2 = "example";
5      String str3 = str1.intern();
6      System.out.println(str1 == str2);
7      System.out.println(str2 == str3);
8      System.out.println(str1 == str3);
9    }
10 }
```

    What is the result?
    A. false, true, false
    B. true, true, true
    C. false, false, false
    D. true, false, false

ans

18. **Este patrón suele representarse como una única librería que contiene la mayoría de la funcionalidad y que otras librerías utilizan incorrectamente de apoyo. La clave del problema es que puede contener un porcentaje muy elevado de responsabilidades dentro de las solución a diseñar:**
    A. Contenedor mágico
    B. Blob
    C. CRUD
    D. Librería

ans

19. **Given:**

```java
public class Test {
  public static void op1(String s) {
    s += new String("Java");
  }
  public static void stringTest() {
    String msg = new String("Hello");
    op1(msg);
    System.out.println(msg);
  }
  public static void main(String... String) {
    stringTest();
  }
}
```

What is the result?
A. HelloJava
B. Hello
C. An exception is thrown at runtime
D. Java
E. Compilation fails
F. null

ans

20. **Given:**

```java
import java.util.List;
import java.util.function.Predicate;
class Person {
  String name;
  public Person(String name) {
    this.name = name;
  }
  @Override
  public String toString() {
    return name;
  }
}
public class Test {
  static List<Person> filter(List<Person> input,
                             Predicate<Person> predicate) {
    return input.stream().filter(predicate).toList();
  }
  public static void main(String[] args) {
    var a = new Person("John");
    var b = new Person("Jane");
    var c = new Person("Mark");
    var input = List.of(a, b, c);
    var output = filter(input, p -> !p.name.startsWith("J"));
    System.out.println(output);
  }
}
```



What is the given program's output?
A. [John, Jane, Mark]
B. null
C. [Jane]
D. [ ]
E. [Mark]
F. [John, Jane]
G. [John]

ans

21. **¿Cuál será el resultado de ejecutar el siguiente código?**

```
class Parent {
  public void display() {
    System.out.println("Parent");
  }
}
class Child extends Parent {
  public void display() {
    System.out.println("Child");
  }
}
public class Test {
  public static void main(String[] args) {
    Parent obj = new Child();
    obj.display();
  }
}
```

A. Parent
B. Error de ejecución
C. Child
D. Error de compilación

ans

22. **Given:**

```java
public class OverloadedMethods {
  public OverloadedMethods() {
    this(10);
    System.out.println("No-argument constructor");
  }
  public OverloadedMethods(int x) {
    System.out.println("Constructor with int argument: " + x);
  }
  public void calculate() {
    System.out.println("No-argument calculate method");
  }
  public void calculate(int x) {
    System.out.println("Calculate method with int argument: " + x);
  }
  public static void main(String[] args) {
    OverloadedMethods obj1 = new OverloadedMethods();
    obj1.calculate();
    obj1.calculate(5);
    OverloadedMethods obj2 = new OverloadedMethods(20);
    obj2.calculate();
    obj2.calculate(15);
  }
}
```

What will be the output of the following code snippet?

A. No-argument constructor Constructor with int argument: 10 No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 Calculate method with int argument: 15 No-argument calculate method

B. No-argument constructor Constructor with int argument: 10 Calculate method with int argument: 5 No-argument calculate method Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15

C. Constructor with int argument: 10 No-argument constructor No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15

D. Constructor with int argument: 10 No-argument constructor Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15 No-argument calculate method

ans

23. **Which of the following is not a part of the Java Collections Framework?**
A. ArrayList
B. Map
C. Dictionary
D. List

ans

24. **Cual de los siguientes comandos de Maven se utiliza para compilar un proyecto JAR, pero sin depositarlo en el repositorio local?**
    A. mvn compile
    B. mvn clean
    C. mvn install
    D. mvn package

ans

25. **What is the result type when a `short` is subtracted from a `double` in Java?**
    A. short
    B. You cannot subtract a short from a double.
    C. int
    D. float
    E. double

ans

26. **Given:**

```java
public class Test {
  static boolean b;
  public static void main(String[] args) {
    int x = 0;
    if (b) {
      x = 1;
    } else if (b = false) {
      x = 2;
    } else if (b) {
      x = 3;
    } else {
      x = 4;
    }
    System.out.println("x = " + x);
  }
}
```

What is the result?
A. x = 3
B. x = 0
C. Compilation fails
D. x = 4
E. x = 1

ans

27. **¿Cuál de las siguientes herramientas se utiliza principalmente para probar y desarrollar APIs, permitiendo enviar solicitudes HTTP y analizar las respuestas?**
    A. Git
    B. Ninguna de las anteriores.
    C. Artifactory
    D. Docker
    E. Maven

    ans

28. **¿Qué puede suceder al intentar ejecutar el siguiente código para establecer una conexión a una base de datos?**

```java
public class TestJDBC {
  public static void main(String[] args) {
    String url = "jdbc:mysql://localhost:3306/mydb";
    String user = "user";
    String password = "password";
    try {
      Connection conn = DriverManager.getConnection(url, user, password);
      System.out.println("Connection established successfully.");
    } catch (SQLException e) {
      System.out.println("Failed to establish connection.");
      e.printStackTrace();
    }
  }
}
```

    A. Establece la conexión exitosamente e imprime "Connection established successfully"
    B. Lanza una ClassNotFoundException.
    C. Lanza una NullPointerException.
    D. Falla al establecer la conexión e imprime "Failed to establish connection."

    ans

29. **Given:**

```java
public class OverloadedExample {
  private String name;
  public OverloadedExample() {
    this("Default");
    System.out.println("No-argument constructor called");
  }
  public OverloadedExample(String name) {
    this.name = name;
    System.out.println("Constructor with argument called: " + name);
  }
  public void display() {
    System.out.println("Name: " + name);
  }
  public void display(String prefix) {
    System.out.println(prefix + ":" + name);
  }
  public static void main(String[] args) {
    OverloadedExample obj1 = new OverloadedExample();
    obj1.display();
    obj1.display("Hello");
    OverloadedExample obj2 = new OverloadedExample("John");
    obj2.display();
    obj2.display("Hi");
  }
}
```

What will be the output of the following code snippet?

A. No-argument constructor called Constructor with argument called: Default Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

B. No-argument constructor called Constructor with argument called: Default Name: Default Hello: Default Constructor with argument called: John Hi: John Name: John

C. Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Hi:John Name: John

D. Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

ans

30. **In Java, which syntax is used to declare a method parameter that accepts a variable number of arguments?**

A. []
B. varargs
C. < >
D. ...
E. { }

ans

31. **En JDBC APX cual es la cantidad máxima de resultados que permite obtener la consulta antes de que se deba de paginar?**
    A. 1000
    B. 2000
    C. 5000
    D. 4000
    E. 3000

ans

32. **Which statement about the following code is correct?**

```java
class Base {
  public Base() {
    System.out.println("Base constructor");
  }
  public Base(String message) {
    System.out.println("Base constructor with message: " + message);
  }
}
class Derived extends Base {
  public Derived() {
    super("Hello");
    System.out.println("Derived constructor");
  }
}
public class Test {
  public static void main(String[] args) {
    Derived derived = new Derived();
  }
}
```

    A. It will print "Base constructor" followed by "Derived constructor".
    B. It will print "Base constructor with message: Hello" followed by "Derived constructor".
    C. It will throw a compilation error.
    D. It will print "Derived constructor" followed by "Base constructor with message: Hello".

ans

33. **¿Qué patrón de diseño es el más adecuado para garantizar que una clase tenga solo una instancia y proporciona un punto de acceso global a esta?**
    A. Strategy Pattern
    B. Factory Pattern
    C. Singleton Pattern
    D. Observer Pattern
    E. Command Pattern

ans

34. **¿Cuál de los siguientes componentes de APX representa una entidad comercial**

en forma de un Bean?
A. Transacciones
B. Jobs
C. Librerías
D. Todas las opciones son correctas
E. DTOS

ans

35. **¿Qué palabra reservada se utiliza para propagar una excepción desde un método?**
A. `throw`
B. `throws`
C. Ambas, dependiendo del contexto.
D. `finally`

ans

36. **¿Cuál es el propósito principal del archivo `pom.xml` en un proyecto Maven?**
A. Para almacenar la configuración del repositorio del proyecto.
B. Para especificar la versión de Java utilizada en el proyecto.
C. Para definir las dependencias del proyecto, la configuración de compilación y los complementos.
D. Para almacenar la configuración específica del usuario.

ans

37. **Which tool is used to package Java classes into a JAR (Java Archive) file?**
A. jconsole
B. java
C. jps
D. javac
E. jar

ans

38. **¿Cuál de los siguientes fragmentos de código generará una `ClassCastException` en tiempo de ejecución? Asumase que B hereda de A.**
A. `A obj = new B(); A a = (A) obj;`
B. `B obj = new B(); A a = (A) obj;`
C. `A obj = new B(); B b = (B) obj;`
D. `A ob = new A(); B b = (B) ob;`

ans

39. **Which of the following operators are valid for boolean variables in Java?**
A. `&&`
B. `|`
C. `+`
D. `||`
E. `==`

ans

40. **What is the result?**

```java
public class LoopQuestion {
  public static void main(String[] args) {
    int count = 0;
    for (int i = 0; i < 10; i++) {
      if (i % 2 == 0) {
        count++;
      }
    }
    System.out.println("Count: " + count);
  }
}
```

A. Compilation fails.
B. Count: 5
C. Count: 4
D. Count: 10
E. Count: 6

ans

41. **What is the output of the following program?**

```java
public class Test {
  public static void main(String[] args) {
    int x = 0;
    while (x++ < 5) {
      // Empty loop body
    }
    String message = x > 5 ? "greater than" : false;
    System.out.println(message + " - " + x);
  }
}
```

A. false than - 6
B. greater than - 6
C. false than - 5
D. Compilation fails.
E. greater than - 5

ans

42. **¿Qué es el APX CLI?**
A. Herramienta Web que permite crear el andamio de un proyecto APX.
B. Herramienta para administrar las dependencias de un proyecto APX.
C. Herramienta web que proporciona a los desarrolladores y administradores gestionar la configuración.
D. Es una interfaz de línea de comandos para realizar operaciones sobre componentes APX.

ans

43. **Which of the following statements about the Singleton pattern is true?**
A. The Singleton pattern allows a class to be instantiated multiple times but provides a global access point to the most recently created instance.
B. The Singleton pattern ensures that a class has only one instance and provides a global point of access to it.
C. The Singleton pattern is mainly used to facilitate inheritance and polymorphism in object-oriented programming.
D. The Singleton pattern is used to create a collection of objects that can be accessed globally.

ans

44. **¿Cuáles dos de las siguientes opciones son funciones de la siguiente instrucción?**
`apx deploy local -runtime online -country=ESP -y`
A. Despliega el componente APX en el enfoque APX Online en el lenguaje Español.
B. Compila, empaqueta y ejecuta pruebas del componente en el que está ubicado.
C. Despliega el componente APX en el enfoque APX Online en la región de España.
D. La instrucción tiene un error.

ans

45. **¿Cuál de las siguientes bases de datos NoSQL es conocida por su modelo de documentos, donde los datos se almacenan en formato BSON (Binary JSON)?**
A. MongoDB
B. Neo4j
C. Cassandra
D. Redis

ans

46. **¿Cómo solucionamos un Blob?**
A. Realizar un refactoring del diseño entre las librerías involucradas moviendo el comportamiento a sus librerías relacionadas.
B. Normalmente es muy complicada de reutilizar y probar.
C. Agregar una gran cantidad de métodos "execute".
D. Realizar un refactoring del diseño sin las librerías involucradas.

ans

47. **¿Cuál es la principal función del servidor de aplicaciones Jboss?**
A. Proporcionar una plataforma para desplegar, ejecutar y gestionar aplicaciones empresariales Java.
B. Ofrecer un sistema de control de versiones para proyectos Java.
C. Actuar como un servicio para el desarrollo de aplicaciones HTML y CSS.
D. Proporcionar un entorno de desarrollo integrado (IDE) para aplicaciones Java.

ans



48. **¿Cuál de los siguientes comandos de Git se utiliza para crear una nueva rama y cambiarse a ella?**
    A. `git branch nombre_de_rama`
    B. `git checkout -m nombre_de_rama`
    C. `git branch -d nombre_de_rama`
    D. `git checkout -b nombre_de_rama`

ans

49. **¿Cuáles de las siguientes excepciones deben ser chequeadas?**
    A. IOException
    B. IndexOutOfBoundsException
    C. InterruptedException
    D. NullPointerException
    E. ClassCastException
    F. ArithmeticException

ans

50. **In a Maven project which file is used to configure project dependencies, build configuration and project metadata?**
    A. package.json
    B. settings.xml
    C. pom.xml
    D. build.gradle
    E. apx.yaml

ans

51. **What is the result?**

```java
import java.util.*;
public class Test {
  public static void main(String[] args) {
    List<Item> items = new ArrayList<>();
    items.add(new Item("Apple", 3));
    items.add(new Item("Banana", 2));
    items.add(new Item("Cherry", 1));
    Collections.sort(items);
    for (Item item : items) {
      System.out.println(item.getName() + ": " + item.getPrice());
}}}
class Item implements Comparable<Item> {
  private String name;
  private Integer price;
  public Item(String name, int price) {
    this.name = name;
    this.price = price;
  }
  public String getName() { return name; }
  public int getPrice() { return price; }
  @Override
  public int compareTo(Item other) {
    return this.price.compareTo(other.price);
  }
}
```

A. Error de compilación debido a una implementación incorrecta de Comparable.
B. Imprimirá los artículos ordenados por precio.
C. Error de tiempo de ejecución debido a una implementación incorrecta de Comparable.
D. Imprimirá los artículos ordenados por nombre.

ans

52. **Dado el siguiente código:**

```java
public class Test {
  doMath(long l, short s) {
    return s / (l * 10);
  }
}
```

¿Cuál es el tipo de retorno válido de punto flotante más restringido para el método?
A. long
B. float
C. short
D. int
E. byte
F. double

ans

53. **¿Cuál de las siguientes anotaciones en JUnit se utiliza para indicar que un método debe ejecutarse antes de cada método de prueba?**
    A. @Before
    B. @Test
    C. @JUnit
    D. @BeforeClass
    E. @After

ans

54. **What is the result?**

```java
import java.util.ArrayList;
public class Test {
  public static void main(String[] args) {
    String v = new String();
    StringBuffer sb = new StringBuffer();
    class MyList extends ArrayList {}
    System.out.println((v instanceof String) &&
      (sb instanceof Object) && (new MyList() instanceof ArrayList));
  }
}
```

    A. false
    B. true
    C. The code runs with no output.
    D. Compilation fails.
    E. An exception is thrown at runtime.
    F. null

ans

55. **¿Cuál de las siguientes sentencias compila dentro del método `test()`?**

```java
class Animal {

  Animal() { }

  Animal(String name) { }

  void sound() {
    System.out.println("Animal");
  }
}

class Dog extends Animal {

  void test() {
    // Insert Code
  }
}
```

    A. `super();`
    B. `super("Dog");`
    C. `super.sound();`
    D. `this.super.sound();`

ans

56. **What is the result?**

```java
public class Test {
  public static void main(String[] args) {
    String str = "a1b2c3d4e5f6";
    String[] parts = str.split("\\D");
    for (String part : parts) {
      System.out.print(part + " ");
    }
  }
}
```

A. 1 2 3 4 5 6
B. Compilation fails.
C. a b c d e f
D. An exception is thrown at runtime.
E. 123456
F. abcdef

ans

57. **What is the result?**

```java
public class Test {
  static int co = 0;
  public static void main(String[] args) {
    int x = 1;
    for (test('1'); test('2') && (x <= 2); test('3')) {
      x++;
      test('4');
    }
    System.out.println(" The co value = " + co);
  }
  static boolean test(char num) {
    System.out.print("" + num);
    co++;
    return true;
  }
}
```

A. 12432432 The co value = 8
B. 1234 The co value = 4
C. 14231423 The co value = 8
D. 12341234 The co value = 8
E. 12342342 The co value = 8
F. Compilation fails

ans

58. **Which of the following statements about ArrayList is true?**
    A. ArrayList allows duplicate elements.
    B. ArrayList does not maintain the insertion order of elements.
    C. ArrayList elements are stored in a tree structure.
    D. ArrayList can store primitive types directly.

ans

59. **What is the result?**

```java
import java.time.LocalDateTime;
import java.time.Period;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
public class Test {
  public static void main(String[] argv) {
    LocalDateTime d = LocalDateTime.of(1997, 5, 15, 23, 55, 30);
    Period p = Period.of(1, 2, 3);
    d = d.minus(p);
    DateTimeFormatter f =
      DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);
    System.out.println(d.format(f));
  }
}
```

    A. 15/05/1997
    B. The code does not compile
    C. 05/15/97 23:55 AM
    D. 0.996875
    E. A runtime exception is thrown
    F. 12/3/1996
    G. 12/03/96 23:55

ans

60. **¿Cuál es el tipo de resultado cuando se resta un short de un double en Java?**
    A. int
    B. No se puede restar un short de un double.
    C. float
    D. short
    E. double

ans

61. **¿Cuál de los siguientes métodos de Mockito se utiliza para verificar que un método de un mock ha sido llamado un número específico de veces?**
    A. doThrow()
    B. verify()
    C. doReturn()
    D. Ninguna opción es correcta.
    E. when() & thenReturn()

ans

62. **¿Cuál de las siguientes afirmaciones es correcta sobre la aserción `assertEquals` en JUnit?**
    A. Se utiliza para verificar que una colección contiene un elemento específico.
    B. Se utiliza para verificar que dos valores son iguales.
    C. Se utiliza para verificar que una condición es verdadera.
    D. Se utiliza para verificar que dos objetos referencian la misma instancia. ans

63. **What is the given code fragment's output?**

```java
import java.util.Arrays;
public class Test {
  public static void main(String[] args) {
    String[] strings = { "2", "3", "1", "4" };
    Arrays.sort(strings, 1, 4);
    System.out.println(Arrays.toString(strings));
  }
}
```

    A. [2, 3, 1, 4]
    B. [2, 1, 3, 4]
    C. [1, 2, 3, 4]
    D. [2, 4, 1, 3]
    E. [4, 3, 2, 1] ans

64. **What is the result?**

```java
class Exc1 extends Exception {}
class Exc2 extends Exception {}
public class Test {
  static void a() throws Exc1 {
    try {
      System.out.print("Try ");
    } catch (Exc1 e) {
      System.out.print("Catch ");
    } finally {
      System.out.println("Finally ");
    }
  }
  static void b() throws Exc1, Exc2 {
    throw new Exc2();
  }
  public static void main(String args[]) {
    try {
      a();
      b();
    } catch (Exc1 e) {
      System.out.println("Caught1");
    } catch (Exc2 ex) {
      System.out.print("Caught2");
    }
  }
}
```

A. Try Finally Caught2
B. La compilación falla
C. Try
D. Try Caught2
E. Try finally

ans

65. **En Java, ¿cuál de las siguientes instrucciones es usada para llamar a un constructor desde otro constructor en la misma clase?**
A. `this.<class_name>()`
B. `super()`
C. `new()`
D. `this`

ans

66. **¿Cuáles son las utilerías que pueden ser usadas en APX Batch?**
A. JDBC Utility
B. API Connector
C. Merge/Sort
D. Neo4J
E. Document Manager

ans

67. **What will be the output of the following code snippet?**

```
import java.io.Closeable;
import java.io.IOException;
class Resource implements AutoCloseable {
  @Override
  public void close() {
    System.out.print("Resource closed ");
  }
  public void use() {
    System.out.print("Resource used ");
  }
}
public class Test {
  public static void main(String[] args) {
    try (Resource resource = new Resource()) {
      resource.use();
    } catch (Exception e) {
      e.printStackTrace();
    }
  }
}
```

A. Compilation error.
B. Resource used Resource closed.
C. Resource closed Resource used.
D. Runtime error.

ans

68. **En el contexto del patrón de diseño de Comando (Command Pattern), ¿cuál es el propósito de la clase invocadora (Invoker)?**
A. Crear objetos que representan acciones y sus parámetros.
B. Definir una interfaz común para todas las clases que representan comandos.
C. Mantener la lógica de los objetos comando y ejecutar el comando solicitado en el momento adecuado.
D. Ejecutar un comando y, opcionalmente, almacenar y deshacer el comando si es necesario.
ans

69. **¿Cuál es el propósito principal del patrón de paginación en el desarrollo de aplicaciones web?**
A. Mejorar la seguridad de la aplicación al evitar ataques de inyección SQL.
B. Incrementar la portabilidad del software al utilizar estándares de codificación universal.
C. Facilitar la integración con servicios externos mediante el uso de API RESTful.
D. Optimizar el rendimiento y la experiencia del usuario al dividir grandes conjuntos de datos en páginas más pequeñas.
ans

70. **What is the result?**

```
public class NestedLoopTest {
  public static void main(String[] args) {
    for (int i = 1; i <= 3; i++) {
      for (int j = 1; j <= 2; j++) {
        System.out.println("i."+i + "," + "j."+j);
}}}}
```

A. i.1,j.1 i.1,j.2 i.2,j.1 i.2,j.2 i.3,j.1 i.3,j.2
B. i.1,j.1 i.2,j.2 i.3,j.3
C. i.1,j.1 i.1,j.2 i.1,j.3 i.2,j.1 i.2,j.2 i.2,j.3
D. i.1,j.1 i.2,j.1 i.3,j.1
ans

71. **¿Cuál es la salida del siguiente código?**

```
import java.time.LocalDate;
import java.time.Month;
public class Test {
  public static void main(String[] argv) {
    LocalDate date = LocalDate.of(1997, Month.MAY, 50);
    System.out.println(date.getYear() + "-" + date.getMonth()
     + "-" + date.getDayOfMonth());
  }
}
```

A. 1997-MAY-15
B. 1997-JUN-15
C. Se genera una excepción en tiempo de ejecución.
D. El código no compila.
E. 14-JUN-1997
ans

72. **Which of the following lines of code correctly uses a method from the String class to replace all occurrences of the character 'a' with 'e' in a given string in Java?**
A. `String result = "banana".replace('a', 'e');`
B. `String result = "banana".change('a', 'e');`
C. `String result = "banana".modify('a', 'e');`
D. `String result = "banana".substitute('a', 'e');`
E. `String result = "banana".swap('a', 'e');`

ans

73. **Consider the following Java Code:**

```java
class Animal {
  public void makeSound() {
    System.out.println("Sound");
  }
  public void makeSound(String sound) {
    System.out.println(sound);
  }
}
class Dog extends Animal {
  @Override
  public void makeSound() {
    System.out.println("Bark");
  }
  public void makeSound(int times) {
    for (int i = 0; i < times; i++) {
      System.out.println("Bark");
    }
  }
}
public class Test {
  public static void main(String[] args) {
    Animal animal = new Animal();
    Animal dog = new Dog();
    animal.makeSound();
    animal.makeSound("Roar");
    dog.makeSound();
    dog.makeSound("Growl");
  }
}
```

What will be the output of the above code?
A. Sound Roar Bark Roard
B. Sound Roar Bark Growl
C. Sound Roar Bark Bark
D. Sound Roar Sound Growl

ans

74. **APX proporciona a las aplicaciones las utilidades para ejecutar una consulta solo a base de datos relacional. Seleccione una:**
    A. Verdadero
    B. Falso

ans

75. **Resultado de la operacion `9 | 4`**
    A. 13

ans

76. **Which of the following are valid ways to initialize a boolean variable in Java?**
    A. `boolean flag = Boolean.FALSE;`
    B. `boolean flag = true;`
    C. `boolean flag = 0;`
    D. `boolean flag = false;`
    E. `boolean flag = "false";`

ans

77. **Which is the primary purpose of the pom.xml file in a Maven project?**
    A. To specify the Java version used in the project.
    B. To manage the project's repository settings.
    C. To define the project's dependencies, build configuration, and plugins.
    D. To store user-specific configuration.

ans

78. **What is the result?**

```java
public class Test {
  public static void main(String[] args) {
    try {
      mf();
      System.out.print("A");
    } catch (Exception e) {
      System.out.print("B");
    } finally {
      System.out.print("C");
    }
    System.out.print("D");
  }
  static void mf() {
  }
}
```

    A. ABCD
    B. ACD
    C. AC
    D. D
    E. compilation fails
    F. BCD

ans

79. **En Java, ¿qué sintaxis es usada para declarar un parámetro en un método que acepte un número variable de argumentos?**
A. `< >`
B. `...`
C. `[ ]`
D. `varargs`
E. `{ }`

ans

80. **¿Qué es un Job en el contexto de Spring Batch?**
A. Una clase que gestiona la persistencia de datos en la base de datos.
B. Una entidad que encapsula todo el proceso por lotes, configurada a través de un archivo XML o una configuración basada en Java.
C. Un componente que ejecuta tareas individuales dentro de un Step.
D. Una interfaz que define métodos para realizar operaciones CRUD en los datos del JobRepository.

ans

81. **Given:**

```java
public class Test {
  public static void main(String[] args) {
    float f = 3.14f;
    new Test().doIt(f);
  }
  void doIt(Float f) {
    System.out.println("Float");
  }
  void doIt(double d) {
    System.out.println("double");
  }
  void doIt(float... fs) {
    System.out.println("varargs");
  }
}
```

What is the result?
A. Compilations fails.
B. An exception is thrown at runtime.
C. The output is not predictable.
D. Float
E. varargs
F. double

ans

82. **¿Cuál de los siguientes componentes de APX encapsula la lógica empresarial y el acceso a los datos, además publica su interfaz para que pueda ser usada por terceros?**
A. Librerías
B. DTOs
C. Jobs
D. Todas las opciones son correctas.
E. Transacciones

ans

83. **Which of the following statements accurately describe the relationships that can exist between classes in Java?**
A. Inheritance represents an "is-a" relationship where one class derives from another class.
B. Composition represents a "has-a" relationship where one class contains an instance of another class.
C. Inheritance should be preferred over composition to promote code reuse and flexibility.
D. Both inheritance and composition can be used together to model complex relationships.
E. Usage (or association) represents a "uses-a" relationship where one class uses methods or instances of another class.
F. Composition should be preferred over inheritance to promote code reuse and flexibility.

ans

84. **What is the result?**

```
public class Test {
    public static void main(String[] args) {

        try (AutoCloseable resource =
                () -> System.out.print("Resource closed ")) {

            System.out.print("Resource used ");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

A. Compilation error.
B. Resource used Resource closed.
C. Resource closed Resource used.
D. Runtime error.

ans

85. **¿Cuál es el comando en Windows para listar el contenido de un directorio/carpeta?**
A. ls
B. list

C. mv
D. dir
E. show

ans

86. **What are the modifiers that a nested class can have?**
A. public
B. final
C. protected
D. abstract
E. static
F. private
G. strictfp
H. All the options are correct.

ans

87. **What is the result?**

```
import java.util.*;
public class ComparableTest {
  public static void main(String[] args) {
    List<Item> items = new ArrayList<>();
    items.add(new Item("Apple", 3));
    items.add(new Item("Banana", 2));
    items.add(new Item("Cherry", 1));
    Collections.sort(items);
    for (Item item : items) {
      System.out.println(item.getName() + ": " + item.getPrice());
}}}
class Item implements Comparable<Item> {
  private String name;
  private int price;
  public Item(String name, int price) {
    this.name = name;
    this.price = price;
  }
  public String getName() { return name; }
  public int getPrice() { return price; }
  @Override
  public int compareTo(Item other) {
    return this.name.compareTo(other.name);
  }
}
```

A. Apple: 3 Banana: 2 Cherry: 1
B. Compilation error
C. Apple: 3 Cherry: 1 Banana: 2
D. Cherry: 1 Banana: 2 Apple: 3

ans

88. **What will be the output of the following code snippet?**

```
class A {
  public A() {System.out.println("A's constructor");
  }}
```

```
class B extends A {
  public B() {
    super();
    System.out.println("B's constructor");
    }}
class C extends B {
  public C() {System.out.println("C's constructor");
  }}
public class Test {
  public static void main(String[] args) {
    C c = new C();
  }
}
```

A. A's constructor B's constructor C's constructor
B. C's constructor A's constructor B's constructor
C. A's constructor C's constructor B's constructor

ans

89. **What will be the output of the following code snippet?**

```
public class StringConcatenationTest {
  public static void main(String[] args) {
    String str1 = "Hello";
    String str2 = "World";
    String str3 = str1 + " " + str2;
    String str4 = str1.concat(" ").concat(str2);
    String str5 = new StringBuilder().append(str1).append(" ").append(
       str2).toString();
    System.out.print(str3.equals(str4) + " ");
    System.out.print(str4.equals(str5) + " ");
    System.out.print((str3 == str4) + " ");
    System.out.println(str4 == str5);
  }
}
```

A. true true true true
B. Compilation fails.
C. true true false false
D. false false false false
E. false true false true
F. false false true true

ans

90. **What is the result?**

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public class TestJDBC {
  public static void main(String[] args) {
    String url = "jdbc:mysql://localhost:3306/testdb";
    String user = "testuser";
    String password = "testpassword";
    try {
      Connection conn = DriverManager.getConnection(url, user, password);
      Statement stmt = conn.createStatement();
      ResultSet rs = stmt.executeQuery("SELECT name FROM users WHERE id =
          1");
      while (rs.next()) {
        System.out.println(rs.getString("name"));
      }
    } catch (SQLException e) {
      e.printStackTrace();
    }
  }
}
```

A. It will throw a NullPointerException.
B. It will print the name of the user with id 1.
C. It will print "Connection established successfully."
D. It will throw a SQLException.

ans

91. **¿Cuáles de las siguientes son herramientas necesarias para desarrollar componentes APX?**
A. Git
B. Docker
C. Artifactory
D. Maven
E. JDK

ans

92. **What is the result?**

```java
public class Test {
  public void op2(String s) {
    s.toUpperCase();
  }
  public void op3() {
    String msg = new String("message");
    op2(msg);
    System.out.println(msg);
  }
  public static void main(String[] args) {
    new Test().op3();
  }
}
```

A. MESSAGE
B. An exception is thrown at runtime.
C. Compilation fails.
D. message
E. null

ans

93. **¿Cuál de las siguientes afirmaciones confirma correctamente los paquetes que usa el siguiente código?**

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public class TestJDBC {
  public static void main(String[] args) {
    String url = "jdbc:mysql://localhost:3306/mydb";
    String user = "user";
    String password = "password";
    Connection conn = null;
    Statement stmt = null;
    ResultSet rs = null;
    try {
      conn = DriverManager.getConnection(url, user, password);
      stmt = conn.createStatement();
      rs = stmt.executeQuery("SELECT name FROM users WHERE id = 1");
      while (rs.next()) {
        System.out.println(rs.getString("name"));
      }
    } catch (SQLException e) {
      e.printStackTrace();
    } finally {
      // Close resources here
    }
  }
}
```

A. `rs.close(); stmt.close(); conn.close();`
B. `if (rs != null) rs.close(); if (conn != null) conn.close(); if (stmt != null) stmt.close();`
C. `if (rs != null) rs.close(); if (stmt != null) stmt.close(); if (conn != null) conn.close();`
D. `stmt.close(); conn.close(); rs.close();`

ans

94. **¿Cuál es la respuesta verdadera?**

```java
public class Test implements Runnable {
  private int i;
  @Override
  public void run() {
    if (i % 10 == 0) {
      i++;
    }
    for (int x = 0; x < 10; x++, i++) {
      if (x == 4) {
        Thread.yield();
      }
      System.out.println(i + " ");
    }
  }
  public static void main(String[] args) {
    Test n = new Test();
    for (int x = 0; x < 100; x++) {
      new Thread(n).start();
    }
  }
}
```

A. La salida nunca puede contener el valor 297.
B. La salida nunca puede contener el valor 1020.
C. La salida nunca puede contener el valor 1130.
D. La salida nunca puede contener el valor 30.

ans

95. **What is the result?**

```java
public class OverloadedStaticMethods {
  public static void display() {
    System.out.println("Display with no arguments");
  }
  public static void display(int num) {
    System.out.println("Display with integer: " + num);
  }
  public static void display(String str) {
    System.out.println("Display with string: " + str);
  }
  public static void main(String[] args) {
    OverloadedStaticMethods.display();
    OverloadedStaticMethods.display(5);
    OverloadedStaticMethods.display("Hello");
  }
}
```

A. Display with no arguments Display with string: Display with integer: Hello
B. Error en compilación
C. Error en tiempo de ejecución
D. Display with no arguments Display with integer: 5 Display with string: Hello

ans

96. **¿Cuál de las siguientes opciones se utiliza principalmente para la integración continua en el desarrollo del software?**
A. Jenkins
B. Artifactory
C. Jira
D. SonarQube
E. Bitbucket

ans

97. **What is the result?**

```java
import java.util.ArrayList;
import java.util.List;
public class Test {
  public static void main(String[] args) {
    List<String> list = new ArrayList<>();
    list.add("Apple");
    list.add("Banana");
    list.add("Cherry");
    printList(list);
    System.out.println("Size: " + list.size());
    System.out.println("Contains 'Banana': " + list.contains("Banana"));
    System.out.println("Element at index 1: " + list.get(1));
  }
  private static void printList(List<String> list) {
    for (String element : list) {
      System.out.println(element);
    }
  }
}
```

A. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana
B. Apple Banana Size: 2 Contains 'Banana': false Element at index 1: Banana
C. Apple Banana Size: 2 Contains 'Banana': true Element at index 1: Banana
D. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Cherry

ans

98. **Todo el acceso a los datos debe estar encapsulado en una transacción para facilitar la reutilización y control de acceso**
A. Verdadero
B. Falso

ans

99. **Given:**

```java
interface IFun {
  int calculate(int i1, int i2);
}
public class Test {
  public static void main(String[] args) {
    IFun f = (i1, i2) -> i1 + i2; // line 1
    int i1=1, i2 = 2; // line 2
    IFun g = (i1, i2) -> i1 * i2; // line 3
    System.out.println(g.calculate(i1, i2)); // line 4
  }
}
```

What will be the output of the above code?
A. Compilation fails on line 4.
B. Compilation fails on line 2.
C. 2
D. Compilation fails on line 1.
E. Compilation fails on line 3.
F. 3

ans

100. **¿Cuál de las siguientes ramas en Gitflow se utiliza para aplicar correcciones rápidas directamente en producción?**
A. master
B. feature
C. hotfix
D. release
E. develop

ans

101. **What is the result?**

```java
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class StreamTest {
    public static void main(String[] args) {
        List<String> items = Arrays.asList("apple", "banana", "cherry", "
            date");
        List<String> result = items.stream().filter(s ->s.startsWith("a")
            ).collect(Collectors.toList());
        System.out.println(result);
        }
    }
```

A. [apple, banana]
B. [banana, cherry]
C. [apple]
D. [apple, cherry]

ans

102. **Which line of code, when inserted at the commented line, causes the compiler to fail?**

```
class ABC {
    public String met() {
        return "snippet 1";
    }
}
    public class ABCD extends ABC {
  // Insert code here
}
```

A. `public String met() { return "4"; }`
B. `public char met(int i) { return '3'; }`
C. `public String met(int i) { return "5"; }`
D. `public char met() { return '2'; }`

ans

103. **What is the result?**

```
    class CA {
      CA() {
        System.out.print("middle");
      }
      CA(String s) {
        System.out.print(s);
      }
    }
    class CB extends CA {
    }
    public class Test {
      public static void main(String[] args) {
        new CA("message ");
        new CB();
        System.out.print(" end");
      }
    }
```

A. message middle end
B. An exception is thrown at runtime.
C. Compilation fails
D. message end middle

ans

104. **Una transacción APX es la unidad aplicativa que se ejecutará en APX Batch. Seleccione una:**
A. Verdadero
B. Falso

ans

105. **¿Cuál es la función principal del JDK (Java Development Kit)?**
A. Permitir la edición de archivos JHTML y JCSS.
B. Servir como un servidor web para aplicaciones Java.
C. Proporcionar un entorno de ejecución para aplicaciones Java.
D. Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.
E. Ninguna opción es correcta.

ans

106. **¿Cuál es el propósito del método `setSeverity()` en el manejo de una transacción APX?**
A. Validar los datos proporcionados por la transacción.
B. Establecer la gravedad de la notificación generada por la transacción
C. Determinar el tipo de transacción a ejecutar
D. Configurar los parámetros de entrada de la transacción.

ans

107. **What will be the output of the above code?**

```java
import java.util.function.*;
public class Test {
    public static void main(String[] args) {
        Predicate<String> p = (s) -> s.length() > 5;
        Function<String, Integer> f = String::length;
        Consumer<String> c = (s) -> System.out.println(s.toUpperCase());
        Supplier<String> s = () -> "Hello, world!";
        String testString = "functional";
        System.out.println(p.test(testString));
        System.out.println(f.apply(testString));
        c.accept(testString);
        System.out.println(s.get());
    }
}
```

A. false 9 FUNCTIONAL Hello, World!
B. A runtime exception is thrown.
C. Compilation fails.
D. true 9 FUNCTIONAL Hello, World!
E. true 10 FUNCTIONAL Hello, World!

ans

108. **What is the result?**

```java
public class Test {
  public static void main(String[] args) {
    String a = "tic";
    String b = "tac";
    a.substring(0, 2);
    a.concat("toc");
    System.out.println(a += "game" + b.toUpperCase());
  }
}
```

A. ticgameTAC
B. titocTAC
C. tictactoc
D. ticgametac
E. Compilation fails.

ans

109. **Which of the following packages in Java do not need to be explicitly imported in a Java program?**
A. java.net
B. java.lang
C. java.function
D. java.util
E. java.io
F. javax.sql

ans

110. **Which of the following statements correctly describes an immutable object in Java?**
A. An immutable object must implement the Cloneable interface.
B. Immutable objects cannot be created using the new keyword.
C. Immutable objects can have setter methods to change their fields.
D. An immutable object does not allow any changes to its state after it is created.
E. An immutable object can have its fields modified after it is created.

ans

111. **What will be the output of the following code snippet?**

```java
public class EnumTest {
  enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
  }
  public static void main(String[] args) {
    for (Day day : Day.values()) {
      System.out.print(day + " ");
    }
  }
}
```

A. Compilation error.
B. MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
C. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY
D. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY

ans

112. **What is the result?**

```
enum Days {
  MONDAY, TUESDAY, WEDNESDAY, THURSDAY
}
public class Test {
  public static void main(String[] args) {
    int y = 1;
    Days d = Days.TUESDAY;
    switch (d) {
      case MONDAY:
        y++;
      case TUESDAY:
        y += 10;
      case WEDNESDAY:
        y += 100;
      case THURSDAY:
        y += 1000;
    }
    System.out.println(y);
  }
}
```

A. 11
B. 111
C. Compilation fails.
D. An exception is thrown at runtime.
E. 1111

ans

113. **Cuando se asigna un byte a un carácter en Java, ¿cuál es el tipo de resultado?**
A. char
B. int
C. byte
D. short
E. No puedes asignar un byte a un carácter.

ans

114. **Dado:**

```
class Test {
  static void m(int a) {
    a += 10;
  }
  public static void main(String[] args) {
    int a = 20;
    m(a);
    Test.m(a);
    System.out.println(a);
  }
}
```

¿Cuál es el resultado?

A. 10
B. 40
C. 20
D. Compilation fails.
E. An exception is throw at run time.
F. 30

ans

115. **What is the result?**

```
abstract class Animal {
  private void speak() {
    System.out.println("Animals make noise");
  }
  public static void main(String[] args) {
    Dog animal = new Dog();
    animal.speak();
  }
}
class Dog extends Animal {
  protected void speak() {
    System.out.println("The Dog barks");
  }
}
public class Test {
  public static void main(String... strings) {
    Animal.main(strings);
  }
}
```

A. The Dog barks
B. Compilation fails.
C. Animals make noise
D. A runtime exception is thrown.

ans



116. **What is the result?**

```java
public class Test {
  public static void main(String[] args) {
    String str = "Hello";
    modify(str);
    System.out.println(str);
    StringBuffer sbf = new StringBuffer("Hello");
    modify(sbf);
    System.out.println(sbf.replace(2, sbf.length(), "o"));
    StringBuilder sbd = new StringBuilder("Hello");
    modify(sbd);
    System.out.println(sbd.reverse());
  }
  private static void modify(String s) {
    s = s.concat(" World");
  }
  private static void modify(StringBuffer sb) {
    sb.append(" World");
  }
  private static void modify(StringBuilder sb) {
    sb.append(" World");
  }
}
```

A. Hello Hello World Hello World
B. Hello Hello Hello
C. Hello Heo dlroW olleH
D. Helio World Heo Hello World
E. Hello Heo World Hello

ans

117. **¿Cuál es la salida del siguiente código?**

```java
    public class Test {
      public void run() {
        try {
          System.out.print("Running ");
          throw new Exception();
        } catch (Exception e) {
          System.err.print("Problem");
        } finally {
          System.out.print("Stopping");
        }  }
      public static void main(String[] args) {
        new Test().run();
      }}
```

A. Running Stopping Problem
B. Running Problem Stopping
C. Running Problem
D. La compilación falla
E. Problem Stopping
F. Stopping Problem

ans

118. **Which two options are true about the default constructor?**
A. The default always invokes the no-arg superclass constructor.
B. The compiler creates a default constructor if the class does not have a no-arg constructor defined.
C. The default constructor is always public.
D. The default constructor is created only when the class has no constructors defined.
E. The default constructor inherits the arguments of the superclass constructor.

ans

119. **¿Cuál es la nomenclatura correcta para un componente de tipo librería?**
A. TESTCO01
B. TESTRO01-MX-01
C. TESTRO01
D. TESTRO01-01-MX

ans

120. **What is the result?**

```
1    boolean x = true;
2    boolean y = false;
3    boolean result = x && y || !x;
4    System.out.println(result);
```

A. Runtime error
B. False
C. True
D. Compilation error

ans

121. **Which file is used to configure user-specific settings in Maven?**
A. build.xml
B. settings.xml
C. user.xml
D. pom.xml

ans

122. **¿Qué significa que un cambio en el software sea retrocompatible?**
A. El cambio puede romper la funcionalidad existente, requiriendo ajustes en el código que depende del software.
B. El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él.
C. El cambio introduce nuevas funcionalidades que no afectan el comportamiento existente del software.
D. El cambio corrige errores menores y realiza mejoras de rendimiento.

ans

123. **Given the following classes, which line of code correctly creates an instance of the Dog class?**

```
class Animal {
  // ...
}
class Dog extends Animal {
  // ...
}
```

A. `Dog dog = new Dog();`
B. `Animal dog = new Dog();`
C. `Dog dog = new Animal();`
D. `Animal dog = new Animal();`

ans

124. **¿Qué declaraciones inicializan correctamente una variable booleana en Java?**
A. `boolean d = (4 - 14);`
B. `boolean f = "true";`
C. `boolean c = (7 >= 7);`
D. `boolean e = (10 > 5 && 2 < 1);`
E. `boolean b = (15 = 2);`
F. `boolean a = (3 < 6);`

ans

125. **En APX CLI, ¿cual opcion permite crear un proyecto Maven con una unidad de despliegue de tipo librería?**
A. `apx init dto -u="TEST" (-c="001") -d="New DTO" -domain="domain" -subpackage="subpackage" -classname="ExampleDTO" -y`
B. `apx init du-online -u="TEST" -n="online-test" -d="New Deployment Unit Online" -y`
C. `apx add dto (-c="010") -domain="customer" -d="Test of DTO" -subpackage="" -classname="ExampleDTO" -y`
D. `apx init du-lib -u="TEST" (-c="000") -d="New Deployment Unit Library" -y`
E. `apx init du-batch -u="TEST" (-n="batch-test") -d="New Deployment Unit Batch" -y`

ans

126. **¿Qué es ENTORNO LOCAL APX?**
A. Ninguna opción es correcta.
B. Es un entorno productivo para desplegar los componentes APX.
C. Herramienta para gestionar proyectos en Java.
D. Es un entorno para que los desarrolladores prueben y depuren sus transacciones y job.
E. Herramienta de línea de comandos para crear componentes APX.

ans

126b **¿Cuál de las siguientes afirmaciones describe mejor un Step en el contexto de Spring Batch?**

A. Una clase que gestiona la configuración de la base de datos utilizada por un trabajo por lotes.

B. Una interfaz que define los métodos para realizar operaciones CRUD en los datos del trabajo por lotes.

C. Un objeto de dominio que encapsula una fase independiente y secuencial de un trabajo por lotes.

D. Un componente que se encarga exclusivamente de la validación de datos en un trabajo por lotes.

ans

127. **¿Qué métodos de anulación (Override) son válidos para las siguientes clases?**

```
    class Parent {
      void display() {
        System.out.println("Parent");
      }
    }
    class Child extends Parent {
      // Override here
    }
```

A. `static void display() { System.out.println("Child"); }`

B. `private void display() { System.out.println("Child"); }`

C. `void display() { System.out.println("InChild"); }`

D. `public void display() { System.out.println("Child"); }`

ans

128. **Which of the following class declarations is not correct?**

A. `abstract class Person ()`

B. `final class Person ()`

C. `class Person ()`

D. `abstract final class Person ()`

E. `public abstract class Person ()`

ans

129. **Given the following code snippet, what will be the output of the program?**

```
    String s1 = "text";
    String s2 = new String("text");
    String s3 = "text";
    String s4 = s2.intern();
    System.out.println(s1 == s3);
    System.out.println(s1 == s2);
    System.out.println(s1 == s4);
```

A. false false true

B. true true true

C. true false false

D. true false true

ans

130. **Which method in `java.lang.Object` should be overridden if the `hashCode()` method is overridden?**
A. clone()
B. compareTo()
C. copy()
D. equals()
E. sort()

ans

131. **¿Cuál será el resultado de compilar y ejecutar el siguiente fragmento de código en Java?**

```
1   public class StaticBlockTest {
2     static {
3       System.out.println("Static block 1");
4     }
5     {
6       System.out.println("Instance block 1");
7     }
8     public StaticBlockTest() {
9       System.out.println("Constructor");
10    }
11    static {
12      System.out.println("Static block 2");
13    }
14    {
15      System.out.println("Instance block 2");
16    }
17    public static void main(String[] args) {
18      new StaticBlockTest();
19      new StaticBlockTest();
20    }
21  }
```

A. Static block 1, Static block 2, Instance block 1, Constructor, Instance block 2, Instance block 1, Constructor, Instance block 2
B. Error de compilación.
C. Static block 1, Static block 2, Instance block 1, Instance block 2, Constructor, Instance block 1, Instance block 2, Constructor
D. Instance block 1, Instance block 2, Constructor, Instance block 1, Instance block 2, Constructor, Static block 1, Static block 2

ans

132. **¿Cuál es el comando en Bash para cambiar el directorio actual a uno especificado?**

A. goto
B. cd
C. move
D. sh
E. chdir

ans

133. **Which three of the following options are considered bad practices in software development in Java?**
A. Ignoring exceptions instead of handling them properly.
B. Using empty and undocumented catch blocks.
C. Duplicating code.
D. Not testing code that is difficult to unit test.
E. Using proper naming conventions.

ans

134. **¿Qué clase puede proveer las siguientes capacidades?: no permitir valores duplicados, mantener el orden en el cual los elementos fueron insertados, proveer iteración rápida y acceso aleatorio eficiente.**
A. TreeSet
B. LinkedHashSet
C. HashSet
D. ArrayList
E. LinkedHashMap

ans

135. **¿Cuántas veces se ejecutará la instrucción println más interna en el siguiente código?**

```
public class NestedForLoopTest {
  public static void main(String[] args) {
    for (int i = 0; i < 4; i++) {
      for (int j = 0; j < 3; j++) {
        for (int k = 0; k < 2; k++) {
          System.out.println("Hello");
        }
      }
    }
  }
}
```

A. 6 veces
B. 12 veces
C. 24 veces
D. 36 veces

ans

136. **Given:**

```
interface I {
  void method();
}
public class C implements I {
  // method declarations
}
```

Which method can be declared within the C class to make it compile?
A. `public abstract void method();`
B. `protected abstract void method() { }`
C. `private void method() {}`
D. `void method() {}`
E. None of the options is valid.

ans

137. **¿Cuál de las siguientes opciones es verdadera acerca de Spring Batch?**
A. Marco de trabajo basado en jobs y steps.
B. Es una abstracción que representa la salida de un Step.
C. Es una abstracción que representa la recuperación de la entrada de un Step.
D. Es una abstracción que representa el procesamiento comercial de un Item.
E. Marco de trabajo para lotes diseñado para permitir el desarrollo de aplicaciones por lotes.
ans

138. **Dado el siguiente código:**

```
import java.util.function.Predicate;
class Foo {
  int i;
  Foo(int i) {
    this.i = i;
  }
}
public class Test {
  static boolean compare(Foo foo, Predicate<Foo> predicate) {
    return predicate.test(foo);
  }
  public static void main(String[] args) {
    Predicate<Foo> p = f -> f.i > 0; // line 1
    p = p.or(f -> f.i > 0).and(f -> f.i < 0); // line 2
    boolean b = compare(new Foo(1), p); // line 3
    System.out.println(b);
  }
}
```

¿Cuál será la salida del código anterior?

A. Compilación falla en la línea 1.
B. Compilación falla en la línea 3.
C. true
D. false
E. Compilación falla en la línea 2.

ans

139. **What is the result?**

```
public class Test {
  public static void main(String... arguments) {
    final char a = 'A', d = 'D';
    char grade = 'B';
    switch (grade) {
      case a:
      case 'B':
        System.out.print("B");
      case 'C':
        System.out.print("C");
        break;
      case d:
      case 'F':
        System.out.print("F");
    }
  }
}
```

A. B
B. Compilation fails
C. C
D. A
E. AC
F. BC

ans

140. **What is the result?**

```
public class Test {
  String value = "A";
  {
    value += "B";
  }
  {
    value += "C";
  }
  public Test() {
    value += "x";
  }
  public Test(String s) {
    value += s;
  }
  public static void main(String[] args) {
    Test m = new Test("M");
    m = new Test();
    System.out.println(m.value);
  }
}
```



A. ABCxM
B. ABCMABCB
C. ABCM
D. ABCx
E. An exception is thrown
F. the code does not compile

ans

141. **Which statement is true about the following code?**

```
abstract class Shape {
  public abstract void draw();
  public void printShape() {
    System.out.println("This is a shape");
  }
}
class Circle extends Shape {
  @Override
  public void draw() {
    System.out.println("Drawing a circle");
  }
}
public class Test {
  public static void main(String[] args) {
    Circle circle = new Circle();
    circle.draw();
    circle.printShape();
  }
}
```

A. The program will print "Drawing a circle" followed by "This is a shape".
B. Compilation error because the Shape class is abstract.
C. The program will print "This is a shape" followed by "Drawing a circle".
D. Compilation error because the Circle class does not implement the draw method.

ans

142. **¿Cuál es un antipatrón de Binary Large Object (BLOB)?**
A. Una librería compuesta de una sola clase con gran cantidad de funcionalidad.
B. Un diseño estructural que agrupa objetos en jerarquías de árbol para que los clientes traten elementos individuales y composiciones de manera uniforme.
C. Una arquitectura donde clases de vida corta existen únicamente para invocar métodos en otras clases.
D. El uso persistente de una misma biblioteca o tecnología para intentar resolver todos los problemas del sistema.
E. Un sistema donde las dependencias entre los módulos están tan acopladas que es imposible aislar un solo componente.
ans

143. **Which of the following code snippets will result in a compilation error when implementing the Vehicle interface?**

```
interface Vehicle {
  void start();
  void stop();
}
```

A. `public class Scooter implements Vehicle { public void start() { } public void stop() { } }`
B. `public class Bike implements Vehicle { public void start() { } void stop() { } }`
C. `public class Truck implements Vehicle { public void start() { } public void stop() { } public void load() { } }`
D. `public class Car implements Vehicle { public void start() { } public void stop() { } }`

ans

144. **¿Cuál de los siguientes fragmentos de código implementa correctamente Comparator para ordenar una lista de objetos Person por su edad?**

```
import java.util.Comparator;
class Person {
  private String name;
  private int age;
  public Person(String name, int age) {
    this.name = name;
    this.age = age;
  }
  public int getAge() { return age; }
  public String getName() { return name; }
}
```

A. `public int compare(Person p1, Person p2) { return p1.getAge() - p2.getAge(); }`
B. `public int compare(Person p1, Person p2) { return p1.getName().compareTo(p2.getName()); }`
C. `public int compare(Person p1, Person p2) { return Integer.compare(p1.getAge(), p2.getAge()); }`
D. `public int compare(Person p1, Person p2) { return p2.getAge() - p1.getAge(); }`

ans



145. **Given:**

```
1    public class Test {
2      ___ doMath (long l, short s) {
3        return s / (l * 10);
4      }
5    }
```

What is the narrowest valid return type for the calculate Method?

A. float
B. short
C. int
D. long
E. double
F. byte

ans

146. **In Java, which class is the superclass of all errors and exceptions?**

A. Object
B. RuntimeException
C. Exception
D. Error
E. Throwable

ans

147. **Todo el acceso a los datos debe estar encapsulado en una biblioteca para facilitar la reutilización y el control de acceso**

A. Verdadero
B. Falso

ans

148. **¿Cuál es la característica que comparten Array y ArrayList?**
A. Pueden almacenar valores nulos.
B. Ambos procesos ocurren en tiempo constante.
C. Es posible que tengan valores duplicados.
D. Array y ArrayList no garantizan la presencia de elementos ordenados.

ans

149. **What is the result?**

```java
abstract class Animal {
  void speak() {
    System.out.println("Animals make noise");
  }
  public static void main(String[] args) {
    Animal animal = new Dog();
    animal.speak();
  }
}
class Dog extends Animal {
  protected void speak() {
    System.out.println("The Dog barks");
  }
}
public class Test {
  public static void main(String... strings) {
    Animal.main(strings);
  }
}
```

A. Animals make noise
B. A runtime exception is thrown
C. Compilation fails
D. The Dog barks

ans

150. **¿Cuál será el resultado del siguiente fragmento de código?**

```java
public class OverloadedConstructorsTest {
  private String name;
  private int age;
  public OverloadedConstructorsTest() {
    this("Unknown", 0);
    System.out.println("Default constructor called");
  }
  public OverloadedConstructorsTest(String name) {
    this(name, 0);
    System.out.println("Constructor with name called");
  }
  public OverloadedConstructorsTest(String name, int age) {
    this.name = name;
    this.age = age;
    System.out.println("Constructor with name and age called");
  }
  public static void main(String[] args) {
    OverloadedConstructorsTest obj1 = new OverloadedConstructorsTest
        ();
    OverloadedConstructorsTest obj2 = new OverloadedConstructorsTest(
        "Alice");
    OverloadedConstructorsTest obj3 = new OverloadedConstructorsTest(
        "Bob", 30);
  }
}
```



A. Default constructor called Constructor with name and age called Constructor with name called Constructor with name and age called Constructor with name and age called
B. Constructor with name and age called Default constructor called Constructor with name called Constructor with name and age called
C. Constructor with name and age called Default constructor called Constructor with name and age called Constructor with name called Constructor with name and age called
D. Default constructor called Constructor with name called Constructor with name and age called

ans

151. **¿Cuál de las siguientes opciones es una herramienta de comprensión y gestión de proyectos basada en el concepto de modelo de objetos de proyecto (POM)?**

A. Git
B. IDE
C. Artifactory
D. Maven
E. Docker

ans

152. **What is the result?**

```
import java.util.Arrays;
import java.util.Comparator;
class SortLongs implements Comparator<Long> {
  @Override
  public int compare(Long a, Long b) {
    return b.compareTo(a);
  }
}
public class Test {
  public static void main(String[] args) {
    Long[] values = {20L, 70L, 50L, 30L};
    SortLongs sort = new SortLongs();
    Arrays.sort(values, sort);
    System.out.println(Arrays.toString(values));
  }
}
```

A. An ArrayIndexOutOfBoundsException is thrown.
B. [20, 70, 50, 30]
C. [20, 30, 50, 70]
D. [70, 50, 30, 20]
E. Compilation fails.

ans

153. **What is the result?**

```java
public class NestedForLoopTest {
  public static void main(String[] args) {
    int[][] matrix = {
      {1, 2, 3},
      {4, 5, 6},
      {7, 8, 9}
    };
    for (int i = 0; i < matrix.length; i++) {
      for (int j = 0; j < matrix[i].length; j++) {
        if (i == j) {
          System.out.print(matrix[i][j] + " ");
        }
      }
    }
  }
}
```

A. 147
B. 123456789
C. 159
D. 159246837

ans

154. **En el contexto de Maven, ¿Cuál es la función principal del archivo settings.xml?**
A. Ninguna opción es correcta.
B. Configurar la información del repositorio local y remoto, así como las credenciales y perfiles de usuario.
C. Especificar las dependencias del proyecto.
D. Definir la estructura de directorios del proyecto.
E. Generar informes de construcción y documentación.

ans

155. **What is the result?**

```java
abstract class Animal {
  private void speak() {
    System.out.println("Animals make noise");
  }
  public static void main(String[] args) {
    Animal animal = new Dog();
    animal.speak();
  }
}
class Dog extends Animal {
  protected void speak() {
    System.out.println("The Dog barks");
  }
}
public class Test {
  public static void main(String... strings) {
    Animal.main(strings);
  }
}
```

A. Animals make noise
B. Compilation fails.
C. The Dog barks
D. A runtime exception is thrown.

ans

156. **¿Cuál de las siguientes afirmaciones sobre la palabra clave throw es verdadera?**
A. Se utiliza para detectar excepciones generadas por otros métodos.
B. Se utiliza para declarar que un método puede generar una excepción.
C. Se utiliza para definir el código de limpieza que se debe ejecutar.
D. Se utiliza para generar una excepción manualmente.

ans

157. **¿Cuál de las siguientes anotaciones es verdadera acerca de la siguiente jerarquía de clases?**

```
class Vehicle {
  String type = "Vehicle";
}
class Car extends Vehicle {
  String type = "Car";
}
class ElectricCar extends Car {
  String type = "ElectricCar";
}
```

A. `ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar`
B. `Car myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle`
C. `ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle`
D. `ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Car`

ans

158. **Which of the following statements correctly identifies checked and unchecked exceptions in Java?**
A. SQLException is an unchecked exception, and ArrayIndexOutOfBoundsException is a checked exception.
B. IOException is a checked exception, and NullPointerException is an unchecked exception.
C. RuntimeException is a checked exception, and FileNotFoundException is an unchecked exception.
D. ArithmeticException is a checked exception, and ClassNotFoundException is an unchecked exception.

ans

159. **¿Cuáles de las siguientes opciones son verdaderas acerca de APX?**
A. Ayuda a aumentar el uso de Mainframe.
B. Ninguna opción es correcta.
C. Ofrece una alternativa confiable para el desarrollo de transacciones no acopladas al canal.
D. Biblioteca de métodos para crear un punto de contacto entre las tecnologías Frontend y Backend.
E. Extensión de la Arquitectura Backend que brinda las mismas capacidades en el mundo distribuido.

ans

160. **¿Cuáles son los tipos de componentes en APX batch?**

A. Librerías
B. Jobs
C. Todas las opciones son correctas
D. DTOs
E. Transacciones

ans

161. **What is the output of the following program?**

```
1    public class Test {
2      public static void main(String[] args) {
3        int x = 0;
4        while (x++ < 5) {
5        }
6        String message = x > 5 ? "greater than" : "smaller than";
7        System.out.println(message + " - " + x);
8      }
9    }
```

A. smaller than - 6
B. greater than - 6
C. smaller than - 5
D. Compilation fails.
E. greater than - 5

ans

162. **Which of the following will be the output?**

```
public class Test implements Runnable {
  boolean flag = true;
  @Override
  public synchronized void run() {
    if (flag) {
      flag = false;
      System.out.print("1 ");
      try {
        this.wait();
      } catch (Exception e) {
      }
      System.out.print("2 ");
    } else {
      flag = true;
      System.out.print("3 ");
      try {
        Thread.sleep(2000);
      } catch (Exception e) {
      }
      System.out.print("4 ");
      notify();
    }
  }
  public static void main(String[] args) {
    Test w = new Test();
    new Thread(w).start();
    new Thread(w).start();
  }
}
```

A. The code outputs 1 3 4.
B. The code outputs 1 2 3 4.
C. The code outputs 3 4 1.
D. The code outputs 1 3 4 2.
E. The code never completes.
F. The code runs to completion.

ans

163. **¿Cuáles son palabras reservadas para manejo de excepciones en Java?**
A. throw & throws
B. Error, Exception, RuntimeException, Throwable
C. final, static, catch, throws
D. try, catch, finally, throw, throws
E. try, catch, final, throw, throws

ans

164. **¿Cuál de las siguientes implementaciones de código utiliza correctamente una clase genérica en Java?**

A. `public class Box { private Object value; public Box(Object value) { this.value = value; } public Object getValue() { return value; } }`

B. `public class Box { private int value; public Box(int value) { this.value = value; } public int getValue() { return value; } }`

C. `public class Box<T extends Number> { private T value; public Box(T value) { this.value = value; } public T getValue() { return value; } }`

D. `public class Box<T> { private T value; public Box(T value) { this.value = value; } public T getValue() { return value; } }`

ans

165. **¿Cuál de las siguientes opciones son capacidades ofrecidas en APX?**

A. Todas las opciones son correctas.

B. Procesamiento transaccional.

C. Procesamiento Batch.

D. Cold Deployment.

E. Sin integración con servicios de seguridad.

ans

166. **What will happen when you try to compile the following code?**

```
class Parent {
  protected void show() {
    System.out.println("Parent show");
  }
}
class Child extends Parent {
  @Override
  void show() {
    System.out.println("Child show");
  }
}
public class Test {
  public static void main(String[] args) {
    Child child = new Child();
    child.show();
  }
}
```

A. It will print "Parent show".

B. It will print "Child show".

C. Compilation error.

D. It will print "Parent show" followed by "Child show".

ans

167. **¿Cuáles de las siguientes opciones son archivos configurados por APX CLI al agregar JDBC Utility?**

A. library-arc-osgi.xml
B. library-arc.xml
C. config.xml
D. settings.xml
E. pom.xml

ans

168. **Con la "APX Operation Console", sólo se pueden ver grafos de dependencias para las librerías. Seleccione una:**

A. Verdadero
B. Falso

ans

169. **¿Cual opción es verdadera acerca de "APX Operation Console"?**

A. Herramienta web que proporciona a los desarrolladores y administradores gestionar la configuración, verificación de componentes de la aplicación o busqueda de jobs ejecutados o en ejecución.
B. Herramienta de línea de comandos para inicializar proyectos de tipo APX.
C. Herramienta para administrar las dependencias de un proyecto APX.
D. Herramienta para agregar los parámetros de entrada y salida de una transacción o job APX.
E. Ninguna opción es correcta.

ans

170. **¿Cuáles son los tipos de componentes en APX?**

A. Librerías
B. Jobs
C. Todas las opciones son correctas
D. DTOs
E. Transacciones

ans

171. **¿Qué significa que un cambio en el software sea retrocompatible?**

A. El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él.
B. El cambio puede romper la funcionalidad existente, requiriendo ajustes en el código que depende del software.
C. El cambio corrige errores menores y realiza mejoras de rendimiento.
D. El cambio introduce nuevas funcionalidades que no afectan el comportamiento existente del software.

ans

172. **¿Qué comando en Bash se utiliza para copiar archivos de un lugar a otro?**

A. copy
B. mv
C. cp
D. zip
E. duplicate

ans

173. **¿Cuál es el propósito principal de un objeto de transferencia de datos (DTO) en el diseño de software?**

A. Encapsular la lógica empresarial dentro de la aplicación.
B. Transferir datos entre diferentes capas o niveles de una aplicación.
C. Manejar operaciones de base de datos directamente.
D. Proporcionar una interfaz de usuario para la entrada de datos.

ans

174. **¿Cuál es la función principal de JobRepository en Spring Batch?**

A. Configurar y gestionar los recursos necesarios para la ejecución de un trabajo.
B. Proporcionar un mecanismo de persistencia para todas las operaciones CRUD relacionadas con JobLauncher, Job y Step.
C. Ejecutar los trabajos (Jobs) en paralelo para mejorar el rendimiento.
D. Realizar la validación de los datos antes de procesarlos en un trabajo.

ans

175. **What is the result?**

```
class Ot {
  enum Colors {
    RED, GREEN, BLUE, YELLOW, BLACK
  };
}
public class Test {
  public static void main(String[] String) {
    for (Ot.Colors c : Ot.Colors.values()) {
      if (Ot.Colors.RED.equals(c)) {
        System.out.print("red ");
      }
      if (c == Ot.Colors.GREEN) {
        System.out.print("green ");
      }
      if (c.equals("BLUE")) {
        System.out.print("blue ");
      }
    }
  }
}
```

A. red blue
B. green blue
C. red green blue
D. red green
E. An exception is thrown at runtime.
F. Compilation fails.

ans

176. **¿Cuál es el archivo generado por el comando 'apx config'?**

A. settings.yaml
B. config.xml
C. pom.xml
D. settings.xml
E. config.yaml

ans

177. **¿Cuál es la nomenclatura correcta para un componente de tipo transacción?**

A. TESTT001-01-MX
B. TESTT001-MX-01
C. TESTTT01-MX-01
D. TEST01-001-MX

ans

178. **Which of the following are valid ways to initialize an array in Java?**
A. `int[] arr = (1, 2, 3);`
B. `int[] arr = {1, 2, 3};`
C. `int[] arr = new int[] {1, 2, 3};`
D. `int[] arr = new int[3];`

ans

179. **¿Cuál es la nomenclatura y la ruta de creación de un archivo que sigue el formato [UUAA]T[CODE]-[VERSION]-[COUNTRY].xml en un proyecto?**
A. Ninguna opción es correcta.
B. [CODE]T[UUAA]-[VERSION]-[COUNTRY].xml y se crea en la ruta src/main/java/.
C. [UUAA]T[CODE]-[VERSION]-[COUNTRY].xml y se crea en la ruta src/main/resources/.
D. [VERSION]-[UUAA]T[CODE]-[COUNTRY].xml y se crea en la ruta src/main/resources/.
E. [COUNTRY]-[VERSION]-[CODE]T[UUAA].xml y se crea en la ruta src/test/resources/.

ans

180. **¿Cuál de las siguientes afirmaciones sobre las excepciones checked y unchecked es correcta?**

A. Las excepciones checked deben capturarse o declararse con `throws`.
B. Las excepciones unchecked deben declararse obligatoriamente con `throws`.
C. Todas las excepciones heredan directamente de `RuntimeException`.
D. Las excepciones checked nunca pueden capturarse con un bloque `catch`.

ans

181. **Dado el siguiente código:**

```java
interface Omega {}
class Gamma {}
class Epsilon extends Gamma implements Omega {}
public class Test {
  public static void main(String[] args) {
    Gamma g = new Epsilon();
    Omega o = (Omega) g;
    Epsilon e = (Epsilon) g;
    Omega m = e;
  }
}
```

¿Cuál es el resultado de la ejecución del programa?

A. An exception is thrown at runtime.
B. No option is correct.
C. Compilation fails.
D. The code runs with no output.

ans

182. **¿Cuáles son los protocolos físicos que admite en APX?**

A. SQL
B. JMS
C. Kermit
D. REST
E. HTTP

ans

183. **Which of the following are unchecked exceptions?**

A. ArrayIndexOutOfBoundsException
B. IOException
C. RuntimeException
D. NumberFormatException
E. IllegalArgumentException
F. InterruptedException

ans

184. **¿Cual de las siguientes opciones de vida de Maven se encarga de compilar el código fuente del proyecto?**

A. compile
B. package
C. test
D. validate

ans

185. **¿Cuál de las siguientes herramientas incluidas en el JDK se utiliza para monitorear y gestionar aplicaciones?**

A. VM
B. javac
C. jar
D. jconsole
E. java

ans

186. **¿Cuál de las siguientes herramientas proporciona análisis de calidad del código, detectando errores, vulnerabilidades y malas prácticas en el código fuente?**

A. Bitbucket
B. Jenkins
C. Jira
D. SonarQube
E. Artifactory

ans

187. **Dado el siguiente código:**

```
interface I {
  void method();
}
public class C implements I {
  // declaraciones de metodos
}
```

¿Qué método puede ser declarado dentro de la clase C para que compile?

A. `protected abstract void method() { }`
B. `private void method() { }`
C. `public abstract void method();`
D. Ninguna de las opciones.
E. `void method() { }`

ans

188. **¿Cuál de los siguientes métodos de Mockito se utilizan conjuntamente para definir el comportamiento de un mock cuando se llama a un método específico?**
A. doReturn()
B. Ninguna opción es correcta.
C. doThrow()
D. when() & thenReturn()
E. verify()

ans

189. **What is the result?**

```
abstract class Animal {
  public void speak() {
    System.out.println("Animals make noise");
  }
  public static void main(String[] args) {
    Animal animal = new Dog();
    animal.speak();
  }
}
class Dog extends Animal {
  protected void speak() {
    System.out.println("The Dog barks");
  }
}
public class Test {
  public static void main(String... strings) {
    Animal.main(strings);
  }
}
```

A. Compilation fails.
B. The Dog barks
C. A runtime exception is thrown.
D. Animals make noise

ans

190. **Which of the following lines of code correctly appends a string to a StringBuilder object in Java?**

A. `StringBuilder sb = new StringBuilder("Hello"); sb.add(" World");`
B. `StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");`
C. `StringBuilder sb = "Hello"; sb += " World";`
D. `StringBuilder sb = new StringBuilder("Hello"); sb.insert(" World");`

ans

191. **Which of the following statements about the `java -jar` command are true?**

A. The `java -jar` command can dynamically add libraries to the classpath using the `-cp` or `-classpath` options.
B. The `java -jar` command ignores the CLASSPATH environment variable and command-line classpath settings.
C. The `java -jar` command can execute a JAR file without specifying the main class if the JAR file contains multiple main classes.
D. The `java -jar` command requires the JAR file to have a `META-INF/MANIFEST.MF` file specifying the main class.
E. The `java -jar` command can only be used to execute JAR files that are executable JARs, i.e., those that contain the `META-INF/MANIFEST.MF` file with the `Main-Class` attribute.

ans



192. **Which two statements are true?**
A. An interface cannot be extended by another interface.
B. An interface can be extended by an abstract class.
C. An abstract class cannot be extended by an abstract class.
D. An abstract class can implement an interface.
E. An abstract class can be extended by a concrete class.

ans

193. **¿Cuál es el propósito principal de la anotación @Test en JUnit?**
A. Ninguna opción es correcta.
B. Marcar un método como un método de prueba.
C. Ejecutar un método antes de todos los métodos de prueba en una clase.
D. Ejecutar un método después de todos los métodos de prueba en una clase.
E. Ignorar un método de prueba.

ans

194. **Una transacción es un conjunto de operaciones que se ejecutan como un solo elemento. Una transacción debe tener propiedades ACID.**

A. Verdadero
B. Falso

ans

195. **¿Cuál es la rama principal de Gitflow en la que se integran las nuevas funcionalidades antes de lanzarlas a producción?**

A. feature
B. release
C. hotfix
D. develop
E. master

ans

196. **¿Cuál de los siguientes comandos de Maven se utiliza para compilar un proyecto y crear un archivo JAR pero sin instalarlo en el repositorio local?**

A. mvn install
B. mvn package
C. mvn clean
D. mvn compile

ans

197. **Which design pattern allows you to compose objects into tree structures to represent part-whole hierarchies, letting clients treat individual objects and compositions uniformly?**

A. Builder
B. Composite
C. Singleton
D. Abstract Factory
E. Facade

ans

198. **¿Cuál es el significado del acrónimo CRUD en el contexto del desarrollo de aplicaciones?**

A. Create, Read, Update, Delete.
B. Compile, Run, Update, Display.
C. Connect, Retrieve, Upload, Download.
D. Create, Retrieve, Upgrade, Delete.

ans

199. **What will be the output of the above code?**

```
import java.util.stream.Stream;
import java.util.Objects;
class Person {
  public String name;
  Person(String name) {
    this.name = name;
  }
  @Override
  public int hashCode() {
    return Objects.hash(name);
  }
  @Override
  public boolean equals(Object obj) {
    if (this == obj)
      return true;
    if (obj == null)
      return false;
    if (getClass() != obj.getClass())
      return false;
    Person other = (Person) obj;
    return Objects.equals(name, other.name);
  }
}
public class Test {
  public static void main(String[] args) {
    Person p1 = new Person("John");
    Person p2 = new Person("Jane");
    Person p3 = new Person("Jane");
    Person p4 = new Person("Mark");
    Stream<Person> stream = Stream.of(p1, p2, p3, p4);
    long count = stream.filter(p -> p.name.startsWith("J")).distinct
       ().count();
    System.out.println(count);}}
```

A. 2
B. A runtime exception is thrown
C. 0
D. 3
E. 4
F. 1
G. Compilation fails

ans

200. **¿Cuál es la principal función de JFrog Artifactory en un entorno de desarrollo de software?**

A. Proporcionar un entorno de desarrollo integrado (IDE) para aplicaciones Java.
B. Ofrecer un sistema de control de versiones para proyectos de software.
C. Actuar como un servidor web para alojar sitios HTML y CSS.
D. Gestionar y almacenar artefactos de software, como dependencias y bibliotecas, en un repositorio.

ans

201. **Which of the following lines of code correctly defines the main method in a Java program?**

A. `static void main(String args[]) {}`
B. `public static int main(String[] args) {}`
C. `public static void main(String... String) {}`
D. `public static void main(String[] args) {}`
E. `private static void main(String[] args) {}`

ans

202. **What will be the result of attempting to compile and run the following code snippet?**

```
1    public class RecursiveConstructorTest {
2      private int value;
3      public RecursiveConstructorTest() {
4        this(10);
5        System.out.println("No-argument constructor called");
6      }
7      public RecursiveConstructorTest(int value) {
8        this();
9        this.value = value;
10       System.out.println("Constructor with int argument called: " +
             value);
11     }
12     public static void main(String[] args) {
13       RecursiveConstructorTest obj = new RecursiveConstructorTest();
14     }
15   }
```

A. Compilation error due to recursive constructor calls.
B. Runtime error due to infinite recursion.
C. No-argument constructor called. Constructor with int argument called: 10.
D. Constructor with int argument called: 10. No-argument constructor called.

ans

203. **What will be the output of the following code?**

```java
abstract class Vehicle {
  public abstract void start();
  public void stop() {
    System.out.println("Vehicle stopped");
  }
}
class Car extends Vehicle {
  @Override
  public void start() {
    System.out.println("Car started");
  }
}
class Bike extends Vehicle {
  @Override
  public void start() {
    System.out.println("Bike started");
  }
}
public class Test {
  public static void main(String[] args) {
    Vehicle car = new Car();
    Vehicle bike = new Bike();
    car.start();
    car.stop();
    bike.start();
    bike.stop();
  }
}
```

A. Car started Vehicle stopped Bike started Vehicle stopped
B. Vehicle stopped Car started Vehicle stopped Bike started
C. Bike started Vehicle stopped Car started Vehicle stopped
D. Car started Vehicle stopped Vehicle stopped Bike started

ans

204. **¿Cuáles son los tipos de componentes en APX Online?**

A. Librerías
B. Jobs
C. Todas las opciones son correctas
D. DTOs
E. Transacciones

ans

205. **¿Cuál es el patrón de diseño que permite a un objeto alterar su comportamiento cuando su estado interno cambia?**
A. Facade Pattern (Fachada)
B. Strategy Pattern
C. Singleton Pattern
D. Builder Pattern
E. State Pattern

ans

206. **Which two statements are true?**
A. A class that is abstract cannot be instantiated or turned into an object (instance).
B. An interface cannot be extended by another interface.
C. If a class has abstract methods, then it must be specified as an abstract class.
D. An abstract class can be extended by an interface.

ans

207. **¿Cuál de las siguientes herramientas es la más adecuada para el seguimiento de problemas y la gestión de proyectos en entornos de desarrollo de software?**

A. Jenkins
B. Jira
C. Artifactory
D. SonarQube
E. Bitbucket

ans

208. **¿Cuáles son los objetivos de la clase abstracta de la transacción APX?**
A. Recuperar información de solicitud HTTP.
B. Agregar parámetros del cuerpo de la respuesta.
C. Todas las opciones son correctas.
D. Unir las operaciones relacionadas con los parámetros de entrada y salida de la lógica de la transacción.
E. Separar las operaciones relacionadas con los parámetros de entrada y salida de la lógica de la transacción.

ans

209. **¿Cuál es el propósito del plugin `maven-surefire-plugin` y en qué etapa del ciclo de vida de Maven se ejecuta principalmente?**
A. Instalar el artefacto en el repositorio local durante la etapa 'install'.
B. Empaquetar el proyecto durante la etapa 'package'.
C. Ejecutar pruebas unitarias durante la etapa de 'test'.
D. Generar la documentación del proyecto durante la etapa de 'site'.

ans

210. **¿Cuál es el propósito principal de OSGi (Open Service Gateway Initiative) en el desarrollo de software moderno?**
A. Gestionar dependencias y permitir el despliegue dinámico de módulos en aplicaciones Java.
B. Proporcionar un entorno de desarrollo integrado (IDE) para aplicaciones Java.
C. Servir como un sistema de control de versiones para proyectos de software.
D. Actuar como un servidor de aplicaciones para ejecutar aplicaciones empresariales Java.

ans

211. **¿Cuál será el resultado del siguiente fragmento de código?**

```
class Animal {
  public void makeSound() {
    System.out.println("Animal sound");
  }
}
class Dog extends Animal {
  @Override
  public void makeSound() {
    System.out.println("Bark");
  }
}
public class Test {
  public static void main(String[] args) {
    Animal myDog = new Dog();
    myDog.makeSound();
  }
}
```

A. Animal sound
B. Error de ejecución
C. Error de compilación
D. Bark

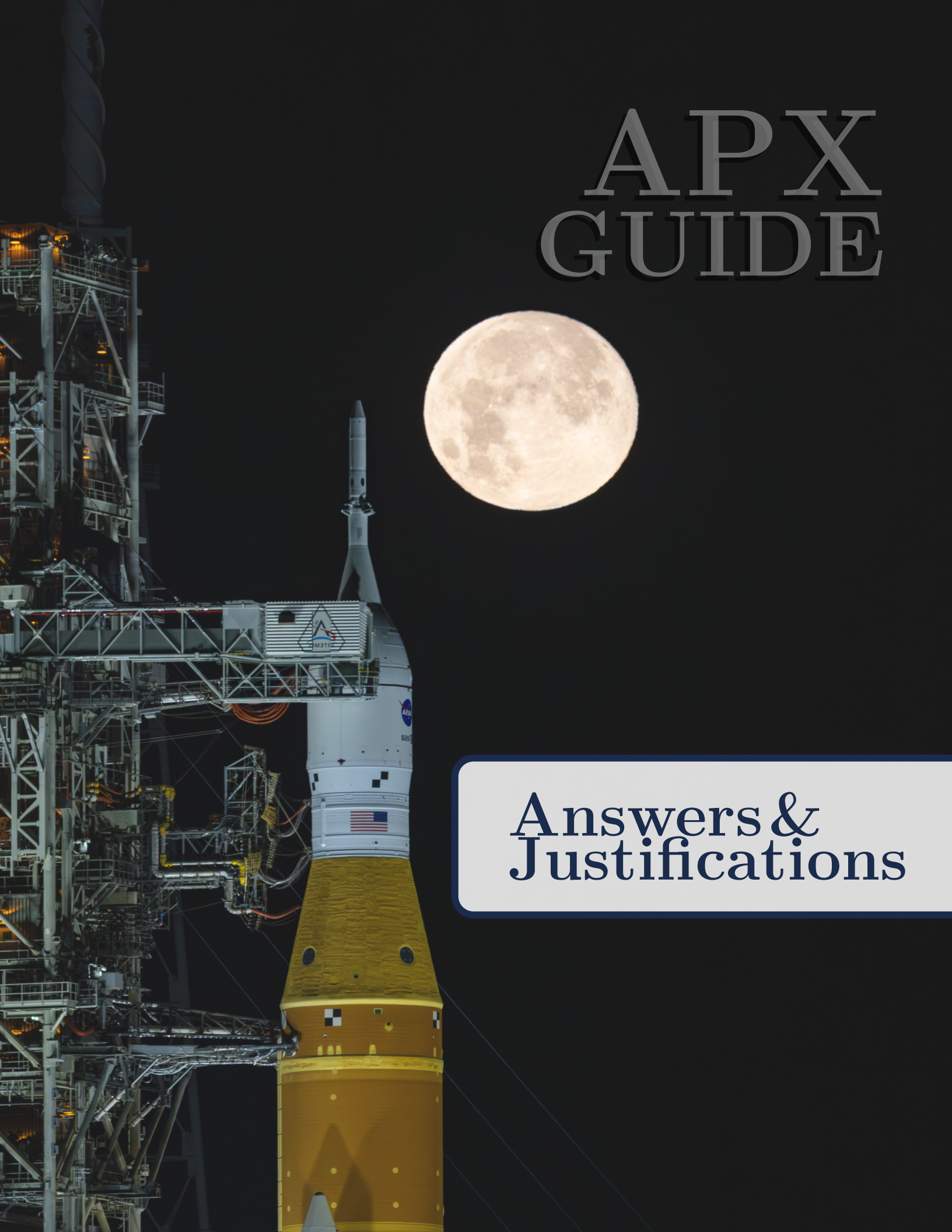
ans

212. **What are the modifiers that can precede a toplevel class declaration?**

A. protected
B. abstract
C. strictfp
D. static
E. private
F. public
G. All the options are correct.
H. final

ans

APX GUIDE



Answers & Justifications

Answers and Justifications

Respuesta de la Pregunta 1
Respuestas correctas: **C.** Checked, **D.** Unchecked.
Justificación:
En Java, las excepciones se dividen principalmente en dos tipos:

- **Checked Exceptions (comprobadas):** Se verifican en tiempo de compilación. El compilador obliga a manejarlas con `try-catch` o declararlas con `throws`. Ejemplos: `IOException`, `SQLException`.
- **Unchecked Exceptions (no comprobadas):** Se verifican en tiempo de ejecución. Son subclases de `RuntimeException`. No es obligatorio capturarlas. Ejemplos: `NullPointerException`, `ArrayIndexOutOfBoundsException`.

Por qué no las otras: `Object` es la clase base, pero no está directamente relacionada con excepciones. `Throwable` es una clase raíz, no una categoría práctica. BACK

Respuesta de la Pregunta 2
Respuestas correctas: **A.** git branch, **B.** git merge, **D.** git checkout.
Justificación:

- `git branch` → Se usa para crear, listar o eliminar ramas.
- `git merge` → Se usa para fusionar ramas.
- `git checkout` → Se utiliza para cambiar de rama o restaurar archivos.

Por qué no las otras: `git init` inicializa un repositorio, no gestiona ramas. `git commit` guarda cambios en el historial, pero no maneja ramas directamente. BACK

Respuesta de la Pregunta 3
Respuesta correcta: **D.** Representa cambios importantes o incompatibles con versiones anteriores, que pueden afectar la compatibilidad.
Justificación:
En Semantic Versioning (SemVer), la versión se define como MAJOR.MINOR.PATCH:

- **MAJOR (primer dígito):** Se incrementa cuando hay cambios incompatibles que pueden romper la compatibilidad con versiones anteriores (Ejemplo: 1.x.x → 2.0.0).
- **MINOR (segundo dígito):** Añade nuevas funcionalidades compatibles hacia atrás.
- **PATCH (tercer dígito):** Corrige errores sin cambios de compatibilidad.

BACK

Respuesta de la Pregunta 4
Respuesta correcta: **D.** URL, headers, cuerpo de la solicitud.

Justificación:
Una solicitud HTTP está compuesta por:

- **URL (o línea de solicitud):** Indica el recurso al que se quiere acceder.
- **Headers (encabezados):** Contienen metadatos como tipo de contenido, autenticación, cookies, etc.
- **Cuerpo (body):** Opcional; se utiliza principalmente en métodos como POST o PUT para enviar datos al servidor.

Por qué no las otras: Los scripts/funciones no forman parte de una solicitud. Los archivos adjuntos no son componentes formales de la solicitud. El estado pertenece a la respuesta HTTP, no a la solicitud. BACK

Respuesta de la Pregunta 5
Respuesta correcta: **A.** Una aplicación o componente que se comunica con otras aplicaciones a través de la red utilizando protocolos estándar.
Justificación:
Un servicio web es un componente de software que permite la comunicación e interoperabilidad entre distintas aplicaciones a través de la red utilizando protocolos estándar como HTTP, SOAP, REST, XML, JSON, etc. No depende del lenguaje de programación ni de la plataforma. BACK

Respuesta de la Pregunta 6
Respuesta correcta: **B.** El ciclo for mejorado se puede utilizar con cualquier tipo de colección y matriz.
Justificación:
El ciclo for mejorado (introducido en Java 5) puede usarse tanto con arreglos (matrices) como con colecciones que implementen la interfaz `Iterable` (por ejemplo: `ArrayList`, `HashSet`). No permite modificar directamente la colección durante la iteración (para eso se usa `Iterator`). BACK

Respuesta de la Pregunta 7
Respuesta correcta: **D.** Organización de datos en tablas con filas y columnas.
Justificación:
En una base de datos relacional (RDBMS), los datos se organizan en tablas (relaciones) compuestas por filas (tuplas) y columnas (atributos). Se utilizan claves primarias y foráneas para establecer relaciones entre las tablas.
Por qué no las otras: JSON es propio de bases NoSQL orientadas a documentos; Nodos y relaciones son de bases gráficas; Archivos distribuidos se relacionan con sistemas como Hadoop. BACK

Respuesta de la Pregunta 8
Respuesta correcta: **C.** pom.xml
Justificación:

En proyectos Maven, el archivo `pom.xml` (Project Object Model) se utiliza para declarar dependencias, como librerías externas. Maven las descarga automáticamente desde repositorios centrales.
Por qué no las otras: `settings.xml` es un archivo de configuración global de Maven, pero no sirve para declarar dependencias de un proyecto específico. `config.yaml` o `pom.yaml` no aplican. BACK

Respuesta de la Pregunta 9
Respuesta correcta: C. A java.lang.StackOverflowError is thrown.
Justificación:
El método `go(int x)` contiene una llamada recursiva `go(x)` sin una condición de parada (caso base). El valor de `x` va cambiando, pero la recursión nunca se detiene. Esto agota la memoria de la pila de ejecución, provocando un `StackOverflowError` en tiempo de ejecución. BACK

Respuesta de la Pregunta 10
Respuesta correcta: A. Child
Justificación:
Este ejercicio evalúa el uso de *casting* explícito (*downcasting*) para cambiar el tipo de referencia que evalúa el compilador y cómo esto resuelve las restricciones de visibilidad de métodos.
1. El problema de visibilidad original: En la clase `Parent`, el método `display()` es `private`. *Si intentáramos llamar a* `obj.display()` *directamente con una referencia de tipo* `Parent`, *el compilador arrojaría un error* de acceso porque los métodos privados no son visibles fuera de su propia clase.
2. Intervención del compilador mediante Casting: Al envolver el objeto con el *downcasting* explícito en `((Child) obj)`, le estamos instruyendo al compilador que cambie temporalmente su lente y evalúe la llamada basándose en la clase `Child`, no en `Parent`.
3. Resolución exitosa: El compilador ahora busca el método `display()` en la clase `Child`. Como `Child` tiene su propia versión del método declarada explícitamente como `public`, la validación de acceso es exitosa y el programa compila. En tiempo de ejecución, la JVM confirma que el objeto real en memoria es un `Child`, ejecuta su método e imprime **Child**.
Nota: Contrastar con el ejercicio 21 BACK

Respuesta de la Pregunta 11
Respuesta correcta: D. El patrón Singleton asegura que una clase tiene solo una instancia y provee un punto global de acceso a ella.
Justificación:
El patrón creacional Singleton garantiza que una clase se instancie una sola vez durante toda la ejecución de la aplicación, proporcionando un punto de acceso global (usualmente a través de un método `static` como `getInstance()`). BACK

Respuesta de la Pregunta 12
Respuesta correcta: C. 3
Justificación:
El flujo del Stream es el siguiente:



1. Lista original: `[1, 2, 3, 4, 5]`
2. Después de `map(n -> n * 2)`: `[2, 4, 6, 8, 10]`
3. Después de `filter(n -> n > 5)`: `[6, 8, 10]`
4. Al final, `count()` cuenta los elementos restantes, que son 3.

BACK

Respuesta de la Pregunta 13
Respuesta correcta: **C.** `apx send req -name="TESTT001-01-ES-test.json" -y -details`
Justificación:
En APX, el comando `apx send req` se utiliza para enviar una solicitud de transacción. El flag `-name` especifica el archivo de la solicitud, `-y` confirma la acción y `-details` muestra información detallada. Los otros comandos (`show`, `add`, `check`) cumplen funciones distintas como mostrar, registrar o reparar.

BACK

Respuesta de la Pregunta 14
Respuesta correcta: **F.** BCF
Justificación:
En Java, la declaración `switch` ejecuta las sentencias de los casos siguientes ("fall-through") si no encuentra una instrucción `break`. Como `grade` es `'B'`, coincide con `case 'B':` e imprime "B". Al no haber `break`, continúa al `case 'C':` (imprime "C") y luego a los siguientes casos hasta el final, imprimiendo "F". Salida total: `BCF`.

BACK

Respuesta de la Pregunta 15
Respuesta correcta: **C.** Static block Static method Instance block Constructor
Justificación:
El orden de inicialización en Java es estricto:

1. **Bloques estáticos:** Se ejecutan una sola vez cuando la clase se carga en memoria (`"Static block"`).
2. **Métodos estáticos:** Se ejecuta al llamarlo en `main` (`"Static method"`).
3. **Bloques de instancia:** Se ejecutan cada vez que se crea un nuevo objeto con `new`, antes del constructor (`"Instance block"`).
4. **Constructor:** Se ejecuta al final de la instanciación (`"Constructor"`).

BACK

Respuesta de la Pregunta 16
Respuesta correcta: **B.** APX Operation Console

Justificación:
En APX Online, la APX Operation Console es la herramienta donde se realiza la configuración y administración de la aplicación. Permite visualizar el estado de proyectos, configurar parámetros de ejecución, y monitorear transacciones y logs.

BACK

Respuesta de la Pregunta 17
Respuesta correcta: **A.** false, true, false
Justificación:

- `str1 == str2` es **false** porque `str1` apunta al objeto en el Heap (creado con `new String`), mientras `str2` apunta al literal en el String Pool.
- `str2 == str3` es **true** porque el método `intern()` devuelve la referencia al String literal del String pool, coincidiendo con `str2`.
- `str1 == str3` es **false** porque `str1` sigue apuntando al objeto en el Heap, distinto del literal.

BACK

Respuesta de la Pregunta 18
Respuesta correcta: **B.** Blob
Justificación:
El patrón Blob (también conocido como "God Object") es un antipatrón de diseño que ocurre cuando una sola clase o módulo concentra demasiadas responsabilidades y funcionalidades, generando un acoplamiento excesivo y dificultando su mantenimiento.

BACK

Respuesta de la Pregunta 19
Respuesta correcta: **B.** Hello
Justificación:
En Java, los `String` son inmutables y las referencias se pasan por valor. Al llamar a `op1(msg)`, la variable `s` es una copia de la referencia. Al hacer `s += new String("Java")`, se crea un nuevo objeto String asignado a `s` de manera local en el método, pero no altera la referencia original `msg` en `stringTest()`.

BACK

Respuesta de la Pregunta 20
Respuesta correcta: **E.** [Mark]
Justificación:
Esta pregunta evalúa el uso de flujos de datos (*Streams*), funciones lambda y expresiones de predicados funcionales en Java para filtrar colecciones.
1. Inicialización de datos y método estático: El método `main` crea una lista inmutable con tres objetos de tipo `Person`: `John`, `Jane` y `Mark`. Luego invoca al método estático `filter`, pasándole la lista completa y una expresión lambda como predicado: `p -> !p.name.startsWith("J")`.



2. Evaluación del predicado: La condición del predicado evalúa que el nombre de la persona **no** empiece con la letra `"J"` (utilizando el operador de negación `!` combinado con `startsWith("J")`):

- Para **John**: `startsWith("J")` es verdadero, al negarlo da **falso** (descartado).
- Para **Jane**: `startsWith("J")` es verdadero, al negarlo da **falso** (descartado).
- Para **Mark**: `startsWith("J")` es falso, al negarlo da **verdadero** (**conservado**).

3. Resultado del flujo: La operación terminal `toList()` recolecta los elementos que superaron el filtro en una nueva lista y los imprime llamando implícitamente al método `toString()` sobrescrito en la clase `Person`, obteniendo como salida [**Mark**]. BACK

Respuesta de la Pregunta 21
Respuesta correcta: C. Child
Justificación:
Esta pregunta evalúa el polimorfismo dinámico en Java y el concepto de sobrescritura de métodos (*Method Overriding*). A diferencia del ejercicio 10, aquí sí existe una relación de sobrescritura legítima.
1. Validación en Tiempo de compilación: Al evaluar la instrucción `obj.display();`, el compilador se guía exclusivamente por el tipo de la referencia, que en este caso es `Parent`. Como la clase `Parent` tiene un método `display()` declarado como `public`, la llamada es completamente válida y el programa compila con éxito.
2. Sobrescritura real (*Override*): La clase `Child` hereda de `Parent` y define un método exactamente con la misma firma (`display()`) y visibilidad (`public`). Al no haber restricciones de acceso como `private`, la clase hija oculta exitosamente el comportamiento de su clase padre.
3. Despacho dinámico de métodos (*Dynamic Method Dispatch*): En tiempo de ejecución, la Máquina Virtual de Java (JVM) delega la ejecución basándose en el tipo real del objeto instanciado en memoria, no en su referencia. Como el objeto fue instanciado con `new Child()`, la JVM invoca la versión más específica del método, imprimiendo **Child** en la consola.

Nota: Contrastar con el ejercicio 10. BACK

Respuesta de la Pregunta 22
Respuesta correcta: D. Constructor with int argument: 10 No-argument constructor Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15 No-argument calculate method
Justificación:
La creación de `obj1` invoca al constructor vacío, el cual llama inmediatamente a `this(10)`. Esto ejecuta primero el constructor con `int` y al regresar finaliza el constructor por defecto. Luego, las llamadas a `calculate()` y `calculate(5)` imprimen sus mensajes. Posteriormente, `obj2` llama directamente al constructor con parámetro (imprimiendo el de 20), seguido por sus dos llamadas a `calculate`. El flujo corresponde a la opción D. BACK

Respuesta de la Pregunta 23
Respuesta correcta: C. Dictionary
Justificación:



Java Collections Framework (JCF) está compuesto por interfaces y clases modernas como `List` (`ArrayList`), `Set` y `Map`. La clase `Dictionary` es una estructura antigua (introducida en Java 1.0) y no es considerada parte del framework de colecciones moderno; fue reemplazada hace mucho tiempo por la interfaz `Map`. BACK

Respuesta de la Pregunta 24
Respuesta correcta: **D.** mvn package
Justificación:
Este reactivo evalúa el conocimiento sobre las fases del ciclo de vida de construcción (*build lifecycle*) en Maven y hasta dónde llega cada comando.
Opción D (Correcta - mvn package): Este comando ejecuta todas las fases previas (validación, compilación y pruebas) y finalmente empaqueta el código en su formato distribuible (como un archivo JAR o WAR) depositándolo únicamente dentro de la carpeta `target` del proyecto. Se detiene justo antes de la fase de instalación.
Opción A (mvn compile): Es una fase muy temprana del ciclo. Este comando se limita exclusivamente a compilar el código fuente principal (generando los archivos `.class`), pero no ejecuta las pruebas unitarias ni empaqueta el proyecto en un JAR.
Opción B (mvn clean): Este comando pertenece a un ciclo de vida distinto (el ciclo *clean*). Su única función es limpiar el proyecto eliminando la carpeta `target` y todos los binarios generados en compilaciones anteriores. No compila ni empaqueta nada.
Opción C (mvn install): Va exactamente un paso más allá que `package`. Una vez que el archivo JAR ha sido empaquetado en la carpeta `target`, este comando lo copia y lo instala de manera persistente en tu repositorio local de Maven (generalmente en la carpeta oculta `.m2` de tu computadora). Esto se hace para que el JAR quede disponible como dependencia para otros de tus proyectos locales. BACK

Respuesta de la Pregunta 25
Respuesta correcta: **E.** double
Justificación:
Java aplica reglas de promoción numérica automática. Al evaluar una expresión aritmética con diferentes tipos de datos primitivos, el tipo con menor capacidad se promueve al tipo de mayor capacidad para no perder precisión en el cálculo. Al restar un `short` de un `double`, el `short` se promueve implícitamente a `double`, por lo que el resultado de la expresión es `double`. BACK

Respuesta de la Pregunta 26
Respuesta correcta: **D.** x = 4
Justificación:
La variable estática `b` se inicializa automáticamente en `false`. El primer bloque `if (b)` se salta. La condición `else if (b = false)` contiene una asignación de variable en lugar de una comparación (==), por lo que a `b` se le asigna `false` y la expresión se evalúa como falso. El siguiente `else if (b)` también es falso. Finalmente, se ejecuta el bloque `else`, asginando `x = 4`. BACK

Respuesta de la Pregunta 27
Respuesta correcta: **B.** Ninguna de las anteriores.
Justificación:
Para probar, documentar y desarrollar APIs mediante la simulación de solicitudes HTTP y el análisis de sus respuestas, se utilizan comúnmente herramientas como Postman, Insomnia o SoapUI. Ninguna de las opciones mostradas es para este propósito: Git (versionado), Artifactory (repositorio), Docker (contenedores) y Maven (gestor de dependencias). BACK

Respuesta de la Pregunta 28
Respuesta correcta: **A.** Establece la conexión exitosamente e imprime "Connection established successfully" (o **D.** Falla al establecer la conexión e imprime "Failed to establish connection." en caso de no conectar).
Justificación:
Si la base de datos existe, el driver está presente y las credenciales son válidas, la conexión se establecerá imprimiendo el mensaje de éxito. Si hay un problema de acceso o red, `DriverManager.getConnection` lanzará una `SQLException`, entrando en el bloque `catch` e imprimiendo la falla. No arrojará excepciones ajenas como `NullPointerException` en estas condiciones. BACK

Respuesta de la Pregunta 29
Respuesta correcta: **D.** Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John
Justificación: Esta pregunta evalúa la sobrecarga de constructores, el uso de la llamada explícita a otro constructor mediante `this(...)`, la sobrecarga de métodos y el orden de ejecución en la pila de llamadas de Java.
1. Creación de `obj1` con el constructor sin argumentos: Al ejecutar `new OverloadedExample()`, se invoca el constructor sin argumentos. Su primera instrucción es `this("Default")`, lo que transfiere el control de flujo inmediatamente al constructor que recibe un `String`. Este constructor secundario se ejecuta por completo primero, imprimiendo **Constructor with argument called: Default**. Solo cuando este finaliza, el control regresa al constructor original sin argumentos para terminar su bloque, imprimiendo **No-argument constructor called**.
2. Invocación de métodos sobrecargados en `obj1`:

- `obj1.display()` imprime **Name: Default**.
- `obj1.display("Hello")` invoca al método sobrecargado que acepta un prefijo, imprimiendo **Hello:Default** (sin espacios adicionales).

3. Creación de `obj2` y sus llamadas: Al ejecutar `new OverloadedExample("John")`, se invoca directamente el constructor con un argumento, imprimiendo **Constructor with argument called: John**. Después se ejecutan de manera secuencial las llamadas a `display()`, imprimiendo **Name: John** y finalmente **Hi:John**.

Respuesta de la Pregunta 30
Respuesta correcta: **D.** ...
Justificación:

En Java, los *varargs* (argumentos de longitud variable) se indican utilizando la sintaxis de tres puntos consecutivos (`...`) después del tipo de dato. Por ejemplo: `int... numbers`. Esto permite que el método reciba un número indeterminado de argumentos de un mismo tipo, iterándolos internamente como un arreglo estándar. BACK

Respuesta de la Pregunta 31
Respuesta correcta: **C.** 5000
Justificación:
Dentro de las buenas prácticas y las utilidades JDBC provistas por el ecosistema APX, el límite estándar de registros devueltos por una consulta antes de que el framework requiera la implementación de un mecanismo de paginación es de 5000 elementos, ayudando a preservar la memoria del sistema. BACK

Respuesta de la Pregunta 32
Respuesta correcta: **B.** It will print "Base constructor with message: Hello" followed by "Derived constructor".
Justificación:
Cuando se crea una instancia de la clase `Derived`, su constructor comienza a ejecutarse. La primera instrucción es `super("Hello")`, la cual llama explícitamente al constructor de la clase `Base` que recibe el `String`. Eso imprime "Base constructor with message: Hello". Luego de que finaliza el bloque del padre, el constructor hijo continúa, imprimiendo "Derived constructor". BACK

Respuesta de la Pregunta 33
Respuesta correcta: **C.** Singleton Pattern
Justificación:
El Singleton es un patrón de diseño creacional cuyo único propósito es garantizar que una clase posea una sola y única instancia a lo largo de toda la memoria de la aplicación, y proporcionar un punto de acceso global (como un método `static getInstance()`) para obtener esa instancia siempre que se necesite. BACK

Respuesta de la Pregunta 34
Respuesta correcta: **E.** DTOS
Justificación:
En el patrón arquitectónico, los DTOs (Data Transfer Objects) son objetos o *Java Beans* que representan entidades del negocio o del dominio. Se usan exclusivamente para transportar estado e información entre las distintas capas (backend, servicios, etc.) sin introducir reglas de lógica empresarial o comportamientos complejos. BACK

Respuesta de la Pregunta 35



Respuesta correcta: **C.** Ambas, dependiendo del contexto.
Justificación:
Este reactivo evalúa la diferencia entre las palabras reservadas `throw` y `throws`.

Opción A: Inválida. `throw` lanza una excepción, pero no se utiliza para declarar que un método puede propagarla.

Opción B: Inválida. `throws` declara las excepciones que un método puede propagar, pero no lanza una excepción.

Opción C: Correcta. `throw` se utiliza para lanzar explícitamente una excepción, mientras que `throws` se emplea en la firma de un método para declarar las excepciones que este puede propagar.

Opción D: Inválida. La palabra reservada `finally` se relaciona con la ejecución de un bloque al finalizar un `try`, pero no se utiliza para propagar excepciones. BACK

Respuesta de la Pregunta 36
Respuesta correcta: **C.** Para definir las dependencias del proyecto, la configuración de compilación y los complementos.
Justificación:
El archivo `pom.xml` (Project Object Model) es el corazón de cualquier proyecto en Maven. Su función es detallar los identificadores del proyecto, gestionar qué librerías o dependencias externas necesita descargar, y configurar la forma en la que debe compilarse utilizando diversos plugins de construcción. BACK

Respuesta de la Pregunta 37
Respuesta correcta: **E.** jar
Justificación:
Este reactivo evalúa el conocimiento sobre las utilidades de línea de comandos base que provee el JDK para el ciclo de vida del desarrollo.
Opción E (Correcta - jar): Es la herramienta de archivo (*Java Archive Tool*). Se utiliza específicamente para agrupar múltiples archivos de clases compiladas (`.class`), recursos y metadatos en un único archivo comprimido con extensión `.jar`, facilitando su distribución e implementación.
Opción A (jconsole): Inválida. Es una herramienta gráfica de monitoreo y gestión (basada en JMX). Sirve para observar en tiempo real el rendimiento, el consumo de memoria y la actividad de los hilos de una Máquina Virtual de Java en ejecución.
Opción B (java): Inválida. Es el lanzador (*launcher*) o intérprete. Su función exclusiva es arrancar la JVM y ejecutar un programa a partir de sus archivos binarios compilados, pero no empaqueta nada.
Opción C (jps): Inválida. `jps` (*Java Virtual Machine Process Status Tool*) es una utilidad de diagnóstico ligera que lista en la terminal todos los procesos o instancias de la JVM que se están ejecutando actualmente en la computadora local.
Opción D (javac): Inválida. Es el compilador de Java. Su trabajo ocurre un paso antes: toma el código fuente escrito por el programador (`.java`) y lo traduce al *bytecode* (`.class`) que la JVM puede entender. BACK



Respuesta de la Pregunta 38
Respuesta correcta: **D.** `A ob = new A(); B b = (B) ob;`
Justificación:
Una `ClassCastException` ocurre durante el *downcasting* si intentas decirle a la JVM que trate un objeto como una clase hija de la cual no fue instanciado. En la opción D, `ob` es netamente un objeto instanciado desde la clase padre `A`, por lo tanto, forzar su casting a `B` rompe las reglas de tipo y lanza la excepción en ejecución.
BACK

Respuesta de la Pregunta 39
Respuestas correctas: **A.** `&&`, **B.** `|`, **D.** `||`, **E.** `==`
Justificación:
Los datos tipo booleanos soportan evaluación lógica con cortocircuito (AND lógico `&&` y OR lógico `||`), evaluación a nivel de bits (como `|` que examina ambos lados de la expresión) y validaciones relacionales de igualdad (`==` o `!=`). El operador `+` lanza un error de compilación al aplicarse directamente a un boolean.
BACK

Respuesta de la Pregunta 40
Respuesta correcta: **B.** Count: 5
Justificación:
El iterador `i` de nuestro bucle `for` corre desde el número 0 y llega hasta el 9 (debido a la instrucción estricta de menor a 10). La condición if hace módulo con 2 para encontrar valores pares. En dicho rango (0, 1, ..., 9), encontramos cinco números pares (0, 2, 4, 6, 8). Cada uno de estos incrementa el contador en 1, arrojando 5 en consola.
BACK

Respuesta de la Pregunta 41
Respuesta correcta: **D.** Compilation fails.
Justificación:
Este reactivo evalúa el funcionamiento del operador condicional (`?:`) y las reglas de compatibilidad de tipos en Java.
Opción A: Inválida. El programa no llega a ejecutarse, ya que presenta un error de compilación.
Opción B: Inválida. Aunque al finalizar el ciclo `while` la variable `x` vale 6, el operador ternario intenta devolver un `String` en una rama y un `boolean` en la otra, lo cual no está permitido.
Opción C: Inválida. Además del error de compilación, el valor final de `x` no sería 5, sino 6.
Opción D: Correcta. La expresión `x > 5 ? "greater than" : false`, no compila porque las dos ramas del operador ternario deben ser de tipos compatibles. En este caso, una devuelve un `String` y la otra un `boolean`.

Opción E: Inválida. Aunque la condición `x > 5` sería verdadera, el programa no compila debido a la incompatibilidad de tipos en el operador ternario.
BACK

Respuesta de la Pregunta 42
Respuesta correcta: **D.** Es una interfaz de línea de comandos para realizar operaciones sobre componentes APX.



Justificación:
CLI significa Command Line Interface. En el ecosistema APX, es la herramienta de terminal que permite a los desarrolladores administrar proyectos, configurar componentes, enviar solicitudes de ejecución y automatizar tareas sin necesidad de usar una interfaz gráfica (web). BACK

Respuesta de la Pregunta 43
Respuesta correcta: **B.** The Singleton pattern ensures that a class has only one instance and provides a global point of access to it
Justificación:
La definición estricta del patrón Singleton es restringir la instanciación de una clase a un solo objeto en todo el ciclo de vida de la aplicación de Java, y proveer una vía estática y global para que cualquier parte del código interactúe con esa única instancia en memoria. BACK

Respuesta de la Pregunta 44
Respuesta correcta: **B.** "Compila, empaqueta y ejecuta pruebas del componente en el que está ubicado" y **C.** "Despliega el componente APX en el enfoque APX Online en la región de España"
Justificación:
Al analizar la instrucción `apx deploy local`, los parámetros que le siguen indican la configuración exacta: `-runtime online` especifica el entorno destino (APX Online) y `-country=ESP` especifica que la lógica o recurso aplica para la región de España. BACK

Respuesta de la Pregunta 45
Respuesta correcta: **A.** MongoDB
Justificación:
MongoDB es el estándar en la industria para bases de datos NoSQL basadas en documentos. Almacena la información usando una estructura binaria de JSON llamada BSON, lo cual otorga extrema flexibilidad de esquema. Neo4j es para grafos, Cassandra es columnar, y Redis es clave-valor. BACK

Respuesta de la Pregunta 46
Respuesta correcta: **A.** Realizar un refactoring del diseño entre las librerías involucradas moviendo el comportamiento a sus librerías relacionadas.
Justificación:
El antipatrón "Blob" centraliza demasiada lógica en una clase gigantesca. La única manera de solucionarlo arquitectónicamente es refactorizando el diseño: extraer responsabilidades y segregarlas en módulos o clases especializadas más pequeñas (alta cohesión), disminuyendo así el acoplamiento. BACK

Respuesta de la Pregunta 47

Respuesta correcta: **A.** Proporcionar una plataforma para desplegar, ejecutar y gestionar aplicaciones empresariales Java.
Justificación:
JBoss (ahora conocido como WildFly) es un Servidor de Aplicaciones desarrollado en Java que implementa la especificación completa de Java EE (Jakarta EE). Permite alojar, gestionar y escalar aplicaciones distribuidas a nivel empresarial. BACK

Respuesta de la Pregunta 48
Respuesta correcta: **D.** `git checkout -b nombre_de_rama`
Justificación:
El parámetro `-b` en `git checkout` es un atajo muy popular. Combina la acción de crear una nueva rama a partir del commit actual (`git branch`) con el movimiento inmediato de tu espacio de trabajo hacia ella (`git checkout`). BACK

Respuesta de la Pregunta 49
Respuestas correctas: **A.** IOException, **C.** InterruptedException
Justificación:
Las Excepciones Chequeadas (Checked) en Java son aquellas de las cuales no hereda `RuntimeException`, obligando al desarrollador a manejarlas en tiempo de compilación mediante un `try-catch`. Tanto `IOException` (error de entrada/salida) como `InterruptedException` (interrupción de hilos) son excepciones obligatorias. BACK

Respuesta de la Pregunta 50
Respuesta correcta: **C.** pom.xml
Justificación:
El archivo `pom.xml` es la piedra angular de cualquier proyecto construido en Apache Maven. Este archivo (Project Object Model) contiene toda la información vital del proyecto, sus dependencias, configuraciones de compilación y los plugins necesarios para ejecutar el ciclo de vida del build. BACK

Respuesta de la Pregunta 51
Respuesta correcta: **B.** Imprimirá los artículos ordenados por precio.
Justificación:
Este reactivo evalúa el funcionamiento del ordenamiento natural de objetos mediante la interfaz `Comparable`. La clase `Item` implementa correctamente el método `compareTo()`, el cual es utilizado por `Collections.sort()` para determinar el orden de los elementos de la lista.

Opción A: Inválida. La implementación de `Comparable<Item>` es correcta. El método `compareTo(Item other)` tiene la firma adecuada y compara los atributos `price` mediante `Integer.compareTo()`, ya que `price` es un objeto de tipo `Integer`. El código compila sin errores.

Opción B (Imprimirá los artículos ordenados por precio): Correcta. El método `compareTo()`

compara los precios de los artículos: `return this.price.compareTo(other.price);` Como consecuencia, `Collections.sort(items)` ordena los elementos en orden ascendente de precio. La salida es: `Cherry: 1 Banana: 2 Apple: 3`

Opción C: Inválida. No existe ninguna condición que provoque una excepción en tiempo de ejecución. Todos los objetos poseen un valor válido para `price` y el método `compareTo()` se ejecuta correctamente.

Opción D : Inválida. Aunque la clase contiene el atributo `name`, este nunca es utilizado durante la comparación. El ordenamiento depende exclusivamente del valor de `price`.

Nota: : Contrastar con la pregunta 87

BACK

Respuesta de la Pregunta 52

Respuesta correcta: **B.** float

Justificación:

La división se realiza con un dividendo `short` y un divisor que termina siendo `long` (l * 10). El resultado interno que da Java será `long`. Si se nos pide la opción válida más restringida (narrowest) que sea de punto flotante para devolver ese valor convirtiéndolo, `float` (32 bits) abarca menos espacio de memoria que `double` (64 bits).

BACK

Respuesta de la Pregunta 53

Respuesta correcta: **A.** @Before

Justificación:

Este reactivo evalúa el conocimiento sobre el ciclo de vida de las pruebas unitarias en el framework JUnit (específicamente en su versión 4) y la correcta configuración del entorno de pruebas.

Opción A (Correcta - @Before): Se utiliza para preparar los recursos necesarios (setup). El framework garantiza que cualquier método marcado con esta anotación se ejecute de manera automática justo antes de **cada uno** de los métodos de prueba de la clase, asegurando un estado limpio y aislado para cada evaluación. *(Nota: En el estándar moderno de JUnit 5, fue renombrada a* `@BeforeEach`*).*

Opción B (@Test): Inválida. Es la anotación núcleo que marca a un método indicándole al motor de JUnit que contiene la lógica asertiva de una prueba que debe ser ejecutada.

Opción C (@JUnit): Inválida. Es un distractor inventado; esta anotación no existe dentro de la API del framework.

Opción D (@BeforeClass): Inválida. Es una trampa común. Aunque también prepara el entorno, un método con `@BeforeClass` (que además debe ser estático) se ejecuta **una sola vez** al inicio, antes de que corra cualquier prueba de la clase, no antes de cada una. *(En JUnit 5, su equivalente es* `@BeforeAll`*).*

Opción E (@After): Inválida. Realiza exactamente la acción opuesta a la pregunta: se ejecuta automáticamente **después** de cada método de prueba, utilizándose habitualmente para la limpieza y liberación de recursos (teardown). *(En JUnit 5, equivale a* `@AfterEach`*).*

BACK

Respuesta de la Pregunta 54
Respuesta correcta: **B.** true
Justificación:
El operador `instanceof` evalúa jerarquías de herencia. `v` se instanció como String, lo cual es verdadero. `sb` es un StringBuffer y, al igual que todos los objetos en Java, hereda implícitamente de `Object`, dando verdadero. `new MyList()` fue declarada heredando de `ArrayList`, lo cual también da verdadero. Toda la evaluación lógica con ANDs es verdadera. BACK

Respuesta de la Pregunta 55
Respuesta correcta: **C.** `super.sound();`
Justificación:
Este reactivo evalúa el uso correcto de la palabra reservada `super` en Java y distingue entre sus dos aplicaciones principales: la invocación de constructores de la superclase y el acceso a miembros heredados. Un aspecto importante es que la sintaxis permitida depende del contexto en el que se utilice.

Opción A (`super();`): Inválida. La expresión `super()` invoca un constructor de la superclase y únicamente puede aparecer como la **primera sentencia** dentro de un constructor. No puede utilizarse dentro de un método ordinario como `test()`.

Opción B (`super("Dog");`): Inválida. Al igual que `super()`, esta sintaxis invoca un constructor específico de la superclase y está restringida exclusivamente a los constructores. Su uso dentro de un método provoca un error de compilación.

Opción C (`super.sound();`): Correcta. Esta expresión invoca el método `sound()` definido en la superclase `Animal`, incluso cuando dicho método ha sido sobrescrito en la subclase. El acceso a métodos heredados mediante `super` es válido dentro de cualquier método de instancia.

Opción D (`this.super.sound();`): Inválida. La sintaxis `this.super` no existe en Java. Las únicas formas válidas de utilizar `super` son para invocar un constructor mediante `super(...)` o para acceder directamente a miembros heredados, por ejemplo `super.sound()` o `super.campo`.

BACK

Respuesta de la Pregunta 56
Respuesta correcta: **A.** 1 2 3 4 5 6
Justificación:
El patrón Regex `\\D` coincide con todos los caracteres que **no** son dígitos. Al usar `str.split("\\D")`, Java corta la cadena "a1b2c3d4e5f6" exactamente donde están las letras, dejando un arreglo que contiene solo los números: `["1", "2", "3", "4", "5", "6"]`. El bucle los imprime secuencialmente.
BACK

Respuesta de la Pregunta 57
Respuesta correcta: **A.** 12432432 The co value = 8
Justificación:
El orden de ejecución de un bucle `for` es: iniciador, condición, cuerpo e incremento. 1. Llama a

`test('1')` (imprime 1). 2. Condición: llama a `test('2')` (imprime 2) y valida `x <= 2`. 3. Cuerpo: `x++` y llama a `test('4')` (imprime 4). 4. Incremento: llama a `test('3')` (imprime 3). Esto se repite hasta que la condición (2) rompe el bucle, llevando el conteo total acumulado en `co` hasta 8.

BACK

Respuesta de la Pregunta 58

Respuesta correcta: **A.** ArrayList allows duplicate elements.

Justificación:

Por definición, las estructuras que implementan la interfaz `List` en el framework de colecciones (como lo es el `ArrayList`) permiten elementos duplicados en su interior y preservan en todo momento el orden secuencial de inserción. Su estructura subyacente es un arreglo dinámico, no un árbol.

BACK

Respuesta de la Pregunta 59

Respuesta correcta: G. 12/03/96 23:55

Justificación:

Se crea una fecha para el 15 de mayo de 1997. Se le resta un periodo exacto de 1 año, 2 meses y 3 días utilizando el método `minus()`. La operación temporal es matemática: 1997 menos 1 año es 1996; mayo menos 2 meses es marzo; día 15 menos 3 días es 12. Resulta en el 12 de marzo de 1996 en un formateo corto de región hispana/europea.

BACK

Respuesta de la Pregunta 60

Respuesta correcta: **E.** double

Justificación:

La máquina virtual de Java previene la pérdida de datos y precisión promoviendo operandos de tipos enteros menores (como `short`) al tipo numérico de mayor precisión que participa en la operación. Al interactuar aritméticamente un `short` con un `double`, todo el resultado interno se almacena como un `double`.

BACK

Respuesta de la Pregunta 61

Respuesta correcta: **B.** verify()

Justificación:

En el framework de pruebas Mockito, el método `verify()` se usa específicamente para comprobar interacciones con los objetos mockeados, permitiendo saber no solo si un método fue ejecutado, sino además validar cuántas veces sucedió esto pasándole parámetros como `times(n)`.

BACK

Respuesta de la Pregunta 62

Respuesta correcta: **B.** Se utiliza para verificar que dos valores son iguales.

Justificación:

La aserción `assertEquals(expected, actual)` es una de las más utilizadas en JUnit. Valida que el valor esperado proporcionado por el desarrollador coincida plenamente con el resultado devuelto por el método ejecutado (a nivel de valor o con la implementación de `equals()` en caso de objetos).
BACK

Respuesta de la Pregunta 63
Respuesta correcta: **B.** [2, 1, 3, 4]
Justificación:
El método `Arrays.sort(array, fromIndex, toIndex)` ordena los elementos desde el índice de inicio (incluido) hasta el índice de cierre (excluido). Para el arreglo `["2", "3", "1", "4"]`, ordenar los índices 1 al 4 (excluyendo el 4) significa que ordenará los subelementos `["3", "1", "4"]` de manera natural. El `"2"` original no se mueve.
BACK

Respuesta de la Pregunta 64
Respuesta correcta: **A.** Try Finally Caught2
Justificación:
El flujo ejecuta el método `a()`, que imprime "Try" y luego, al pasar siempre por `finally`, imprime "Finally". Luego se llama al método `b()`, el cual levanta intencionalmente una nueva `Exc2`. El bloque `catch` de la clase `main` la atrapa y procede a imprimir "Caught2". El resultado final unido en consola será "Try Finally Caught2".
BACK

Respuesta de la Pregunta 65
Respuesta correcta: **D.** this()
Justificación:
La instrucción `this()` se usa dentro de un bloque constructor para mandar a llamar y ejecutar a otro constructor perteneciente a la misma clase. `super()` realizaría lo mismo pero delegando hacia el constructor de la clase superior o padre.
BACK

Respuesta de la Pregunta 66
Respuestas correctas: **A.** JDBC Utility, **B.** API Connector, **C.** Merge/Sort, **E.** Document Manager.
Justificación:
Las utilerías APX Batch son herramientas preconstruidas para automatizar grandes procesamientos de datos dentro del banco. Lista de utilidades que pueden ser usadas en APX Batch; JDBC, Document Creator, Rules, Shell, API Connector, Merge/Sort y Mongo DB.
BACK

Respuesta de la Pregunta 67
Respuesta correcta: **B.** Resource used Resource closed.
Justificación:

El código implementa el bloque `try-with-resources` asegurando que la clase instanciada implemente `AutoCloseable`. El bloque `try` manda llamar la función `use()` imprimiendo "Resource used". Al finalizar el alcance, el entorno Java invoca automática e implícitamente el método `close()` imprimiendo "Resource closed", independientemente de si hubo error o no.

Nota: Contrastar con ejercicio 84. BACK

Respuesta de la Pregunta 68

Respuesta correcta: **C.** Mantener la lógica de los objetos comando y ejecutar el comando solicitado en el momento adecuado.

Justificación:

En el patrón de comportamiento `Command`, el *Invoker* (Invocador) es el responsable exclusivo de retener las solicitudes parametrizadas (los comandos) y decidir en qué momento de la ejecución deben dispararse hacia el *Receiver* utilizando la función `execute()`. BACK

Respuesta de la Pregunta 69

Respuesta correcta: **D.** Optimizar el rendimiento y la experiencia del usuario al dividir grandes conjuntos de datos en páginas más pequeñas.

Justificación:

La paginación fragmenta inmensas tablas o arreglos devueltos por el servidor. De esta forma, ni el backend satura la memoria despachando todos los objetos, ni el cliente sobrecarga su renderizado al mostrar una lista sin final. Todo redunda en menor latencia y mayor rendimiento. BACK

Respuesta de la Pregunta 70

Respuesta correcta: **A.** i.1,j.1 i.1,j.2 i.2,j.1 i.2,j.2 i.3,j.1 i.3,j.2

Justificación:

El bucle for anidado iterará sobre `i` de 1 a 3. Por cada repetición de `i`, el bucle interior `j` se iterará de 1 a 2. Esto producirá una secuencia de impresión estrictamente pareada donde para el valor `i=1`, se imprime `j=1` y luego `j=2`. Luego la `i` salta a 2, y se imprime `j=1` y `j=2`, repitiéndose por última vez con `i=3`. BACK

Respuesta de la Pregunta 71

Respuesta correcta: **C.** Se genera una excepción en tiempo de ejecución.

Justificación:

El código intenta crear una fecha invocando `LocalDate.of(1997, Month.MAY, 50)`. Dado que el día 50 no es válido para ningún mes del calendario, la API nativa de fecha de Java lanza directamente una `DateTimeException` en tiempo de ejecución. BACK

Respuesta de la Pregunta 72

Respuesta correcta: **A.** `String result = "banana".replace('a', 'e');`

Justificación:

La clase `String` en Java provee el método `replace(char oldChar, char newChar)` para sustituir todas las ocurrencias de un carácter específico. Los métodos como `change`, `modify`, `substitute` o

`swap` simplemente no existen dentro de la API oficial de `String`.

BACK

Respuesta de la Pregunta 73
Respuesta correcta: **B.** Sound Roar Bark Growl
Justificación:
La variable `animal` imprime "Sound" y "Roar" lógicamente por sobrecarga. Para `dog`, la referencia es `Animal` pero la instancia real es `Dog`. `makeSound()` está sobreescrito (override), imprimiendo "Bark". Sin embargo, `makeSound("Growl")` busca una firma con String en `Animal` (ya que el compilador se guía por el tipo de referencia) e imprime "Growl".

BACK

Respuesta de la Pregunta 74
Respuesta correcta: **B.** Falso
Justificación:
Las utilidades integradas en la plataforma APX no se limitan en absoluto al ecosistema de bases de datos relacionales. Su arquitectura está preparada para integrarse tanto con recursos SQL como con bases de datos documentales y otros orígenes de la información.

BACK

Respuesta de la Pregunta 75
Respuesta correcta: **A.** 13
Justificación:
El operador `|` es un OR bit a bit. El número 9 en binario es `1001` y el número 4 es `0100`. Si aplicamos la compuerta lógica OR columna por columna, obtenemos `1101`, el cual se traduce como el número decimal 13.

BACK

Respuesta de la Pregunta 76
Respuestas correctas: **A.** `boolean flag = Boolean.FALSE;`, **B.** `boolean flag = true;`, **D.** `boolean flag = false;`
Justificación:
Un primitivo booleano solo puede contener literales lógicas puras (`true` o `false`). También es válido pasarle una constante estática desempacada como `Boolean.FALSE`. No acepta números (`0`) como en C/C++ ni cadenas de texto con comillas (`"false"`).

BACK

Respuesta de la Pregunta 77
Respuesta correcta: **C.** To define the project's dependencies, build configuration, and plugins.
Justificación:
El `pom.xml` (Project Object Model) es el núcleo declarativo de todo proyecto de Maven. Su objetivo principal es estructurar las directivas de compilación, invocar plugins y mantener el listado exacto de librerías y dependencias externas que el software requerirá.

BACK



Respuesta de la Pregunta 78
Respuesta correcta: **B.** ACD
Justificación:
El método `mf()` se ejecuta exitosamente sin lanzar excepciones. En consecuencia, el bloque `try` imprime "A" y se omite el bloque `catch`. El bloque `finally` se ejecuta de forma obligatoria imprimiendo "C". Al finalizar la estructura de excepciones, el programa continúa imprimiendo "D". Salida total: ACD. BACK

Respuesta de la Pregunta 79
Respuesta correcta: **B.** ...
Justificación:
La sintaxis para los argumentos de longitud variable (*Varargs*) consiste en colocar tres puntos consecutivos (...) justamente después del tipo de dato en la firma del método. Internamente, Java tratará todos los parámetros que reciba como si se tratara de un arreglo. BACK

Respuesta de la Pregunta 80
Respuesta correcta: **B.** Una entidad que encapsula todo el proceso por lotes, configurada a través de un archivo XML o una configuración basada en Java.
Justificación:
En la arquitectura de Spring Batch, el Job es el contenedor y orquestador maestro. Funciona como la entidad suprema que agrupa lógicamente toda la secuencia de procesamiento (los diferentes Steps) de un lote de tareas, administrada desde la configuración del framework. BACK

Respuesta de la Pregunta 81
Respuesta correcta: **F.** double
Justificación:
Las reglas de resolución de sobrecarga de métodos en Java siguen una jerarquía de prioridad: primero buscan coincidencias exactas, luego ensanchamiento primitivo (widening), seguido de autoboxing y, por último, varargs. Un `float` se ensancha (widening) a `double` de forma preferente antes que realizar un autoboxing hacia el objeto `Float`. Por eso llama al método `double`. BACK

Respuesta de la Pregunta 82
Respuesta correcta: **A.** Librerías
Justificación:
En el modelo de APX, las Librerías funcionan como componentes centralizados de lógica empresarial reutilizable y de interacciones a bases de datos. Al encapsular estos procesos publican una interfaz para que otros consumidores (como Transacciones o Jobs) eviten la duplicación de código. BACK

Respuesta de la Pregunta 83

Respuestas correctas: **A, B, D, E, F.**
Justificación:
La herencia dicta que un objeto "es un" derivado de otro (is-a), mientras que la composición estipula que un objeto "tiene a" otro (has-a) e involucra la instanciación como propiedades. En el buen diseño orientado a objetos, se prefiere siempre la composición sobre la herencia para evitar acoplamiento vertical.

BACK

Respuesta de la Pregunta 84
Respuesta correcta: **B.** `Resource used Resource closed.`
Justificación:
Este reactivo evalúa el uso de expresiones lambda con la interfaz `AutoCloseable` y el funcionamiento de `try-with-resources`.

Opción A (Compilation error): Inválida. `AutoCloseable` es una interfaz funcional, por lo que puede implementarse mediante una expresión lambda.

Opción B (Resource used Resource closed): Correcta. Primero se ejecuta el cuerpo del bloque `try`, imprimiendo `Resource used`. Al finalizar el bloque, Java invoca automáticamente el método `close()`, imprimiendo `Resource closed`.

Opción C (Resource closed Resource used): Inválida. El método `close()` se ejecuta únicamente al salir del bloque `try`, nunca antes.

Opción D (Runtime error): Inválida. El código se ejecuta correctamente y el recurso se cierra de forma automática al finalizar el bloque.

Nota: Contrastar con ejercicio 67.

BACK

Respuesta de la Pregunta 85
Respuesta correcta: **D.** dir
Justificación:
En el intérprete nativo de comandos de Microsoft Windows (CMD o Símbolo del Sistema), la instrucción estandarizada para imprimir y listar los elementos contenidos dentro del directorio activo es `dir`. Herramientas como `ls` son propias de consolas Unix/Linux (o PowerShell).

BACK

Respuesta de la Pregunta 86
Respuesta correcta: **H.** All the options are correct.
Justificación:
A diferencia de las clases declaradas en nivel superior que solo permiten ser públicas,package,private, las clases anidadas pueden declararse con cualquier modificador de acceso y estado de comportamiento existente (`public`, `protected`, `private`, `static`, `final`, etc.). *Nota: strictfp es un modificador que asegura que las operaciones matemáticas de punto flotante produzcan exactamente el mismo resultado en cualquier plataforma de hardware.*

Nota: Contrastar con pregunta 212.

BACK

Respuesta de la Pregunta 87
Respuesta correcta: **A.** Apple: 3 Banana: 2 Cherry: 1
Justificación:
El método de la clase implementa el contrato de `Comparable` y redirige la evaluación lógica hacia `this.name.compareTo(other.name)`. Debido a esto, la función utilitaria `Collections.sort()` alfabéticamente respetará el orden de "Apple", "Banana" y "Cherry", imprimiéndose en ese exacto formato sin alterar la disposición original. **Nota:** : Contrastar con la pregunta 51 BACK

Respuesta de la Pregunta 88
Respuesta correcta: **A.** A's constructor B's constructor C's constructor
Justificación:
En un árbol de herencia, cuando creamos una nueva instancia de la clase hija final (`C`), la máquina virtual inicia su construcción invocando recursivamente primero al constructor del padre (`B`), el cual a su vez llama al abuelo (`A`). La construcción se despliega entonces en cascada descendente de A, luego B, luego C. BACK

Respuesta de la Pregunta 89
Respuesta correcta: **C.** true true false false
Justificación:
Este ejercicio evalúa la diferencia crucial entre la comparación de contenido (`equals()`) y la comparación de referencias de memoria (`==`) cuando se construyen cadenas de texto dinámicamente en Java.

1. Construcción de los objetos: Las variables `str3`, `str4` y `str5` utilizan enfoques distintos (el operador `+`, el método `concat()` y `StringBuilder`) para unir las cadenas. Dado que la concatenación involucra variables que se resuelven en tiempo de ejecución, Java no utiliza el *String Pool* para los resultados. En su lugar, crea tres objetos `String` completamente nuevos e independientes en la memoria *Heap*, aunque los tres guarden exactamente el mismo texto: `"Hello World"`.
2. Comparación de contenido (`equals`): Las instrucciones `str3.equals(str4)` y `str4.equals(str5)` evalúan el valor interno de las cadenas carácter por carácter. Como todas contienen `"Hello World"`, ambas expresiones devuelven **true**.
3. Comparación de referencias (`==`): Las instrucciones `str3 == str4` y `str4 == str5` verifican si las variables apuntan a la misma dirección física de memoria. Como vimos en el paso 1, cada operación de concatenación instanció un objeto distinto en el *Heap*. Al ser instancias diferentes, estas comparaciones devuelven **false**.
Al concatenar los resultados en la salida estándar, el programa imprime de forma secuencial **true true false false**. BACK

Respuesta de la Pregunta 90
Respuesta correcta: **B.** It will print the name of the user with id 1.
Justificación:
Asumiendo que tanto el servidor se encuentre habilitado, el origen de datos (testdb) sea accesible y la credencial no sea refutada, el `ResultSet` avanzará el puntero con el método `rs.next()` logrando

extraer e imprimir correctamente el registro almacenado en la columna "name" del usuario requerido.
BACK

Respuesta de la Pregunta 91
Respuestas correctas: **A.** Git, **D.** Maven, **E.** JDK.
Justificación:
El desarrollo estándar de componentes APX requiere herramientas fundamentales del ecosistema Java: Git para el control de versiones y el trabajo colaborativo, Maven para la gestión de dependencias y el empaquetado del proyecto, y el JDK (Java Development Kit) para proporcionar el compilador y el entorno de ejecución necesarios.
BACK

Respuesta de la Pregunta 92
Respuesta correcta: **D.** message
Justificación:
En Java, los objetos `String` son completamente inmutables. Dentro del método `op2()`, la instrucción `s.toUpperCase()` crea y devuelve un nuevo objeto `String` en mayúsculas, pero este nuevo valor no se le reasigna a la variable `s` original. Al imprimir `msg` en el método `op3()`, este conserva intacto su valor original "message".
BACK

Respuesta de la Pregunta 93
Respuesta correcta: **C.** `if (rs != null) rs.close(); if (stmt != null) stmt.close(); if (conn != null) conn.close();`
Justificación:
Las buenas prácticas con JDBC exigen que los recursos se liberen (cierren) exactamente en el orden inverso al que fueron abiertos: primero el `ResultSet`, luego el `Statement`, y finalmente la `Connection`. Además, es indispensable validar que no sean nulos (`!= null`) antes de llamar a `close()` para evitar un `NullPointerException`.
BACK

Respuesta de la Pregunta 94
Respuesta correcta: **D.** La salida nunca puede contener el valor 30.
Justificación:
La variable `i` es compartida por los 100 hilos (ya que todos usan la misma instancia de `Test`). Debido a la condición `if (i % 10 == 0) { i++; }` que se ejecuta antes del bucle `for`, si `i` llega a ser un múltiplo exacto de 10 (como el 30), el hilo la incrementará inmediatamente. Esto asegura que ningún hilo inicie su ciclo de impresión con un valor exacto múltiplo de 10.
BACK

Respuesta de la Pregunta 95
Respuesta correcta: **D.** Display with no arguments Display with integer: 5 Display with string: Hello
Justificación:
En Java es perfectamente válido sobrecargar (Overload) métodos estáticos dentro de una misma

clase, siempre y cuando sus firmas (los parámetros que reciben) sean diferentes. El compilador sabe exactamente qué método estático invocar dependiendo del tipo de argumento que se le pase, por lo que el código se ejecuta sin errores. BACK

Respuesta de la Pregunta 96
Respuesta correcta: **A.** Jenkins
Justificación:
Jenkins es el servidor de automatización de código abierto más utilizado en la industria para orquestar la integración continua (CI) y entrega continua (CD). Bitbucket es para versionado, SonarQube para calidad de código, Jira para gestión de tickets y Artifactory para repositorios de binarios. BACK

Respuesta de la Pregunta 97
Respuesta correcta: **A.** Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana
Justificación:
La clase `ArrayList` preserva el orden de inserción de los elementos. El método `printList()` los imprime secuencialmente. El tamaño total (`size()`) es 3. El método `contains("Banana")` arroja `true` porque la cadena existe exacta. Finalmente, `get(1)` devuelve "Banana" ya que los índices en Java comienzan en 0. BACK

Respuesta de la Pregunta 98
Respuesta correcta: **B.** Falso
Justificación:
En arquitecturas bien definidas, el acceso a datos debe estar encapsulado en una **librería** (como objetos Data Access Object o DAOs), no en una "transacción". La transacción APX en sí se utiliza para agrupar lógica de negocio orquestada y no debe contener acceso a la base de datos de forma directa en su capa. BACK

Respuesta de la Pregunta 99
Respuesta correcta: **E.** Compilation fails on line 3.
Justificación:

Esta pregunta evalúa las reglas de ámbito de variables y redefinición de nombres dentro de un mismo bloque de código (shadowing) en Java.
1. Definición del primer ámbito y variable funcional: En la línea 1 se declara una expresión lambda que implementa la interfaz funcional `IFun` utilizando los parámetros `i1` e `i2`. Estos nombres de parámetros existen únicamente dentro del ámbito local de esa expresión lambda.
2. Declaración de variables locales: En la línea 2 se declaran e inicializan las variables locales `int i1 = 1, i2 = 2;` dentro del método `main`.
3. Conflicto de nombres (Shadowing/Redefinición): En la línea 3 se intenta declarar una

segunda expresión lambda reutilizando los nombres `i1` e `i2` como parámetros. Sin embargo, en Java, una expresión lambda no puede declarar un parámetro con el mismo nombre que una variable local que ya se encuentra activa en su ámbito circundante inmediato dentro del mismo bloque de código, lo que provoca un error de compilación en dicho punto. BACK

Respuesta de la Pregunta 100
Respuesta correcta: **C.** hotfix
Justificación:
En la metodología de control de versiones Gitflow, la rama `hotfix` es la única autorizada para derivarse directamente de `master`. Su propósito es aislar y resolver rápidamente errores críticos detectados en producción. Una vez validada, se fusiona nuevamente en `master` y en `develop`. BACK

Respuesta de la Pregunta 101
Respuesta correcta: **C.** [apple]
Justificación:
El `Stream` evalúa cada elemento de la lista utilizando la función intermedia `filter()`, la cual verifica el predicado `s.startsWith("a")`. De la lista provista, la única cadena de texto que comienza con la letra "a" minúscula es "apple". La recolección terminal lo depositará en una nueva lista con ese único valor. BACK

Respuesta de la Pregunta 102
Respuesta correcta: **D.** `public char met() { return '2'; }`
Justificación:
Al tratar de sobrescribir un método padre (Override), no puedes alterar el tipo de dato de retorno por uno que no sea covariante (compatible). Declarar `char` en lugar de `String` pero manteniendo la firma vacía del método hace que el compilador arroje un error por incompatibilidad de retorno. Las otras opciones representan una *sobrecarga* (Overload) válida, no una *sobrescritura*. BACK

Respuesta de la Pregunta 103
Respuesta correcta: **A.** message middle end
Justificación:
La ejecución `new CA("message ")` entra al constructor con String e imprime "message ". Al hacer `new CB()`, Java inserta una llamada implícita a `super()` en la clase hija porque no se especificó constructor. Esto invoca al constructor vacío de la clase padre (`CA()`), imprimiendo "middle". Finalmente el código principal imprime " end". BACK

Respuesta de la Pregunta 104
Respuesta correcta: **B.** Falso

Justificación:
Una "transacción" APX no es la unidad aplicativa base del marco Batch. Las transacciones son para la arquitectura en línea (APX Online). En la rama APX Batch, la unidad aplicativa principal que rige las operaciones y rutinas secuenciales masivas se denomina "Job". BACK

Respuesta de la Pregunta 105
Respuesta correcta: **D.** Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.
Justificación:
A diferencia del JRE (Java Runtime Environment) que solo permite ejecutar aplicaciones, el JDK (Java Development Kit) es el paquete completo diseñado para creadores de software. Incluye el compilador nativo (`javac`), las herramientas de ejecución (`java`) y otras utilidades vitales de desarrollo (como el depurador y generador de documentación). BACK

Respuesta de la Pregunta 106
Respuesta correcta: **B.** Establecer la gravedad de la notificación generada por la transacción
Justificación:
En el marco de APX, cuando una transacción se ejecuta, puede generar diversos mensajes de bitácora y notificaciones. El método `setSeverity()` se utiliza explícitamente para definir el nivel de gravedad de estas alertas, tales como informativo (INFO), advertencias (WARN) o errores críticos (ERROR). BACK

Respuesta de la Pregunta 107
Respuesta correcta: **E.** true 10 FUNCTIONAL Hello, World!
Justificación:

Esta pregunta evalúa el uso de las interfaces funcionales básicas del paquete `java.util.function` y sus métodos abstractos principales: `Predicate.test()`, `Function.apply()`, `Consumer.accept()` y `Supplier.get()`.

1. Evaluación del Predicado (`Predicate.test`): Se define un predicado con la interfaz `Predicate<String>` para evaluar si la longitud de una cadena es mayor a 5. Al evaluar la cadena `"functional"` (que tiene 10 caracteres), el método `p.test(testString)` retorna **true**.

2. Aplicación de la Función (`Function.apply`): Se utiliza una referencia a método (`String::length`) mediante la interfaz `Function<String, Integer>` para obtener la longitud de la cadena. Al invocar `f.apply(testString)`, el resultado entero es **10**.

3. Consumo y transformación (`Consumer.accept`): La interfaz `Consumer<String>` recibe la cadena, la convierte a mayúsculas mediante `s.toUpperCase()` y la imprime en consola, mostrando **FUNCTIONAL**.

4. Proveedor de datos (`Supplier.get`): La interfaz `Supplier<String>` provee un valor de cadena sin recibir argumentos. Al invocar `s.get()`, se obtiene y se imprime el texto **Hello, world!**.
BACK



Respuesta de la Pregunta 108
Respuesta correcta: **A.** ticgameTAC
Justificación:
Este ejercicio evalúa el concepto fundamental de la inmutabilidad de la clase `String` en Java y qué sucede cuando los resultados devueltos por sus métodos no se almacenan.
1. Inicialización en memoria: Se instancian las variables `a` y `b` apuntando a `"tic"` y `"tac"` respectivamente.
2. Operaciones descartadas: En las líneas `a.substring(0, 2);` y `a.concat("toc");`, se ejecutan los métodos y se crean nuevos objetos `String` en el *Heap* (con valores `"ti"` y `"tictoc"`). Sin embargo, no se asignan estos resultados a alguna variable de referencia. Los objetos recién creados quedan elegibles para el *Garbage Collector*. La variable `a` permanece intacta con su valor original `"tic"`.
3. Evaluación de la impresión: En la instrucción `System.out.println(a += "game" + b.toUpperCase());`, la expresión se resuelve en este orden: (I) `b.toUpperCase()` devuelve `"TAC"`, luego (II) se concatena con la literal `"game"`, formando en memoria la cadena temporal `"gameTAC"`.
4. Reasignación final: Por último, el operador `+=` toma el valor actual de `a` (que sigue siendo `"tic"`), lo concatena con `"gameTAC"`, y ahora sí reasigna la variable `a` hacia este nuevo objeto. El programa imprime el valor final modificado, dando como resultado **ticgameTAC**. BACK

Respuesta de la Pregunta 109
Respuesta correcta: **B.** java.lang
Justificación:
El paquete `java.lang` contiene clases tan fundamentales para el lenguaje de Java (tales como `String`, `Object`, `Math`, `System`) que el compilador lo importa siempre de forma automática e implícita en todos los archivos. Para dar contexto;
Opción A (java.net): Requiere importación explícita (RIE). Contiene las clases e interfaces necesarias para implementar aplicaciones de red, como manejo de sockets, URLs y conexiones HTTP.
Opción C (java.function): RIE (Cabe aclarar que en la API real de Java el paquete exacto es `java.util.function`). Se introdujo en Java 8 y contiene las interfaces funcionales estándar (como `Predicate`, `Consumer`, `Function`) que sirven como base para las expresiones lambda.
Opción D (java.util): RIE. Es el paquete de utilidades generales por excelencia. Aquí reside todo el *Collections Framework* (`List`, `Map`, `Set`), así como utilidades para fechas, escaneo de datos y concurrencia básica.
Opción E (java.io): RIE. Provee el sistema de entrada/salida de datos del lenguaje, permitiendo leer y escribir en flujos (*streams*), serializar objetos y manipular el sistema de archivos del sistema operativo.
Opción F (javax.sql): RIE. Provee la API orientada a servidores para el acceso a fuentes de datos, extendiendo las capacidades básicas de JDBC (`java.sql`) con características como *connection pooling* y transacciones distribuidas. BACK

Respuesta de la Pregunta 110
Respuesta correcta: **D.** An immutable object does not allow any changes to its state after it is created.
Justificación:
La inmutabilidad se define por la incapacidad de alterar el estado interno (las propiedades o atrib-

utos) de un objeto después de haber sido instanciado. Generalmente se logra omitiendo deliberadamente los métodos de modificación (setters) y declarando todas las variables como final y privadas.

BACK

Respuesta de la Pregunta 111

Respuesta correcta: **C.** MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY

Justificación:

Todos los objetos de tipo `enum` de Java reciben un método implícito estático llamado `values()`, el cual devuelve un arreglo con las constantes enumeradas, respetando rigurosamente el orden textual en el que fueron declaradas. El ciclo for-each imprime cada una seguida de un espacio en blanco.

Opción B: Es incorrecta porque asume que se imprimirán las comas. En Java, las comas separan los elementos únicamente en la sintaxis de declaración. Al imprimir, el comportamiento lo dicta la instrucción `System.out.print(day + " ");`, la cual explícitamente usa un espacio en blanco como separador.

Opción D: Es incorrecto, omite el último elemento (`SUNDAY`). Esta es una trampa muy bien pensada que juega con la costumbre del programador: en muchas APIs de Java (como `substring()`, `subList()` o rangos en *Streams*), el límite superior suele ser exclusivo. Sin embargo, al invocar `values()`, el arreglo devuelto siempre es totalmente inclusivo y abarca todos los elementos declarados.

BACK

Respuesta de la Pregunta 112

Respuesta correcta: **E.** 1111

Justificación:

Comportamiento de cascada (Fall-through): Este ejercicio es una trampa clásica que evalúa el *fall-through* del `switch` tradicional. Cuando se encuentra una coincidencia, la ejecución salta a ese bloque, pero no se detiene automáticamente al terminar sus instrucciones. Si no encuentra la palabra reservada `break`, la JVM ignora las siguientes etiquetas `case` y continúa ejecutando todo el código que encuentre hacia abajo hasta topar con un `break` o el fin de la estructura.

Traza de ejecución: Dado que `d` es `TUESDAY`, el `switch` salta al caso respectivo incrementando `y += 10` (el valor pasa de 1 a 11). Al carecer de comandos `break`, el código continúa derramándose por los casos siguientes sumando 100 (quedando en 111) y luego 1000 (terminando en 1111).

Cuidado con las nuevas versiones de Java: Es crucial fijarse muy bien en la sintaxis de las preguntas. Si el código utilizara el *Switch Expression* moderno (introducido formalmente en Java 14), los casos usarían una flecha en lugar de dos puntos (`case TUESDAY ->`). En esa nueva sintaxis **no existe el efecto cascada** por defecto, y solo se ejecutaría el bloque coincidente, dando un resultado completamente distinto.

BACK

Respuesta de la Pregunta 113

Respuesta correcta: **E.** No puedes asignar un byte a un carácter.

Justificación:

En Java, `byte` es un tipo numérico firmado (admite negativos) de 8 bits, mientras que `char` es un tipo sin signo de 16 bits. Esta incompatibilidad arquitectónica impide su asignación directa. Para forzar al compilador a aceptar la asignación es obligatorio utilizar un *cast* explícito. BACK

Respuesta de la Pregunta 114
Respuesta correcta: C. 20
Justificación:
Este ejercicio evalúa el concepto fundamental del *paso por valor* (pass-by-value) en Java, específicamente cuando se trabaja con tipos de datos primitivos.

1. Declaración inicial: En el método `main`, se declara e inicializa la variable primitiva `int a = 20;`.

2. Llamadas al método: Se invoca al método estático mediante `m(a)` y luego nuevamente mediante `Test.m(a)` (ambas formas son completamente válidas e idénticas). Al pasar la variable `a` como argumento, Java envía una **copia** del valor (20), no la referencia a la variable original.

3. Ejecución interna: Dentro del método `m(int a)`, el parámetro local `a` recibe el valor 20 y luego se le suma 10 (quedando en 30). Sin embargo, esta modificación ocurre únicamente en el *Stack* del método `m`. En cuanto el método termina su ejecución, esa variable local se destruye.

4. Resultado final: La variable `a` original que vive en el método `main` jamás fue modificada y permanece intacta. Por lo tanto, la instrucción `System.out.println(a);` imprime **20**. BACK

Respuesta de la Pregunta 115 Respuesta correcta: A. The Dog barks
Justificación:
1. Ejecución inicial: Al arrancar el programa desde la clase `Test`, su método `main` invoca directamente al método `main` de `Animal`. Tener múltiples métodos `main` es completamente válido en Java, ya que se tratan simplemente como métodos estáticos normales.
2. Visibilidad y métodos privados en la herencia: Aunque el método `speak()` en la clase `Animal` está declarado como `private`, esto solo restringe su visibilidad al cuerpo de la propia clase `Animal`. No impide que la clase subclase `Dog` declare su propio método `speak()` con otra visibilidad (en este caso `protected`), tratándose de miembros independientes en lugar de un error de sobreescritura.
3. Resolución de tipos y ejecución: Con la variable declarada directamente como `Dog animal = new Dog();`, la llamada en la línea 7 resuelve de forma estática y dinámica al método `speak()` definido en la clase `Dog`, imprimiendo correctamente **The Dog barks** en la consola.

Nota: Contrastar los ejercicios 115, 149, 155 y 189. BACK

Respuesta de la Pregunta 116
Respuesta correcta: C. Hello Heo dlroW olleH
Justificación:
`String`: Al llamar a `modify(str)`, la referencia se pasa por valor. Como los `String` son inmutables en Java, reasignar `s` dentro del método no afecta a la variable original `str`. Por lo tanto, se imprime **Hello**. **`StringBuffer`:** La llamada a `modify(sbf)` altera el objeto mutable agregándole `" World"`.

Al aplicar el método `sbf.replace(2, sbf.length(), "o")` al texto `"Hello World"`, se reemplazan los caracteres desde el índice 2 (la primera 'l') hasta el final de la cadena por una única `"o"`, resultando en **Heo**. **`StringBuilder`:** La llamada a `modify(sbd)` también altera el objeto dejándolo como `"Hello World"`. El método `sbd.reverse()` invierte toda la cadena de forma mutante (*in-place*), resultando en **dlroW olleH**.

BACK

Respuesta de la Pregunta 117
Respuesta correcta: **B.** Running Problem Stopping
Justificación:
La sintaxis arrojará primero la cadena "Running ". Al lanzar manualmente el error, el sistema colapsa dentro del `try` y entra en la ruta alterna del bloque `catch`, procesando "Problem". El camino llega finalmente a la rutina de limpieza `finally` imprimiendo "Stopping". Al unirse las salidas se observa la secuencia descrita.
BACK

Respuesta de la Pregunta 118
Respuestas correctas: **A.** The default always invokes the no-arg superclass constructor. / **D.** The default constructor is created only when the class has no constructors defined.
Justificación:
El constructor predeterminado por el compilador solo existe y es insertado si el desarrollador no provee de manera explícita un solo constructor para la clase. Este constructor invisible está programado además para lanzar siempre la directriz `super()` hacia la clase padre.
BACK

Respuesta de la Pregunta 119
Respuesta correcta: **C.** TESTRO01
Justificación:
Basado en el diseño estándar de las arquitecturas APX, las siglas internas del componente dictan su función. La nomenclatura correcta inicia con el código o nombre del proyecto (TEST), el indicador "RO" que denota ser una Librería (Reusable Object) y el indicador de versión o consecutivo numérico (01).
BACK

Respuesta de la Pregunta 120
Respuesta correcta: **B.** False
Justificación:
Aplicando leyes lógicas paso a paso sobre `x&&y || !x`. Conociendo los datos de `x(true)` e `y(false)`: La izquierda `(true && false)` equivale a `false`. La derecha indica una negación de `x`, así que `(!true)` también equivale a `false`. Finalmente, evaluar la condición central `false || false` da un resultado irremediablemente `False`.
BACK

Respuesta de la Pregunta 121
Respuesta correcta: **B.** settings.xml

Justificación:
A diferencia del `pom.xml` que configura el proyecto en sí, el archivo `settings.xml` (comúnmente ubicado en el directorio `.m2` del entorno local) es específico de la instalación y el usuario. Sirve para almacenar contraseñas, configurar servidores proxy y declarar repositorios remotos o locales privados. BACK

Respuesta de la Pregunta 122

Respuesta correcta: **B.** El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él.
Justificación:
La retrocompatibilidad (backward compatibility) es un principio de diseño que asegura que las actualizaciones, nuevas versiones o parches de un software o API no rompan ni alteren el funcionamiento de los clientes y sistemas heredados que ya lo consumían anteriormente. BACK

Respuesta de la Pregunta 123

Respuestas correctas: **A.** `Dog dog = new Dog();`, **B.** `Animal dog = new Dog();`
Justificación:
En Java puedes referenciar a un objeto usando su propio tipo (Opción A) o usando el tipo de cualquiera de sus superclases o interfaces gracias al polimorfismo (Opción B). No puedes hacer lo inverso (asignar un padre a una referencia hija como en la Opción C) ni instanciar un padre creyendo que es un hijo. BACK

Respuesta de la Pregunta 124

Respuestas correctas: **C.** `boolean c = (7 >= 7);`, **D.** `boolean e = (10 > 5 && 2 < 1);`, **F.** `boolean a = (3 < 6);`
Justificación:
Una variable de tipo `boolean` únicamente puede albergar el resultado de una expresión lógica o relacional válida que decante en `true` o `false`. A diferencia de otros lenguajes, no acepta literales de texto `"true"`, ni sustracciones numéricas, ni asignaciones erróneas con un solo `=`. BACK

Respuesta de la Pregunta 125

Respuesta correcta: **D.** `apx init du-lib -u="TEST" (-c="000") -d="New Deployment Unit Library" -y`
Justificación:
En la CLI de APX, la sintaxis correcta para inicializar una nueva Unidad de Despliegue (Deployment Unit) enfocada en ser una librería reutilizable es utilizando explícitamente el comando y argumento `du-lib`. Los otros comandos son para aplicaciones batch o para DTOs. BACK

Respuesta de la Pregunta 126

Respuesta correcta: **D.** Es un entorno para que los desarrolladores prueben y depuren sus transacciones y job.
Justificación:
El "Entorno Local" en el ecosistema de APX funciona como una caja de arena de desarrollo. Permite ejecutar y probar las lógicas empresariales de las transacciones sin afectar las bases de datos de integración o producción, garantizando un ciclo de vida de desarrollo seguro y controlado. BACK

Respuesta de la Pregunta 126b
Respuesta correcta: **C.** Un objeto de dominio que encapsula una fase independiente y secuencial de un trabajo por lotes.
Justificación:
Dentro de la arquitectura de Spring Batch, el trabajo principal (*Job*) está lógicamente dividido en múltiples *Steps*. Cada Step es un segmento aislado que posee su propio ciclo de lectura, procesamiento y escritura, permitiendo encadenar tareas complejas fase por fase. BACK

Respuesta de la Pregunta 127
Respuestas correctas: **C.** `void display() { System.out.println("InChild"); }`, **D.** `public void display() { System.out.println("Child"); }`
Justificación:
Las reglas de anulación (Override) en Java prohíben restringir la visibilidad del método padre. El método padre es *package-private* (sin modificador). Mantenerlo igual (C) o ampliar su visibilidad a `public` (D) es válido. Hacerlo `private` arroja error. Hacerlo `static` es Ocultamiento (Hiding), no Override. BACK

Respuesta de la Pregunta 128
Respuesta correcta: **D.** `abstract final class Person ()`
Justificación:
La declaración `abstract final` constituye una contradicción técnica insalvable. El modificador `abstract` exige y diseña a la clase para que sea obligatoriamente heredada, mientras que el modificador `final` prohíbe absolutamente que la clase tenga descendencia o herencia. BACK

Respuesta de la Pregunta 129
Respuesta correcta: **D.** true false true
Justificación:
Este ejercicio evalúa el conocimiento profundo sobre cómo Java gestiona la memoria para las cadenas de texto, específicamente el uso del *String Pool* y el método `intern()`.

El *String Pool*: Es un área especial de memoria dentro del *Heap* donde Java almacena una única copia de cada cadena literal para optimizar recursos. Este patrón de diseño (Flyweight) no es exclusivo de los Strings; es un concepto muy común en Java. Por ejemplo, las clases envolventes (*wrapper classes*) como `Integer` o `Long` tienen un caché interno estático que guarda en memoria y recicla automáticamente los valores entre -128 y 127.

1. `String s1 = "text";` y `s3 = "text";`
Al usar literales (entre comillas dobles), Java coloca `"text"` en el *String Pool*. Cuando `s3` se inicializa con la misma literal, la JVM busca en el Pool, encuentra que ya existe y recicla esa misma referencia. Por lo tanto, `s1` y `s3` apuntan exactamente a la misma dirección física. Esto hace que `s1 == s3` evalúe a **true**.

2. `String s2 = new String("text");`
El uso de la palabra reservada `new` siempre fuerza a la JVM a crear un objeto completamente nuevo en la memoria *Heap* general, saltándose el *String Pool*. Aunque el texto interno sea el mismo, `s2` es un bloque de memoria distinto a `s1`. Por ello, `s1 == s2` evalúa a **false**.

3. `String s4 = s2.intern();`
El método `intern()` funciona como un puente. Le indica a la JVM: "toma el valor de esta cadena y búscalo en el *String Pool*; si el texto ya existe ahí, devuelve su referencia, si no, agrégalo". Como el texto `"text"` ya vivía en el Pool (creado en la línea 1), el método devuelve la referencia original. Como resultado, `s4` termina apuntando al mismo lugar que `s1` y `s3`, por lo que `s1 == s4` evalúa a **true**.

BACK

Respuesta de la Pregunta 130
Respuesta correcta: **D.** equals()
Justificación:
Existe un contrato fundamental en la plataforma Java: Si decides sobrescribir `hashCode()`, estás estrictamente obligado a sobrescribir también `equals()`. De no hacerlo, destruirás el funcionamiento lógico de estructuras de datos basadas en el cálculo de colisiones por Hash, tales como `HashMap` y `HashSet`.

BACK

Respuesta de la Pregunta 131
Respuesta correcta: **C.** Static block 1, Static block 2, Instance block 1, Instance block 2, Constructor, Instance block 1, Instance block 2, Constructor
Justificación:
El motor de Java dispara los bloques estáticos (*Static block*) exactamente una sola vez cuando la clase es compilada y cargada a memoria. Posteriormente, por cada instrucción `new` ejecutada, lanza todos los bloques de instancia (*Instance block*) seguidos finalmente por la rutina del constructor. Como hay dos llamadas a `new`, este último proceso se repite dos veces.

BACK

Respuesta de la Pregunta 132
Respuesta correcta: **B.** cd
Justificación:
En las interfaces Bash de sistemas Linux y Unix, el acrónimo `cd` proviene de "Change Directory". Es la instrucción estandarizada universalmente a nivel de sistema operativo para transitar entre las jerarquías y subdirectorios de la terminal.

BACK



Respuesta de la Pregunta 133
Respuestas correctas: **A.** Ignoring exceptions instead of handling them properly., **B.** Using empty and undocumented catch blocks., **C.** Duplicating code.
Justificación:
Omitir el tratamiento de excepciones o capturarlas con bloques vacíos (sin arrojar logs o alertas) oculta errores críticos del sistema convirtiéndolos en "fallos silenciosos", lo cual imposibilita su diagnóstico. Por otro lado, la duplicación de código dispara directamente la deuda técnica, haciendo el sistema inmanejable. BACK

Respuesta de la Pregunta 134
Respuesta correcta: **B.** LinkedHashSet
Justificación:
El conjunto de características requeridas define a la perfección a `LinkedHashSet`. Al implementar `Set` rechaza la inserción de duplicados. Sin embargo, su virtud radica en que implementa internamente una lista doblemente enlazada, lo que le permite garantizar y mantener un estricto orden cronológico de cómo fueron insertados los datos. BACK

Respuesta de la Pregunta 135
Respuesta correcta: **C.** 24 veces
Justificación:
La ejecución de los bucles anidados actúa como un proceso multiplicador. El bucle más externo se repite 4 veces, el de en medio se repite 3 veces y el interno se repite 2 veces. La cantidad total de ocasiones que se ejecutará la instrucción más profunda es la multiplicación secuencial de estas limitantes ($4 \times 3 \times 2 = 24$). BACK

Respuesta de la Pregunta 136
Respuesta correcta: **E.** None of the options is valid.
Justificación:
En Java, todos los métodos declarados en el cuerpo de una `interface` son implícitamente `public`. Cuando una clase concreta los implementa, está forzada a mantener ese nivel público (`public void method() {}`). Las opciones ofrecen visibilidades menores o implementan abstractos, lo que causaría inmediatamente un error de compilación por reducción de visibilidad. BACK

Respuesta de la Pregunta 137
Respuestas correctas: **E.** Marco de trabajo para lotes diseñado para permitir el desarrollo de aplicaciones por lotes.
Justificación:
Spring Batch es el framework insignia de Java diseñado ex profeso para crear aplicaciones de alto rendimiento enfocadas en procesar volúmenes masivos de datos fuera de línea (Batch). Para lograrlo, su esquema arquitectónico se sostiene en la subdivisión lógica de tareas utilizando agrupadores maestros (*Jobs*) segmentados en ejecuciones individuales (*Steps*). BACK

Respuesta de la Pregunta 138
Respuesta correcta: D. false
Justificación:
Este ejercicio evalúa la composición de interfaces funcionales en Java, específicamente el encadenamiento de condiciones lógicas usando los métodos por defecto (*default methods*) de la interfaz `Predicate`.
1. Línea 1: Se define el predicado inicial `p` mediante una expresión lambda que evalúa si el atributo `i` del objeto `Foo` es mayor que cero (`f.i > 0`).
2. Línea 2 (Composición): Se reasigna `p` encadenando nuevas condiciones. En `Predicate`, estas operaciones devuelven un nuevo predicado y su agrupación respeta el orden de llamada (de izquierda a derecha). Primero se aplica `or(f -> f.i > 0)`, formando el equivalente a `(i > 0) || (i > 0)`. A todo ese bloque resultante se le aplica `and(f -> f.i < 0)`. El predicado final compilado es lógicamente equivalente a: `((i > 0) || (i > 0)) && (i < 0)`.
3. Línea 3 (Evaluación): Se invoca el método `compare` inyectando el predicado compuesto y una nueva instancia de `Foo` inicializada con `i = 1`.
4. Resolución matemática: Al ejecutarse internamente `predicate.test(foo)`, la JVM sustituye los valores: Primero evalúa el bloque del OR: `((1 > 0) || (1 > 0))` lo cual da **true**. Finalmente evalúa el AND: `true && (1 < 0)`. Como 1 no es menor que 0, esta parte es falsa, y `true && false` resulta en **false**.
El programa compila sin errores e imprime **false** en la consola.

BACK

Respuesta de la Pregunta 139
Respuesta correcta: F. BC
Justificación:
En las arquitecturas `switch` sin interrupciones, se activa la caída de cascada (*fall-through*). La instrucción entra en `case 'B'`, imprimiendo "B". Como no existe un `break` que la frene, continúa derramándose por la instrucción `case 'C'` e imprime "C". Justamente ahí encuentra un `break`, abortando el resto e impidiendo que imprima "F".

BACK

Respuesta de la Pregunta 140
Respuesta correcta: D. ABCx
Justificación:
1. `Test m = new Test("M");`
Se crea el primer objeto. El orden de inicialización es el siguiente:
`value = "A"` (Asignación inicial de la variable)
`value = "AB"` (Primer bloque de instancia)
`value = "ABC"` (Segundo bloque de instancia)
`value = "ABCM"` (Constructor con parámetro)

2. `m = new Test();`
Aquí la referencia `m` pasa a apuntar a una instancia completamente nueva. El objeto anterior se pierde (se "borra el historial") y el ciclo de carga arranca nuevamente desde cero:
`value = "A"` (Asignación inicial de la variable)
`value = "AB"` (Primer bloque de instancia)
`value = "ABC"` (Segundo bloque de instancia)

`value = "ABCx"` (Constructor sin parámetros)

3. `System.out.println(m.value);`
El código imprime el valor del objeto actual al que apunta `m`, dando como resultado **ABCx**.

BACK

Respuesta de la Pregunta 141
Respuesta correcta: A. The program will print "Drawing a circle" followed by "This is a shape".
Justificación:
La clase abstracta `Shape` dictamina correctamente el contrato del método `draw()`. La clase heredera `Circle` respeta y sobrescribe debidamente este contrato imprimiendo su cadena. En la segunda instrucción, el objeto ejecuta la función concreta `printShape()` sin errores, ya que la heredó de forma íntegra e intacta de su padre.

BACK

Respuesta de la Pregunta 142
Respuesta correcta: A. Una librería compuesta de una sola clase con gran cantidad de funcionalidad.
Justificación:
Antipatrón BLOB / God Object: Hace referencia a una clase u objeto que concentra demasiada responsabilidad y funcionalidad, monopolizando el procesamiento y convirtiendo al resto de las clases del sistema en simples contenedores de datos. **B corresponde al patrón de diseño estructural Composite:** Hace exactamente lo que describe, permite componer objetos en estructuras de árbol para que el cliente trate de manera uniforme tanto a los objetos individuales como a las composiciones. **C corresponde al antipatrón Poltergeist:** Son clases inútiles y sin estado con ciclos de vida muy cortos, cuya única responsabilidad es inicializar o invocar métodos en otras clases, saturando la arquitectura de código innecesario. **D corresponde al antipatrón Golden Hammer (Martillo de Oro):** Es la mala práctica de forzar el uso de una herramienta, patrón o tecnología conocida para resolver cualquier tipo de problema ("si tu única herramienta es un martillo, todo problema parece un clavo"). **E corresponde al antipatrón Spaghetti Code / Big Ball of Mud:** Describe un sistema con un acoplamiento excesivo y nula estructura, donde el flujo de dependencias está tan enredado que realizar un cambio rompe el funcionamiento de otras partes no relacionadas.

BACK

Respuesta de la Pregunta 143
Respuesta correcta: B. `public class Bike implements Vehicle { public void start() { } void stop() { } }`
Justificación:
Por regla estricta de compilación en Java, absolutamente todos los métodos definidos en una interfaz tienen visibilidad `public`. Cuando una clase concreta (como `Bike`) los implementa, debe ratificar obligatoriamente esa visibilidad `public`. Omitir el modificador en el método `stop()` (dejándolo como *package-private*) supone una reducción de visibilidad que causa un error de sintaxis en el compilador.

BACK



Respuesta de la Pregunta 144
Respuesta correcta: **C.** `public int compare(Person p1, Person p2) { return Integer.compare(p1.getAge(), p2.getAge()); }` (y también A como alternativa válida matemáticamente).
Justificación:
La interfaz `Comparator` requiere el retorno de un entero negativo, cero o positivo basado en el cotejo de dos elementos. Emplear el API unificado `Integer.compare` previene fallos técnicos de desbordamiento (Integer Overflow) al clasificar arreglos. La opción A realiza una resta directa, que funciona bien para conjuntos de datos numéricos pequeños y ordenamiento ascendente rudimentario. BACK

Respuesta de la Pregunta 145
Respuesta correcta: **D.** long
Justificación:
Durante el cálculo, la expresión `(l * 10)` transforma la ecuación inferior a un valor de 64 bits (`long`). Al segmentar y dividir este número con la variable `short`, las reglas de promoción imponen que el resultado final completo sea asimilado como un dato `long`. Por lo tanto, el tipo mínimo o más restringido requerido en la firma para no desbordar variables es `long`. BACK

Respuesta de la Pregunta 146
Respuesta correcta: **E.** Throwable
Justificación:
De acuerdo al marco oficial de la especificación de Java (Java Language Specification), el ápice absoluto de la jerarquía de anomalías lo ocupa la superclase `java.lang.Throwable`. De ella se derivan sistemáticamente sus dos vertientes ramificadas de incidentes: `java.lang.Exception` y `java.lang.Error`. BACK

Respuesta de la Pregunta 147
Respuesta correcta: **A.** Verdadero
Justificación:
La arquitectura fundamentada de software sugiere ampliamente delegar todas las transacciones de SQL, rutinas o consultas a la base de datos hacia capas estandarizadas de persistencia o *Bibliotecas de Acceso a Datos*. Esta técnica promueve alta cohesión, aisla controles lógicos y simplifica notoriamente la reutilización de consultas repetitivas. BACK

Respuesta de la Pregunta 148
Respuesta correcta: **C.** Es posible que tengan valores duplicados.
Justificación:
A distinción directa con los arreglos de tipo `Set`, ambas estructuras de recolección de memoria (los `Arrays` estáticos primitivos y las `ArrayLists` dinámicas orientadas a objetos) pueden acomodar y guardar réplicas exactas del mismo objeto o variable cuantas veces se requiera en ubicaciones o índices distintos sin ocasionar problemas en tiempo de ejecución. BACK



Respuesta de la Pregunta 149
Respuesta correcta: **D.** "The dog barks"
Justificación:

1. **Ejecución inicial:** Al arrancar el programa desde la clase `Test`, su método `main` invoca directamente al método `main` de `Animal`. Tener múltiples métodos `main` es completamente válido en Java, ya que se tratan simplemente como métodos estáticos normales.

2. **Reglas de visibilidad:** La clase `Dog` hereda de la clase abstracta `Animal` y sobrescribe el método `speak()`. Al hacerlo, amplía la visibilidad de *package-private* (por defecto) a `protected`. Esto compila sin problemas porque la regla exige que la visibilidad se mantenga igual o se haga más permisiva, nunca más estricta.

3. **Polimorfismo (Dynamic Method Dispatch):** En la instrucción `Animal animal = new Dog();`, la referencia es de tipo padre (`Animal`), pero el objeto real construido en memoria es de tipo hijo (`Dog`). En tiempo de ejecución, la JVM prioriza y ejecuta el comportamiento del objeto real, invocando el método `speak()` de `Dog` y dando como resultado **The Dog barks**.

Nota: Contrastar los ejercicios 115, 149, 155 y 189.

BACK

Respuesta de la Pregunta 150
Respuesta correcta: **C.** Constructor with name and age called Default constructor called Constructor with name and age called Constructor with name called Constructor with name and age called
Justificación:
Este código evalúa el concepto de *encadenamiento de constructores* (constructor chaining) usando la palabra clave `this()`. Cuando un constructor invoca a otro de la misma clase, el constructor llamado debe terminar toda su ejecución antes de devolver el control al constructor original.
1. `OverloadedConstructorsTest obj1 = new OverloadedConstructorsTest();`
Se invoca el constructor sin parámetros. Su primera instrucción es `this("Unknown", 0)`, lo que transfiere el control al constructor principal (el que recibe `String` e `int`). Este último imprime **Constructor with name and age called**. Al terminar, regresa al constructor por defecto, el cual imprime **Default constructor called**.
2. `OverloadedConstructorsTest obj2 = new OverloadedConstructorsTest("Alice");`
Se invoca el constructor que recibe solo un `String`. Su primera línea es `this(name, 0)`, transfiriendo el control nuevamente al constructor principal, el cual vuelve a imprimir **Constructor with name and age called**. Al regresar, el constructor actual imprime **Constructor with name called**.
3. `OverloadedConstructorsTest obj3 = new OverloadedConstructorsTest("Bob", 30);`
Se invoca directamente el constructor principal (que recibe `String` e `int`). Al no haber redirección mediante `this()`, simplemente asigna los valores a las variables de instancia e imprime por última vez **Constructor with name and age called**.
Concatenando todas las salidas en el orden de ejecución, obtenemos la secuencia descrita en C.

BACK

Respuesta de la Pregunta 151
Respuesta correcta: **D.** Maven
Justificación:
Maven es una herramienta robusta de gestión y comprensión de proyectos construida en torno al concepto de POM (Project Object Model). Su archivo de configuración `pom.xml` administra el ciclo

de vida de construcción, las dependencias y los artefactos generados. BACK

Respuesta de la Pregunta 152
Respuesta correcta: **D.** [70, 50, 30, 20]
Justificación:

Esta pregunta evalúa la implementación y el uso de la interfaz `Comparator` para personalizar el ordenamiento de arreglos en Java mediante el método `Arrays.sort()`.
1. Definición del comparador personalizado: La clase `SortLongs` implementa `Comparator` y sobrescribe el método `compare(Long a, Long b)`. Al retornar `b.compareTo(a)` en lugar del orden ascendente predeterminado (`a.compareTo(b)`), se invierte la lógica de comparación para ordenar los elementos de forma descendente (de mayor a menor).
2. Ejecución del ordenamiento: En el método `main`, se inicializa un arreglo con los valores `20L, 70L, 50L, 30L`. Al invocar `Arrays.sort(values, sort)`, el algoritmo de ordenamiento utiliza el comparador personalizado para reorganizar los elementos basándose en la inversión del signo devuelto por `compareTo`.
3. Resultado del arreglo: Tras aplicar el comparador descendente, el arreglo original pasa de estar desordenado a organizarse colocando el número mayor al inicio y el menor al final, imprimiendo en la consola **[70, 50, 30, 20]**. BACK

Respuesta de la Pregunta 153
Respuesta correcta: **C.** 159
Justificación:
El algoritmo recorre una matriz bidimensional (3x3). La condición lógica `if (i == j)` filtra e imprime de manera exclusiva los elementos que conforman la diagonal principal de la matriz (posiciones 0,0 luego 1,1 y 2,2), los cuales corresponden a los valores 1, 5 y 9. BACK

Respuesta de la Pregunta 154
Respuesta correcta: **B.** Configurar la información del repositorio local y remoto, así como las credenciales y perfiles de usuario.
Justificación:
Mientras que el archivo `pom.xml` gestiona los parámetros internos del proyecto, el archivo `settings.xml` configura el entorno local de Maven para el desarrollador. Se usa típicamente para declarar tokens, usuarios de servidores remotos y repositorios empresariales privados. BACK

Respuesta de la Pregunta 155
Respuesta correcta: **A.** Animals make noise
Justificación:
Esta variación evalúa un concepto fundamental sobre la herencia en Java: los métodos `private` no se heredan **ni participan en el polimorfismo**.

1. Enlace estático (Static Binding): Al marcar `speak()` como `private` en la clase `Animal`, la

llamada a este método se vincula en tiempo de compilación basándose en el tipo de la *referencia*, no en el objeto en memoria. Las reglas de despacho dinámico (*Dynamic Method Dispatch*) que vimos antes no aplican para métodos privados.

2. Métodos independientes: Como el método de `Animal` es privado, es invisible para sus clases hijas. Por lo tanto, el método `speak()` declarado en `Dog` no es una sobrescritura (*override*), sino un método completamente nuevo e independiente de la clase `Dog`.

3. Resolución de la llamada: En la línea `Animal animal = new Dog();`, la referencia es de tipo `Animal`. Al ejecutar `animal.speak()` desde el propio `main` de `Animal` (donde el método privado sí es accesible), el compilador llama a su propia versión privada de la clase `Animal`. El programa ignora el objeto real `Dog` e imprime **Animals make noise**.

Nota: Contrastar con ejercicios 115, 149, 155 y 189. BACK

Respuesta de la Pregunta 156

Respuesta correcta: **D.** Se utiliza para generar una excepción manualmente.

Justificación:

La palabra clave `throw` seguida de la instanciación de un objeto de error (ej. `throw new Exception()`) permite al programador detener el flujo normal de la aplicación y disparar deliberadamente una anomalía o excepción hacia la pila de llamadas. BACK

Respuesta de la Pregunta 157

Respuesta correcta: **A.** `ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar`

Justificación:

A diferencia de los métodos de instancia, los atributos o campos de las clases en Java no admiten sobreescritura polimórfica, sino que aplican la técnica de "ocultamiento" (*Shadowing*). El campo al que se accede depende estrictamente del tipo de la variable de referencia (`ElectricCar` en la opción A). BACK

Respuesta de la Pregunta 158

Respuesta correcta: **B.** IOException is a checked exception, and NullPointerException is an unchecked exception.

Justificación:

Las excepciones "Checked" (como `IOException` o `SQLException`) obligan al programador a declararlas o capturarlas para que el código compile. Las excepciones "Unchecked" (como `NullPointerException` o las derivadas de `RuntimeException`) son errores lógicos o de ejecución que el compilador no verifica anticipadamente. BACK

Respuesta de la Pregunta 159

Respuestas correctas: **C.** Ofrece una alternativa confiable para el desarrollo de transacciones no acopladas al canal, y **E.** Extensión de la Arquitectura Backend que brinda las mismas capacidades en el mundo distribuido.

Justificación:
La arquitectura APX permite construir servicios independientes desacoplando la lógica de negocio de los canales físicos o de entrada, brindando capacidades empresariales avanzadas directamente en entornos y ecosistemas distribuidos. BACK

Respuesta de la Pregunta 160
Respuestas correctas: **A.** Librerías, **B.** Jobs, **D.** DTOs.
Justificación:
En el marco de la arquitectura APX, la vertical *Batch* (Procesamiento por Lotes) está orquestada principalmente mediante "Jobs" masivos que se apoyan de componentes reutilizables como "Librerías" y transportan información mediante "DTOs". Las "Transacciones" son la unidad atómica reservada comúnmente para el entorno interactivo APX Online. BACK

Respuesta de la Pregunta 161
Respuesta correcta: **B.** greater than - 6
Justificación:
El bucle `while` emplea el operador de post-incremento `x++`, por lo que valida la condición primero y suma a la variable después. Cuando `x` es 5, se evalúa si es menor a 5 (arrojando `false` y finalizando el bucle), pero el `x` recibe un último incremento terminando en 6. El condicional ternario asume entonces `x > 5` como verdadero. BACK

Respuesta de la Pregunta 162
Respuesta correcta: **D.** The code outputs 1 3 4 2.
Justificación:
El primer hilo en iniciar bloquea el método sincronizado, cambia la bandera, imprime "1" e invoca a `wait()`, soltando el bloqueo para pausarse. El segundo hilo entra al método `else` por la bandera falsa, imprime "3", duerme 2 segundos, imprime "4" y dispara `notify()`, despertando al hilo inicial que finaliza su rutina original imprimiendo "2". BACK

Respuesta de la Pregunta 163
Respuesta correcta: **D.** try, catch, finally, throw, throws
Justificación:
El ecosistema de control y tratamiento de excepciones en Java se sostiene formalmente sobre este vocabulario reservado. `try/catch` definen las barreras de control, `finally` la limpieza forzosa, y las variaciones `throw / throws` definen la invocación o declaración explícita de anomalías. BACK

Respuesta de la Pregunta 164
Respuesta correcta: **D.** `public class Box<T> { private T value; public Box(T value) { this.value = value; } public T getValue() { return value; } }`

Justificación:
La programación Genérica (Generics) en Java se define acompañando la declaración de la clase con un parámetro de tipo, usualmente representado entre corchetes angulares como `<T>`. Esto permite que el marco de compilación construya una clase segura en tipos evitando moldes (*casting*) propensos a errores en ejecución. BACK

Respuesta de la Pregunta 165
Respuestas correctas: **B.** Procesamiento transaccional, y **C.** Procesamiento Batch.
Justificación:
La plataforma integral de APX se bifurca funcionalmente para atender las dos vertientes cardinales de negocio: el flujo rápido e interactivo orquestado a través de transacciones en línea (APX Online), y los cálculos y procesos masivos asíncronos encolados por procesos por lotes (APX Batch). BACK

Respuesta de la Pregunta 166
Respuesta correcta: **C.** Compilation error.
Justificación:
Las estrictas directivas de herencia orientada a objetos en Java estipulan que no se puede "debilitar" o "restringir" el alcance original de un método durante su sobreescritura (*Override*). El método padre era `protected`, pero el método de la clase hija no definió un modificador asumiendo así visibilidad *package-private* (que es menor), rompiendo el contrato y fallando en compilación. BACK

Respuesta de la Pregunta 167
Respuestas correctas: **A.** library-arc-osgi.xml **C.** config.xml, y **E.** pom.xml.
Justificación:
La inyección de la herramienta JDBC en un proyecto APX conlleva dos pasos automatizados: incluir en el `pom.xml` las librerías binarias necesarias, y definir en el archivo de propiedades del proyecto (`config.xml` comúnmente) las directivas de acceso hacia el origen de datos. BACK

Respuesta de la Pregunta 168
Respuesta correcta: **B.** Falso
Justificación:
La Consola de Operaciones es el panel frontal unificado para administrar el ecosistema completo, por ende su visibilidad no se limita a librerías, sino que abarca diagramas, estados y auditorías de transacciones completas en línea, así como el historial de las bitácoras y trabajos batch. BACK

Respuesta de la Pregunta 169
Respuesta correcta: **A.** Herramienta web que proporciona a los desarrolladores y administradores gestionar la configuración, verificación de componentes de la aplicación o busqueda de jobs ejecu-

tados o en ejecución.
Justificación:
Esta respuesta resume a la perfección el alcance de APX Operation Console: funcionar como la plataforma de monitoreo y ajustes de gestión (GUI Web) sobre todos los motores del marco, independientemente de si operan en su faceta en línea o como trabajos programados por lotes. BACK

Respuesta de la Pregunta 170
Respuesta correcta: **C.** Todas las opciones son correctas
Justificación:
El universo estructural de APX se consolida mediante cuatro componentes pilares con responsabilidades aisladas: DTOs para acarrear datos, Librerías para almacenar lógica compartida, Transacciones para agrupar servicios en línea y Jobs para coreografiar los flujos asíncronos del procesamiento Batch. BACK

Respuesta de la Pregunta 171
Respuesta correcta: **A.** El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él.
Justificación:
Cuando se implementa o inyecta una modificación calificada como "Retrocompatible", la API o software modificado mantiene sus contratos públicos previos inmutables. Esto da la absoluta certidumbre de que cualquier tecnología cliente previamente integrada no fallará ni requerirá adaptaciones en su propio código fuente. BACK

Respuesta de la Pregunta 172
Respuesta correcta: **C.** cp
Justificación:
Dentro de las distribuciones y consolas POSIX/Unix (como Bash), el acrónimo clásico y universal para realizar clonado o copia idéntica de ficheros o directorios hacia nuevos destinos es el utilitario `cp` (*copy*). El comando `mv` transfiere el archivo en lugar de clonarlo. BACK

Respuesta de la Pregunta 173
Respuesta correcta: **B.** Transferir datos entre diferentes capas o niveles de una aplicación.
Justificación:
Los DTOs se diseñan única y exclusivamente como contenedores anémicos (sin lógica operativa, repletos de métodos Get y Set). Son las cápsulas o "sobres" que utiliza la arquitectura para que las múltiples capas internas hablen entre sí utilizando el mismo lenguaje estructural. BACK

Respuesta de la Pregunta 174
Respuesta correcta: **B.** Proporcionar un mecanismo de persistencia para todas las operaciones

CRUD relacionadas con JobLauncher, Job y Step.
Justificación:
Para garantizar que un flujo Batch de Spring soporte cortes de energía, reintentos e interrupciones, el componente `JobRepository` provee una abstracción sobre las tablas de sistema en donde asienta crónicamente la auditoría y estado exacto de cada trabajo ejecutado. BACK

Respuesta de la Pregunta 175
Respuesta correcta: **D.** red green
Justificación:
El bucle `for-each` evalúa lógicamente los elementos del enumerador de uno por uno. Cuando el puntero inspecciona a la constante `RED`, la primera validación arroja positivo. Al inspeccionar a `GREEN`, el operador de igualdad avala positivamente. Sin embargo, en el último filtro de `BLUE`, el método `equals` se evalúa comparando un objeto tipo enum contra un `String` genérico, notar las comillas en "BLUE". Al no ser del mismo tipo la validación devuelve silenciosamente `false` y lo evade. BACK

Respuesta de la Pregunta 176
Respuesta correcta: **E.** config.yaml
Justificación:
Al invocar la utilidad de parametrización desde la CLI hacia un proyecto APX, el entorno suele generar de manera automática un archivo YAML descriptor (`config.yaml`) donde se depositarán y agruparán las directivas configurables que requerirá el proyecto. BACK

Respuesta de la Pregunta 177
Respuesta correcta: **A.** TESTT001-01-MX
Justificación:
Esta codificación rígida garantiza la ubicación y singularidad de un componente en el ecosistema. Su estructura consta del acrónimo del proyecto seguido por "T" (denotando Transacción), el número de servicio ("001"), su etapa evolutiva o versión ("01"), y culmina invariablemente con el sufijo referencial del país implementador ("MX"). BACK

Respuesta de la Pregunta 178
Respuestas correctas: **B**, **C** y **D.**
Justificación:
Las matrices pueden instanciarse de tres formas equivalentes en Java: de forma sintética apoyada por el compilador con literales `{1,2,3}`, de manera declarativa explícita mediante un bloque `new int[]{1,2,3}`, o pre-dimensionando sus espacios vírgenes con una métrica absoluta `new int[3]`. Los delimitadores por paréntesis redondos de la opción A provocan error en sintaxis de conjuntos. BACK

Respuesta de la Pregunta 179
Respuesta correcta: **C.** `[UUAA]T[CODE]-[VERSION]-[COUNTRY].xml` y se crea en la ruta `src/main/resources/`.
Justificación:
Bajo los esquemas corporativos unificados de construcción definidos por Apache Maven, todos los manifiestos, firmas y configuraciones complementarias con terminación `.xml` que dictan y definen el servicio Transaccional (T) de la aplicación, deben inyectarse y salvaguardarse ineludiblemente en la carpeta de recursos de compilación (`resources`), y nunca mezclados entre el código fuente nativo de las clases lógicas. BACK

Respuesta de la Pregunta 180
Respuesta correcta: **A.** Las excepciones checked deben capturarse o declararse con `throws`.
Justificación:
Este reactivo evalúa la diferencia entre las excepciones *checked* y *unchecked* en Java.

Opción A: Correcta. Toda excepción *checked* debe ser capturada mediante un bloque `try-catch` o declarada con la cláusula `throws`.

Opción B: Inválida. Las excepciones *unchecked*, derivadas de `RuntimeException`, no están obligadas a declararse ni capturarse.

Opción C: Inválida. Todas las excepciones heredan de `Throwable`; únicamente las excepciones *unchecked* derivan de `RuntimeException`.

Opción D: Inválida. Las excepciones *checked* pueden capturarse mediante un bloque `catch`; de hecho, esa es una de las formas de manejarlas. BACK

Respuesta de la Pregunta 181
Respuesta correcta: **D.** The code runs with no output.
Justificación:

Este código evalúa los conceptos de polimorfismo, *upcasting* (moldeado hacia la superclase/interfaz) y *downcasting* (moldeado hacia la subclase) verificados en tiempo de ejecución.
1. `Gamma g = new Epsilon();`
Se crea un objeto de tipo `Epsilon` en memoria, pero se referencia a través de su clase padre `Gamma`. Esto es un *upcasting* implícito y perfectamente válido. Lo importante aquí es recordar que el objeto real subyacente es un `Epsilon`.
2. `Omega o = (Omega) g;`
El compilador permite el casteo explícito de una clase (`Gamma`) a una interfaz (`Omega`). En tiempo de ejecución, la JVM verifica si el objeto real en memoria implementa dicha interfaz. Como el objeto real es `Epsilon` (que sí declara `implements Omega`), el casteo es exitoso y no lanza `ClassCastException`.
3. `Epsilon e = (Epsilon) g;`
Se realiza un *downcasting* explícito de la referencia `Gamma` a la clase hija `Epsilon`. Como el objeto que vive en memoria realmente fue instanciado originalmente como `Epsilon`, la operación es segura.
4. `Omega m = e;`
Se asigna la referencia de tipo `Epsilon` a una de tipo `Omega`. Al ser `Omega` una interfaz implementada directamente por `Epsilon`, esto es un *upcasting* automático que no requiere casteo explícito.
Como todas las asignaciones y casteos respetan la jerarquía del objeto original en memoria, el

código compila y se ejecuta de principio a fin sin arrojar ninguna excepción y sin imprimir ningún texto en consola.

BACK

Respuesta de la Pregunta 182
Respuestas correctas: **B.** JMS, **E.** HTTP
Justificación:
Dentro del ecosistema transaccional y de comunicaciones de la plataforma APX, las integraciones físicas estandarizadas para el consumo y exposición de servicios se realizan primordialmente a través del protocolo HTTP (para la arquitectura REST en APX Online) y mediante colas de mensajería asíncrona JMS (Java Message Service).

BACK

Respuesta de la Pregunta 183
Respuestas correctas: **A**, **C**, **D**, **E.**
Justificación:
Las excepciones "Unchecked" (no comprobadas por el compilador) son todas aquellas que heredan directamente de la clase `RuntimeException`. Esto incluye errores lógicos en tiempo de ejecución como salir de los límites de un arreglo (`ArrayIndexOutOfBoundsException`), proveer argumentos inválidos (`IllegalArgumentException`) o fallar al convertir un número (`NumberFormatException`).

BACK

Respuesta de la Pregunta 184
Respuesta correcta: **A.** compile
Justificación:
El ciclo de vida de Maven está segmentado en múltiples fases. La fase de `compile` tiene la responsabilidad exclusiva de procesar el código fuente principal del proyecto (los archivos `.java`) y transformarlo en *bytecode* (archivos `.class`) en el directorio de salida (target), sin proceder al empaquetado.

BACK

Respuesta de la Pregunta 185
Respuesta correcta: **D.** jconsole
Justificación:
`JConsole` es una herramienta gráfica de monitoreo incorporada nativamente en el Java Development Kit (JDK). Utiliza el protocolo JMX (Java Management Extensions) para conectarse a una Máquina Virtual de Java en ejecución y permitir al administrador inspeccionar en tiempo real el consumo de memoria, la actividad de los hilos y la carga del procesador.

BACK

Respuesta de la Pregunta 186
Respuesta correcta: **D.** SonarQube

Justificación:
SonarQube es la plataforma de inspección continua más utilizada para el análisis estático del código. Evalúa el código fuente para detectar automáticamente vulnerabilidades de seguridad, errores lógicos (*bugs*), deuda técnica, *code smells* y el incumplimiento de las buenas prácticas de programación dictadas por la industria. BACK

Respuesta de la Pregunta 187
Respuesta correcta: D. Ninguna de las opciones.
Justificación:
Esta pregunta evalúa las reglas de implementación de interfaces y visibilidad de métodos en Java, específicamente el principio de que no se pueden reducir los privilegios de acceso al sobrescribir o implementar un método heredado.
1. Visibilidad predeterminada en interfaces: Todos los métodos declarados dentro de una interfaz (a menos que sean `default` o `static`) son implícitamente `public` y abstractos. Por lo tanto, la clase `C` al implementar la interfaz `I` debe proporcionar una implementación con visibilidad `public`.
2. Análisis de las opciones incorrectas:

- **Opción A (`protected abstract void method()`):** Incorrecta por dos razones: reduce la visibilidad de `public` a `protected` (lo cual prohibiría el compilador) y además marca un método con cuerpo como `abstract`, lo cual genera un error de compilación.
- **Opción B (`private void method()`):** Incorrecta porque reduce drásticamente la visibilidad a `private`, violando el contrato de la interfaz.
- **Opción C (`public abstract void method();`):** Incorrecta porque la clase `C` es una clase concreta (no declarada como `abstract`), por lo que no puede contener métodos abstractos sin cuerpo.
- **Opción E (`void method()`):** Incorrecta porque omite el modificador de acceso, lo que asigna una visibilidad por defecto (package-private). Al ser más restrictiva que `public`, el compilador arroja un error indicando que no se pueden reducir los permisos de visibilidad de un método heredado.

Respuesta de la Pregunta 188
Respuesta correcta: D. when() & thenReturn()
Justificación:
En el marco de pruebas Mockito, el comportamiento simulado (*stubbing*) se define mediante la sintaxis encadenada de `when(...).thenReturn(...)`. Esto instruye al objeto falso (mock) para que, "cuando" reciba la llamada a un método específico con ciertos parámetros, "entonces retorne" el valor programado de manera estática. BACK

Respuesta de la Pregunta 189
Respuesta correcta: A. Compilation fails.
Justificación:

En Java, una subclase no tiene permitido reducir la visibilidad de un método que está sobrescribiendo (*Override*) de su superclase. El método `speak()` en la clase `Animal` fue declarado como `public`. Al intentar definirlo como `protected` en la clase hija `Dog`, el compilador detecta una reducción de privilegios y detiene la compilación con un error.

Nota: Contrastar con ejercicios 115, 149, 155 y 189. BACK

Respuesta de la Pregunta 190
Respuesta correcta: B. `StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");`
Justificación:
Este ejercicio evalúa el conocimiento de los métodos específicos de la clase `StringBuilder` y cómo se diferencia de la manipulación tradicional de cadenas en Java.

Análisis de los distractores: La opción A falla porque el método `add()` no pertenece a `StringBuilder` (es propio de colecciones como `List`). La opción C no compila porque en Java la sobrecarga del operador `+=` está reservada estricta y únicamente para la clase `String`. Finalmente, la opción D falla porque aunque el método `insert()` sí existe, exige obligatoriamente un índice de posición como primer argumento (ej. `sb.insert(5, " World")`).

Contraste de clases para manejo de texto: Para afianzar este concepto, es vital diferenciar las tres clases principales que Java ofrece para trabajar con texto:

`String` (Inmutable): Cualquier operación de concatenación (usando el operador `+` o el método `concat()`) no altera el texto original. En su lugar, fuerza a la máquina virtual a instanciar un objeto completamente nuevo en el *Heap*, lo que puede generar una sobrecarga severa en la memoria si se hace dentro de bucles.

`StringBuilder` (Mutable y veloz): Diseñado explícitamente para construir y modificar texto (usando `append()`) trabajando siempre sobre el mismo bloque de memoria subyacente. Es la opción estándar y la más rápida por defecto, pero **no** es segura para entornos multihilo (*non-thread-safe*).

`StringBuffer` (Mutable y concurrente): Es el gemelo estructural de `StringBuilder` y también utiliza `append()`. La gran diferencia es que todos sus métodos internos están fuertemente sincronizados mediante candados de exclusión mutua (*mutual exclusion locks*). Esto introduce un costo de rendimiento por la sincronización, pero garantiza operaciones *thread-safe*, siendo la opción obligatoria en arquitecturas concurrentes o de memoria compartida donde múltiples hilos intentan alterar la misma estructura al mismo tiempo. BACK

Respuesta de la Pregunta 191
Respuestas correctas: B, D, E.
Justificación:
Opción A (Falsa): Es una trampa muy común. Cuando utilizas el comando `java -jar`, la JVM **ignora por completo** las banderas `-cp` o `-classpath`. Si tu JAR ejecutable depende de otras librerías externas, la única forma de enlazarlas es declarándolas internamente usando el atributo `Class-Path` dentro del archivo manifiesto del propio JAR.
Opción B (Verdadera): Por diseño y seguridad, arrancar un programa con `java -jar` hace que la JVM anule cualquier configuración de classpath externa. Ignorará tanto la variable de entorno `CLASSPATH` del sistema operativo como los argumentos de línea de comandos. El archivo JAR

encapsulado se convierte en la única fuente de verdad para cargar las clases.
Opción C (Falsa): La JVM no tiene la capacidad de adivinar el punto de entrada. Incluso si tu proyecto tiene decenas de clases con un método `main`, al usar `java -jar` la JVM no te preguntará cuál ejecutar; simplemente fallará a menos que el manifiesto indique explícitamente a cuál debe dirigirse.
Opciones D y E (Verdaderas): Ambas afirman el mismo principio técnico. Para que el comando `java -jar` funcione sin lanzar el error *"no main manifest attribute"*, el archivo comprimido debe ser formalmente un "JAR ejecutable". Esto significa que requiere estrictamente incluir el archivo de metadatos en la ruta interna `META-INF/MANIFEST.MF`, y dicho archivo debe contener la declaración explícita del punto de arranque mediante el atributo `Main-Class: nombre.de.tu.ClasePrincipal`.

BACK

Respuesta de la Pregunta 192
Respuestas correctas: **D, E.**
Justificación:
Una clase abstracta actúa como puente en la herencia: tiene toda la libertad de implementar una interfaz (total o parcialmente dejando métodos sin definir) y, simultáneamente, está diseñada ex profeso para ser extendida (heredada) por una clase concreta hija que proporcionará la implementación final de dichos métodos ausentes.

abstract class	`implements`	**interface**
interface	*(No permitido)*	**abstract class**
interface	`extends`	**interface**
abstract class	`extends`	**abstract class**

BACK

Respuesta de la Pregunta 193
Respuesta correcta: **B.** Marcar un método como un método de prueba.
Justificación:
Dentro del entorno de *testing* de JUnit, la etiqueta decorativa `@Test` sirve para indicarle al ejecutor de pruebas (Test Runner) que el método anotado contiene un escenario de aserción válido y, por lo tanto, debe ser ejecutado automáticamente durante el ciclo de validación del software.

BACK

Respuesta de la Pregunta 194
Respuesta correcta: **A.** Verdadero
Justificación:
En ciencias de la computación, para que un bloque de transacciones a una base de datos sea confiable, debe cumplir incondicionalmente con las propiedades **ACID**: **A**tomicidad (todo se hace o todo se deshace), **C**onsistencia (deja los datos en un estado válido), **A**islamiento (no interfiere con otras transacciones) y **D**urabilidad (los cambios sobreviven a fallos).

BACK

Respuesta de la Pregunta 195
Respuesta correcta: **B.** release
Justificación:
Esta pregunta evalúa el conocimiento del modelo de ramificación de Gitflow y el propósito de cada uno de sus componentes principales en el ciclo de vida del desarrollo de software.

1. Rama `develop` (Desarrollo): Es la rama principal de integración donde se van incorporando todas las nuevas funcionalidades y evolutivos desarrollados en paralelo antes de ser preparados para un futuro lanzamiento a producción.

2. Propósito de las demás ramas en Gitflow:

- **feature:** Ramas temporales utilizadas por los desarrolladores de forma aislada para implementar nuevas características o evolutivos de la versión actual o futura, las cuales siempre se terminan fusionando de regreso hacia `develop`.
- **release:** Rama que sirve para preparar el despliegue de una nueva versión a producción, permitiendo realizar pruebas finales de último momento, correcciones menores y empaquetado sin interrumpir el desarrollo continuo de la siguiente versión en `develop`.
- **hotfix:** Rama de emergencia que se crea directamente desde `master` (o `main`) para solucionar errores críticos detectados en producción de manera urgente, asegurando que el parche se integre tanto en producción como en desarrollo.
- **master / main:** Rama que contiene exclusivamente el código estable y listo para producción, reflejando siempre las versiones que están disponibles para los usuarios finales.

BACK

Respuesta de la Pregunta 196
Respuesta correcta: **B.** mvn package
Justificación:
El comando `mvn package` ejecuta la compilación, las pruebas unitarias y finalmente agrupa el bytecode en un archivo distribuible (como JAR o WAR) en el directorio `/target`. Se detiene justo ahí. El comando `mvn install` continuaría el proceso copiando ese archivo final hacia tu repositorio local oculto (`.m2`) para que otros proyectos puedan utilizarlo.

BACK

Respuesta de la Pregunta 197
Respuesta correcta: **B.** Composite
Justificación:
El patrón estructural Composite (Compuesto) permite diseñar jerarquías en forma de árbol, de tal manera que el desarrollador pueda interactuar con un único elemento base (una "hoja") o con una agrupación compleja de elementos (una "rama") utilizando la misma interfaz idéntica y uniforme, simplificando drásticamente el código de los clientes.

BACK

Respuesta de la Pregunta 198
Respuesta correcta: **A.** Create, Read, Update, Delete.

Justificación:
CRUD es el acrónimo universal en el desarrollo de software y administración de bases de datos que define a las cuatro operaciones transaccionales elementales indispensables para gestionar cualquier tipo de información persistente: Crear nuevos registros, Leer la información, Actualizar registros existentes y Eliminar los registros que ya no se necesitan. BACK

Respuesta de la Pregunta 199
Respuesta correcta: **A.** 2
Justificación:
El flujo del `Stream` evalúa a los cuatro objetos `Person`. El filtro elimina a "Mark" porque no empieza con "J", dejando a "John", "Jane" y "Jane". Posteriormente, el método `distinct()` elimina a los duplicados gracias a que la clase sobreescribió lógicamente `equals()` y `hashCode()`, dejando únicamente a un "John" y a un "Jane". El conteo final de elementos en la secuencia es 2. BACK

Respuesta de la Pregunta 200
Respuesta correcta: **D.** Gestionar y almacenar artefactos de software, como dependencias y bibliotecas, en un repositorio.
Justificación:
En las canalizaciones modernas de CI/CD, JFrog Artifactory cumple el rol de repositorio universal de binarios. Es el almacén corporativo seguro donde las aplicaciones suben sus archivos ya construidos (JARs, contenedores Docker, paquetes npm) para ser versionados y distribuidos eficientemente a otros equipos y servidores. BACK

Respuesta de la Pregunta 201
Respuesta correcta: **D.** `public static void main(String[] args) {}`
Justificación:
Opción A: Inválida porque omite el modificador `public`. Al quedar con visibilidad *package-private*, la JVM no tiene permisos para invocarlo desde fuera.

Opción B: Inválida porque el método `main` debe retornar estrictamente `void`. La JVM no espera recibir ningún valor (como un `int`) al terminar la ejecución.

Opción C: En Java, `String` **no** es una palabra reservada, es solo el nombre de una clase. Por lo tanto, la sintaxis `String... String` (donde el segundo "String" es el nombre del arreglo *varargs*) **sí compila** y la JVM sí lo reconoce como un método `main` válido. Sin embargo, no sigue la convención estándar de usar `args` como nombre del parámetro.

Opción E: Inválida porque el método está marcado como `private`, haciéndolo completamente invisible e inaccesible para la JVM. BACK

Respuesta de la Pregunta 202
Respuesta correcta: **A.** Compilation error due to recursive constructor calls.

Justificación:
El constructor vacío envía el control hacia `this(10)`, y este segundo constructor envía forzosamente el control de regreso hacia `this()`. Al no existir una vía de escape, se genera un bucle de dependencia cíclica infinita. El compilador inteligente de Java detecta esta recursividad a nivel de inicialización y aborta el proceso de empaquetado para evitar un `StackOverflowError`. BACK

Respuesta de la Pregunta 203
Respuesta correcta: **A.** Car started Vehicle stopped Bike started Vehicle stopped
Justificación:
Ambas subclases (`Car` y `Bike`) heredaron el método concreto `stop()` de la clase abstracta padre sin sobreescribirlo, por lo que ambas imprimirán "Vehicle stopped". Sin embargo, sobreescribieron el método abstracto `start()`, por lo que la instanciación de `Car` imprimirá su versión ("Car started") y la instanciación de `Bike` imprimirá la suya ("Bike started"). BACK

Respuesta de la Pregunta 204
Respuestas correctas: **A.** Librerías, **D.** DTOs, **E.** Transacciones.
Justificación:
En el modelo operativo de APX Online (enfocado a la inmediatez e interacción en tiempo real), la orquestación del negocio se realiza a través del componente **Transacción**, apoyado estructuralmente por los envoltorios **DTOs** para acarrear datos y las **Librerías** con código común. Los Jobs están estrictamente reservados para el esquema asíncrono de APX Batch. BACK

Respuesta de la Pregunta 205
Respuesta correcta: **E.** State Pattern
Justificación:
A corresponde al patrón estructural Facade: Proporciona una interfaz unificada y simplificada para acceder a un subsistema complejo o a un conjunto de interfaces, ocultando su complejidad.
B corresponde al patrón de comportamiento Strategy: Aunque su estructura es casi idéntica a *State*, su propósito es distinto: encapsular una familia de algoritmos y hacerlos intercambiables para que el cliente elija cuál usar, sin depender de estados internos.
C corresponde al patrón creacional Singleton: Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.
D corresponde al patrón creacional Builder: Separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes representaciones con el mismo proceso de construcción.
E es correcta (patrón de comportamiento State): Permite que un objeto modifique su comportamiento en tiempo de ejecución dependiendo de su estado interno, pareciendo que cambia de clase. BACK

Respuesta de la Pregunta 206
Respuestas correctas: **A.** A class that is abstract cannot be instantiated or turned into an object

(instance)., **C.** If a class has abstract methods, then it must be specified as an abstract class.
Justificación:

A (Válida): Las clases abstractas sirven como plantillas base y, por diseño, no se pueden instanciar directamente (no puedes usar `new`).
B (Inválida): Una interfaz **sí** puede heredar de otra interfaz utilizando la palabra reservada `extends`.
C (Válida): Es una regla estricta de Java: si una clase contiene al menos un método abstracto (sin cuerpo), toda la clase debe declararse con `abstract`.
D (Inválida): Una interfaz nunca puede heredar de una clase (ni normal ni abstracta), solo puede extender otras interfaces.

abstract class	`implements`	**interface**
interface	*(No permitido)*	**abstract class**
interface	`extends`	**interface**
abstract class	`extends`	**abstract class**

BACK

Respuesta de la Pregunta 207
Respuesta correcta: **B.** Jira
Justificación:
Desarrollada por Atlassian, Jira es la plataforma líder mundial en seguimiento de incidencias, trazabilidad de bugs y administración de metodologías ágiles (Scrum, Kanban). Las demás herramientas tienen objetivos distintos: SonarQube revisa código, Bitbucket versiona archivos y Jenkins automatiza implementaciones.
BACK

Respuesta de la Pregunta 208
Respuesta correcta: **A, B, E**
Justificación:
En la construcción de transacciones APX, la clase abstracta funge como el arnés de vinculación lógica. Actúa conectando orgánicamente la clase orquestadora de negocio con el modelo de entrada (Request DTO) y el modelo de respuesta (Response DTO), permitiendo manipular limpiamente los parámetros de transferencia hacia los sistemas centrales de datos.
BACK

Respuesta de la Pregunta 209
Respuesta correcta: **C.** Ejecutar pruebas unitarias durante la etapa de 'test'.
Justificación:
El plugin `maven-surefire-plugin` está ligado estrechamente a la fase `test` de la ruta de compilación de Maven. Su objetivo único es localizar todas las aserciones construidas bajo las librerías JUnit o TestNG, ejecutarlas de forma silenciosa e imprimir un informe con el balance final de éxitos, bloqueos y fracasos del lote de pruebas.
BACK

Respuesta de la Pregunta 210
Respuesta correcta: **A.** Gestionar dependencias y permitir el despliegue dinámico de módulos en aplicaciones Java.
Justificación:
OSGi es un marco arquitectónico modular especializado para la Máquina Virtual de Java. Transforma aplicaciones monolíticas en colecciones de módulos fuertemente cohesivos denominados "Bundles". Su fortaleza yace en administrar complejos árboles de dependencia e instalar o desinstalar estos paquetes en tiempo real sin requerir un apagado o reinicio del sistema. BACK

Respuesta de la Pregunta 211
Respuesta correcta: **D.** Bark
Justificación:
A pesar de que el apuntador de variable (`myDog`) está clasificado como `Animal`, el objeto real instanciado subyacente es un perro (`new Dog()`). En Java, las llamadas de métodos de instancia son resueltas de forma dinámica buscando el método sobreescrito de la clase real en ejecución, lo que dispara irremediablemente la lógica del perro imprimiendo "Bark". BACK

Respuesta de la Pregunta 212
Respuestas correctas: **B.** abstract, **C.** strictfp, **F.** public, **H.** final
Justificación:
Esta pregunta evalúa los modificadores de acceso y restricciones permitidos al declarar una clase de nivel superior (top-level class) en un archivo de Java.
1. Modificadores de acceso permitidos: A nivel superior, una clase solo puede ser `public` (visible globalmente) o no tener modificador (package-private, visible solo en su paquete). Los modificadores de visibilidad restrictivos como **`private`** y **`protected`** están estrictamente prohibidos porque private y protected tienen sentido únicamente respecto a una clase contenedora. `private` significa *solo la clase que la contiene puede verla*. `protected` significa *la clase que la contiene, sus subclases y el paquete*. Pero una top-level class no está contenida dentro de otra clase, por lo que esos modificadores no tienen un contexto donde aplicar.
2. Modificadores estructurales y de comportamiento:

- **`abstract`** (B): Permitido. Define una clase incompleta que no puede ser instanciada directamente y está diseñada para ser extendida.
- **`strictfp`** (C): Permitido. Aplica la semántica estricta de punto flotante IEEE 754 a todos los métodos y expresiones de la clase.
- **`public`** (F): Permitido. Otorga visibilidad universal a la clase desde cualquier otro paquete.
- **`final`** (H): Permitido. Indica que la clase está completa y no puede ser extendida (no se pueden crear subclases a partir de ella).
- **`static`** (D): No permitido en clases de nivel superior (solo aplica para clases anidadas o miembros internos).

Nota: Contrastar con pregunta 86 BACK

APX GUIDE



Clasification of Questions

Classification of Questions

A continuación, se presenta una clasificación temática de las preguntas con el propósito de orientar y estructurar el estudio por áreas específicas de conocimiento.

1. Fundamentos y Sintaxis de Java (Core Java) 43
QS 006, QS 009, QS 014, QS 017, QS 019, QS 025, QS 026, QS 030, QS 039, QS 040, QS 041, QS 052, QS 056, QS 057, QS 059, QS 060, QS 070, QS 072, QS 075, QS 076, QS 079, QS 089, QS 092, QS 108, QS 109, QS 110, QS 111, QS 112, QS 113, QS 114, QS 120, QS 124, QS 129, QS 135, QS 138, QS 139, QS 145, QS 153, QS 161, QS 175, QS 178, QS 190, QS 201

2. Programación Orientada a Objetos (OOP) 43
QS 010, QS 015, QS 021, QS 022, QS 029, QS 032, QS 038, QS 054, QS 055, QS 065, QS 073, QS 081, QS 083, QS 086, QS 088, QS 095, QS 102, QS 103, QS 115, QS 116, QS 118, QS 123, QS 127, QS 128, QS 131, QS 136, QS 140, QS 141, QS 143, QS 149, QS 150, QS 155, QS 157, QS 166, QS 181, QS 187, QS 189, QS 192, QS 202, QS 203, QS 206, QS 211, QS 212

3. Arquitectura y Ecosistema APX 33
QS 013, QS 016, QS 031, QS 034, QS 042, QS 044, QS 066, QS 074, QS 080, QS 082, QS 098, QS 104, QS 106, QS 119, QS 125, QS 126, QS126b, QS 137, QS 159, QS 160, QS 165, QS 167, QS 168, QS 169, QS 170, QS 173, QS 174, QS 176, QS 177, QS 179, QS 182, QS 204, QS 208

4. Manejo de Excepciones y Errores 16
QS 001, QS 035, QS 049, QS 064, QS 067, QS 071, QS 078, QS 084, QS 117, QS 133, QS 146, QS 156, QS 158, QS 163, QS 180, QS 183

5. Herramientas de Construcción y Gestión (Maven y JDK) 18
QS 008, QS 024, QS 036, QS 037, QS 047, QS 050, QS 077, QS 091, QS 105, QS 121, QS 151, QS 154, QS 184, QS 185, QS 191, QS 196, QS 209, QS 210

6. Colecciones y Programación Funcional (Java 8+) 18
QS 012, QS 020, QS 023, QS 051, QS 058, QS 063, QS 087, QS 097, QS 099, QS 101, QS 107, QS 130, QS 134, QS 144, QS 148, QS 152, QS 164, QS 199

7. DevOps, CI/CD y Control de Versiones 9
QS 002, QS 003, QS 048, QS 096, QS 100, QS 186, QS 195, QS 200, QS 207

8. Patrones de Diseño y Antipatrones 9
QS 011, QS 018, QS 033, QS 043, QS 046, QS 068, QS 142, QS 197, QS 205

9. Bases de Datos y Persistencia 12
QS 007, QS 028, QS 045, QS 090, QS 093, QS 147, QS 194, QS 198

10. Pruebas Unitarias (Testing) 5
QS 053, QS 061, QS 062, QS 188, QS 193

11. Protocolos, Web y Misceláneos 11
QS 004, QS 005, QS 027, QS 069, QS 085, QS 094, QS 122, QS 132, QS 162, QS 171, QS 172

Basado en algunas experiencias de personas que han presentado el examen, si se quisiera hacer un simulacro de examen de 40 preguntas sugerimos hacer la siguiente distribución de preguntas por área temática:

- 5 p. de Core Java
- 8 p. de Arquitectura y Ecosistema APX
- 2 p. de Maven y JDK
- 2 p. de DevOps y Control de Versiones
- 2 p. de Bases de Datos y Persistencia
- 2 p. de Protocolos, Web y Misceláneos
- 5 p. de OOP
- 5 p. de Manejo de Excepciones
- 5 p. de Colecciones y Funcional
- 2 p. de Patrones de Diseño
- 2 p. de Pruebas Unitarias

Como ejemplo se proporcionan 2 hojas de respuestas interactiva para un simulacro de examen de 40 preguntas, pueden ser utilizadas para practicar y familiarizarse con el formato del examen real. En el pie de página se encuentran las respuestas correctas para que puedas autoevaluarte.

Hoja de Respuestas Interactiva - Simulación APX (Examen 1)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 014) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 014	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 041	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 075	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 114	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 190	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 022	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 083	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 128	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 157	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 206	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 031	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 042	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 080	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 126b	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 165	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 169	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 177	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 204	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 035	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 067	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
21. QS 117	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 156	☐	☐	☐	☐	☐	☐	☐	☐
23. QS 183	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 037	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 185	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 020	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 058	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 099	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 148	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 199	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 003	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 195	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 011	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 142	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 045	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 194	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 053	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 188	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 069	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 172	☐	☐	☐	☐	☐	☐	☐	☐



Hoja de Respuestas Interactiva - Simulación APX (Examen 2)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 009) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 009	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 039	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 109	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 129	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 201	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 015	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 054	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 116	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 187	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 212	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 016	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 066	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 098	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 125	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 160	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 170	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 182	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 208	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 001	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 049	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
21. QS 078	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 146	☐	☐	☐	☐	☐	☐	☐	☐
23. QS 163	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 024	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 191	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 023	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 063	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 101	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 134	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 164	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 048	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 207	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 033	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 197	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 007	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 093	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 061	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 193	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 027	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 122	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - Simulación APX (Examen 3)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 014) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 026	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 041	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 057	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 112	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 175	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 015	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 054	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 102	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 127	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 203	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 031	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 044	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 082	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 106	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 126b	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 167	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 173	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 182	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 035	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 064	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
21. QS 078	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 117	☐	☐	☐	☐	☐	☐	☐	☐
23. QS 156	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 036	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 184	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 020	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 058	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 099	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 134	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 199	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 003	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 195	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 011	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 142	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 045	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 194	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 053	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 188	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 069	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 172	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - Simulación APX (Examen 4)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 014) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 009	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 052	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 076	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 111	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 161	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 032	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 088	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 103	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 140	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 211	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 016	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 034	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 074	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 098	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 125	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 168	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 176	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 208	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 049	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 067	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
21. QS 084	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 146	☐	☐	☐	☐	☐	☐	☐	☐
23. QS 183	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 024	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 050	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 023	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 063	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 101	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 130	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 164	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 048	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 100	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 018	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 068	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 007	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 090	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 061	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 193	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 004	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 132	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - Simulación APX (Examen 5)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 014) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 178	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 030	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 009	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 072	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 060	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 088	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 055	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 038	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 181	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 032	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 176	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 031	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 013	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 204	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 066	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 074	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 126b	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 160	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 001	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 117	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
21. QS 064	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 156	☐	☐	☐	☐	☐	☐	☐	☐
23. QS 146	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 210	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 185	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 097	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 148	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 099	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 134	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 144	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 002	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 048	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 142	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 068	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 093	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 028	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 061	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 062	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 005	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 172	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - Simulación APX (Examen 6)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 014) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 110	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 109	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 056	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 161	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 052	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 088	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 095	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 055	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 032	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 102	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 170	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 044	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 182	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 160	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 126	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 066	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 119	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 169	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 078	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 158	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
21. QS 180	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 071	☐	☐	☐	☐	☐	☐	☐	☐
23. QS 183	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 077	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 047	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 148	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 063	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 107	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 020	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 130	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 207	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 003	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 197	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 068	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 194	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 007	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 062	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 053	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 122	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 069	☐	☐	☐	☐	☐	☐	☐	☐



Hoja de Respuestas Interactiva - 1. Fundamentos y Sintaxis de Java (Core Java)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 006) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 006	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 009	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 014	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 017	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 019	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 025	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 026	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 030	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 039	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 040	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 041	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 052	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 056	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 057	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 059	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 060	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 070	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 072	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 075	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 076	☐	☐	☐	☐	☐	☐	☐	☐
21. QS 079	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 089	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
23. QS 092	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 108	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 109	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 110	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 111	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 112	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 113	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 114	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 120	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 124	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 129	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 135	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 138	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 139	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 145	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 153	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 161	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 175	☐	☐	☐	☐	☐	☐	☐	☐
41. QS 178	☐	☐	☐	☐	☐	☐	☐	☐
42. QS 190	☐	☐	☐	☐	☐	☐	☐	☐
43. QS 201	☐	☐	☐	☐	☐	☐	☐	☐



Hoja de Respuestas Interactiva - 2. Programación Orientada a Objetos (OOP)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 010) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 010	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 015	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 021	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 022	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 029	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 032	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 038	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 054	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 055	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 065	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 073	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 081	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 083	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 086	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 088	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 095	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 102	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 103	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 115	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 116	☐	☐	☐	☐	☐	☐	☐	☐
21. QS 118	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 123	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
23. QS 127	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 128	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 131	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 136	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 140	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 141	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 143	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 149	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 150	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 155	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 157	☐	☐	☐	☐	☐	☐	☐	☐
34. QS 166	☐	☐	☐	☐	☐	☐	☐	☐
35. QS 181	☐	☐	☐	☐	☐	☐	☐	☐
36. QS 187	☐	☐	☐	☐	☐	☐	☐	☐
37. QS 189	☐	☐	☐	☐	☐	☐	☐	☐
38. QS 192	☐	☐	☐	☐	☐	☐	☐	☐
39. QS 202	☐	☐	☐	☐	☐	☐	☐	☐
40. QS 203	☐	☐	☐	☐	☐	☐	☐	☐
41. QS 206	☐	☐	☐	☐	☐	☐	☐	☐
42. QS 211	☐	☐	☐	☐	☐	☐	☐	☐
43. QS 212	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - 3. Arquitectura y Ecosistema APX

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 013) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 013	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 016	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 031	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 034	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 042	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 044	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 066	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 074	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 080	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 082	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 098	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 104	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 106	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 119	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 125	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 126	☐	☐	☐	☐	☐	☐	☐	☐
17. QS126b	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
18. QS 137	☐	☐	☐	☐	☐	☐	☐	☐
19. QS 159	☐	☐	☐	☐	☐	☐	☐	☐
20. QS 160	☐	☐	☐	☐	☐	☐	☐	☐
21. QS 165	☐	☐	☐	☐	☐	☐	☐	☐
22. QS 167	☐	☐	☐	☐	☐	☐	☐	☐
23. QS 168	☐	☐	☐	☐	☐	☐	☐	☐
24. QS 169	☐	☐	☐	☐	☐	☐	☐	☐
25. QS 170	☐	☐	☐	☐	☐	☐	☐	☐
26. QS 173	☐	☐	☐	☐	☐	☐	☐	☐
27. QS 174	☐	☐	☐	☐	☐	☐	☐	☐
28. QS 176	☐	☐	☐	☐	☐	☐	☐	☐
29. QS 177	☐	☐	☐	☐	☐	☐	☐	☐
30. QS 179	☐	☐	☐	☐	☐	☐	☐	☐
31. QS 182	☐	☐	☐	☐	☐	☐	☐	☐
32. QS 204	☐	☐	☐	☐	☐	☐	☐	☐
33. QS 208	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - 4. Manejo de Excepciones y Errores

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 001) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 001	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 035	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 049	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 064	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 067	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 071	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 078	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 084	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
9. QS 117	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 133	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 146	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 156	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 158	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 163	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 180	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 183	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - 5. Herramientas de Construcción y Gestión (Maven y JDK)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 008) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 008	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 024	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 036	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 037	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 047	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 050	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 077	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 091	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 105	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
10. QS 121	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 151	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 154	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 184	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 185	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 191	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 196	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 209	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 210	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - 6. Colecciones y Programación Funcional (Java 8+)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 012) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 012	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 020	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 023	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 051	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 058	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 063	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 087	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 097	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 099	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
10. QS 101	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 107	☐	☐	☐	☐	☐	☐	☐	☐
12. QS 130	☐	☐	☐	☐	☐	☐	☐	☐
13. QS 134	☐	☐	☐	☐	☐	☐	☐	☐
14. QS 144	☐	☐	☐	☐	☐	☐	☐	☐
15. QS 148	☐	☐	☐	☐	☐	☐	☐	☐
16. QS 152	☐	☐	☐	☐	☐	☐	☐	☐
17. QS 164	☐	☐	☐	☐	☐	☐	☐	☐
18. QS 199	☐	☐	☐	☐	☐	☐	☐	☐



Hoja de Respuestas Interactiva - 7. DevOps, CI/CD y Control de Versiones

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 002) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 002	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 003	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 048	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 096	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 100	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
6. QS 186	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 195	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 200	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 207	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - 8. Patrones de Diseño y Antipatrones

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 011) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 011	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 018	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 033	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 043	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 046	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
6. QS 068	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 142	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 197	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 205	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - 9. Bases de Datos y Persistencia

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 007) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 007	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 028	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 045	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 090	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
5. QS 093	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 147	☐	☐	☐	☐	☐	☐	☐	☐
7. QS 194	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 198	☐	☐	☐	☐	☐	☐	☐	☐



Hoja de Respuestas Interactiva - 10. Pruebas Unitarias (Testing)

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 053) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 053	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 061	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 062	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
4. QS 188	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 193	☐	☐	☐	☐	☐	☐	☐	☐

Hoja de Respuestas Interactiva - 11. Protocolos, Web y Misceláneos

Instrucciones: Haz clic en las casillas para seleccionar tus respuestas. Puedes seleccionar más de una opción por pregunta si es necesario. Haz clic en el identificador (ej. QS 004) para navegar a la pregunta.

#	A	B	C	D	E	F	G	H
1. QS 004	☐	☐	☐	☐	☐	☐	☐	☐
2. QS 005	☐	☐	☐	☐	☐	☐	☐	☐
3. QS 027	☐	☐	☐	☐	☐	☐	☐	☐
4. QS 069	☐	☐	☐	☐	☐	☐	☐	☐
5. QS 085	☐	☐	☐	☐	☐	☐	☐	☐
6. QS 094	☐	☐	☐	☐	☐	☐	☐	☐

#	A	B	C	D	E	F	G	H
7. QS 122	☐	☐	☐	☐	☐	☐	☐	☐
8. QS 132	☐	☐	☐	☐	☐	☐	☐	☐
9. QS 162	☐	☐	☐	☐	☐	☐	☐	☐
10. QS 171	☐	☐	☐	☐	☐	☐	☐	☐
11. QS 172	☐	☐	☐	☐	☐	☐	☐	☐

